

markovlib & markovlv

Discrete-Time Markov Models for Life Insurance

Technical Reference and User Guide

Michael Koller

Original C++ implementation, 2011 / 2026

Python translation and documentation

May 2026

Version: markovlib.py v3.9.2(corrected Python translation)

Version: markovlv.py vv3.9.2(compiled C++ version via pypi)

Standard library only. No external dependencies.

Reference: M. Koller, *Stochastische Modelle in der Lebensversicherung*, Springer, 2000 (2nd ed. 2012).

Cited throughout as **[Koller]**.

Eulyr Advisory LTD

Feldstrasse 14

CH-8704 Herrliberg

michael.koller@eulyr.com

Contents

1	Introduction	3
2	Theory: Discrete-Time Markov Models in Life Insurance	3
2.1	The State Process	3
2.2	Chapman–Kolmogorov Equations	3
2.3	Cash Flows	4
2.4	Thiele’s Difference Equation	4
2.5	Technical Premium Decomposition	5
3	Overview	5
4	Global Functions	7
5	MARKOVLV	7
6	ANNUITYLV	13
7	ANNUITYLV2	16
8	ANNUITYNLV — N-Life Annuity	20
9	CAPITALLV	22
10	MULTDECRLV — Multiple-Decrement Model	26
11	CAPITAL2LV — Two-Life Capital Insurance	33
12	CAPITALNLV — N-Life Capital Insurance	40
13	WIDDOWLV	43
14	DISABILITYLV V0.62	46
15	WIDDOW2SPEC — Two-Life Pension with Guarantee	52
16	Premium Decomposition	61
17	Reserve Distribution (CDF)	62
17.1	Backward recursion (Theorem 5.5.1)	63
17.2	Value grid and bucket discretisation	63
17.3	Support bounds and automatic grid sizing	63
17.4	Consistency check	64
17.5	Example: <code>ex_distr.py</code>	64
18	TABLESERVER	68
19	TSREF — Multi-Column Table Server	72
20	REF_DAV97I — DAV 1997 I Disability Tariff	75

21 REF_DAV22I — DAV 2021 I Disability Tariff	79
22 Static Demographic Data Headers	83
23 JSON Utility Scripts	83
24 Regression Test: <code>reference_dav.py</code>	94
25 License Module System (<code>omarkov_v2S2</code>)	94
26 Build and Installation	97
27 Other Language Implementations	99

1 Introduction

`markovlib` is a pure-Python translation of the C++ actuarial library *omarkov*, originally developed by Michael Koller (2011, revised 2026). The latter one has the same interface and can be installed via `pip install markovlv`. In that case replace `import markovlib` by `import markovlv`. After installation it is recommended to run the bundled integrity check (Section 26) to verify correct results on your platform. It provides a complete toolkit for pricing and reserving life-insurance products modelled as discrete-time Markov chains. The library uses only the Python standard library; no third-party packages are required.

The library covers:

- A generic multi-state Markov engine (`MARKOVLV`) implementing Thiele's difference equation, Chapman–Kolmogorov forward recursion, and technical premium decomposition.
- Special-purpose subclasses for the most common actuarial products: endowments (`CAPITALLV`), single-life annuities (`ANNUITYLV`), two-life annuities (`ANNUITYLV2`), and widow pensions (`WIDDOVLV`), etc.
- Reference Implementation of published Tables such as DAV1997i and DAV2022i.
- Two table server (`TABLESERVER` and `TSREF`) backed by `tables.json` with actuarial tables.

All theory in this document follows the notation and treatment of **[Koller]**, to which the reader is referred for proofs and extended discussion.

2 Theory: Discrete-Time Markov Models in Life Insurance

2.1 The State Process

Let time be measured in integers $t = 0, 1, 2, \dots$. We model an insured life (or a group of lives) by a *finite-state Markov chain* $\{I_t\}_{t \geq 0}$ taking values in the state space $\mathcal{S} = \{0, 1, \dots, n-1\}$.

Definition 2.1 (Markov property). $\{I_t\}$ is a Markov chain if for all t and all states i, j ,

$$\Pr(I_{t+1} = j \mid I_t = i, I_{t-1}, \dots, I_0) = \Pr(I_{t+1} = j \mid I_t = i) =: p_{ij}(t).$$

The numbers $p_{ij}(t) \geq 0$ are the *one-step transition probabilities* at time t . They satisfy $\sum_j p_{ij}(t) = 1$ for every i . In insurance models typical states are:

- **0 = alive / healthy** (or *active*)
- **1 = dead** (absorbing)
- Additional states for disability, guarantee periods, etc.

2.2 Chapman–Kolmogorov Equations

The k -step transition probabilities are obtained by the *Chapman–Kolmogorov* equation:

$$p_{ij}^{(k)}(s, t) := \Pr(I_{X_{s+k}=j} \mid I_{X_s=i}) = \sum_{\ell \in \mathcal{S}} p_{i\ell}(s) p_{\ell j}^{(k-1)}(s+1, t), \quad k \geq 1, \quad (1)$$

with $p_{ij}^{(0)}(s, s) = \mathbf{1}_{X_{i=j}}$. In matrix notation, with $\mathbf{P}(t) = (p_{ij}(t)) \in \mathcal{M}(\mathbb{R}^n)$:

$$\mathbf{P}^{(k)}(s, s+k) = \mathbf{P}(s) \mathbf{P}(s+1) \cdots \mathbf{P}(s+k-1),$$

where the multiplication in this context is the matrix multiplication in $\mathcal{M}(\mathbb{R}^n)$. The forward recursion propagates state distributions and is the basis of the cash-flow computation in `_compute_cf`.

2.3 Cash Flows

We distinguish two types of benefit payment:

Definition 2.2 (Pre- and post-transition benefits).

- $a_i^{\text{pre}}(t)$: benefit paid at the *beginning* of period $[t, t + 1)$ while in state i (before any transition).
- $a_{ij}^{\text{post}}(t)$: benefit paid at the *end* of period $[t, t + 1)$ conditional on the transition $I_t = i \rightarrow I_{t+1} = j$.

The *expected cash flow* conditional on being in state i_0 at time s is

$$\mathbb{E}[CF(t) \cdot \mathbf{1}_{X_t=j} \mid I_{X_s} = i_0] = p_{i_0j}(s, t) \left[a_j^{\text{pre}}(t) + \sum_k p_{jk}(t) a_{jk}^{\text{post}}(t) \cdot v_i(t) \right], \quad (2)$$

where $v_i(t)$ is the one-period discount factor in state i at time t . Summing over j gives the total expected (discounted) cash flow at t .

We note that the total expected cash flow at time t can be calculated as the sum of the above as follows:

$$\mathbb{E}[CF(t) \mid I_{X_s} = i_0] = \sum_j \left(p_{i_0j}(s, t) \left[a_j^{\text{pre}}(t) + \sum_k p_{jk}(t) a_{jk}^{\text{post}}(t) \cdot v_i(t) \right] \right), \quad (3)$$

2.4 Thiele's Difference Equation

The *prospective reserve* (mathematical provision) in state i at time t is the expected present value of all future net obligations:

$$V_i(t) = \mathbb{E} \left[\sum_{\tau \geq t} (\text{discounted net benefit at } \tau) \mid I_t = i \right].$$

Theorem 2.3 (Thiele's difference equation, discrete time – [Koller, Ch. 4]). The reserves satisfy the backward recursion: $V_i(T) = 0$ for all i (terminal boundary condition at the end of the contract), and for $t = T - 1, T - 2, \dots, s$:

$$V_i(t) = a_i^{\text{pre}}(t) + \sum_{j \in \mathcal{S}} p_{ij}(t) v_i(t) \left[a_{ij}^{\text{post}}(t) + V_j(t + 1) \right]. \quad (4)$$

Remark 2.4. The boundary condition implemented in `markovlib` is $V_i(\text{start_time}) = 0$ for all i , corresponding to $T = \text{start_time}$ (typically $\omega + 1$, where ω is the maximum age in the mortality table).

Equation (4) is the exact recursion implemented in `MARKOV.LV._compute_dk`. The key steps are:

1. Initialise $V_i(T) = 0$.
2. Loop $t = T - 1$ down to `stop_time`:
3. For each state i : accumulate $a_i^{\text{pre}}(t)$ plus the weighted sum $\sum_j p_{ij}(t) v_i(t) [a_{ij}^{\text{post}}(t) + V_j(t + 1)]$.

2.5 Technical Premium Decomposition

The reserve can be decomposed (see [Koller, Ch. 5]) as:

$$\text{Savings premium: } SP_i(t) = v_i(t) V_{\sigma(i)}(t + 1) - V_i(t), \quad (5)$$

$$\text{Risk premium: } RP_i(t) = \sum_{j \neq \sigma(i)} v_i(t) p_{ij}(t) [V_j(t + 1) - V_{\sigma(i)}(t + 1) + a_{ij}^{\text{post}}(t)], \quad (6)$$

$$\text{Regular payment: } \text{RegP}_i(t) = a_i^{\text{pre}}(t) + a_{i, \sigma(i)}^{\text{post}}(t), \quad (7)$$

where $\sigma(i)$ is the designated *successor state* of state i (set via `lSetFolgezustand`).

3 Overview

MarkovLV is a compiled Python extension (C++20 back-end, SWIG wrapper) for actuarial present-value calculations based on discrete-time Markov chains with time-inhomogeneous transition matrices. All six public classes share a common backward-recursion engine (`MARKOVLV`) and expose a consistent Python interface.

Import

```
import markovlv as ml
```

The shared library `_markovlv.cpython-3xx-darwin.so` (or the platform-equivalent) must be on `sys.path`. The module file `markovlv.py` sits in the same directory.

License loading

Before any result method (`dGetDK`, `dGetCF`, ...) is called, a valid license string must be supplied. The license is stored in a plain text file; the caller reads the file and passes the *string* to `iSetLicense`:

```
lic_txt = open("/path/to/michael_2039.lic").read().strip()
rc = ml.iSetLicense(lic_txt) # returns 1 on success, 0 on failure
```

Mortality / actuarial tables

Standard actuarial tables (Swiss annuity, endowment, disability tables) are bundled in a JSON registry file. Load the entire file once at application startup:

```
ml.iAddTables("/path/to/tables.json")
```

After loading, table identifiers such as `"CH-QX-ERF-1995"` or `"CH-QX-EKM-1995"` may be passed to class-level `iSetTable` methods.

Naming conventions

Prefix	Meaning
d	double parameter / return value
i / l	int / long parameter
v	void return (setter with side effects)
b	boolean parameter
pc	const char* (C-string)
Set...	Assigns a value and marks the cache invalid
Get...	Triggers backward recursion (if needed) and returns a result
vSet...	Void setter (no meaningful return value)
dSet...	Setter that also returns the stored value

Shared example bootstrap: `ex_setup.py`

Every worked example in the `Examples/` directory starts with

```
import ex_setup    # sets sys.path, loads license and tables.json
```

```
1 """
2 ex_setup.py  --  shared bootstrap for all Manual/Examples scripts.
3 Import this at the top of every example script.
4
5 Loads markovlv from the MarkovLib root directory (where setup.py,
6 markovlv.py, and _markovlv.so all live). Tables are loaded from
7 CPP20/Test/tables.json (their canonical location).
8 """
9 import os
10 import sys
11
12 import markovlv as ml
13
14 # -- License
15 -----
16 _LIC_PATH = "/Users/michaelkoller/Documents/michael_2039.lic"
17 _lic_txt = open(_LIC_PATH).read().strip()
18 if ml.iSetLicense(_lic_txt) != 1:
19     raise RuntimeError(f"License rejected ({_LIC_PATH})")
20
21 # -- Standard actuarial tables
22 -----
23 _TABLES = "/Users/michaelkoller/Eulyr/Repo/tables.json"
24 n = ml.iAddTables(_TABLES)
25 if n == 0:
26     raise RuntimeError(f"No tables loaded from {_TABLES}")
27
28 print(ml.__file__)
```

Listing 1: `ex_setup.py` — shared bootstrap

4 Global Functions

The following free functions are available at module level:

Signature	Description
<code>iSetLicense(str) -> int</code>	Validate and activate the license string. Returns 1 on success, 0 if the string is invalid or expired. Must be called before any <code>dGetDK/dGetCF</code> call.
<code>iAddTables(path: str) -> int</code>	Load a JSON actuarial table registry from path. Returns the number of tables successfully loaded. May be called multiple times to add further registries.
<code>iAddTsrefTables(path: str) -> int</code>	Load a TSREF multi-column disability table registry (e.g. <code>premv3.json</code>). Returns the number of tables loaded.
<code>iLicenseValid() -> int</code>	Returns 1 if a valid license is currently active, 0 otherwise.
<code>pcLicenseInfo() -> str</code>	Returns a human-readable summary of the active license: customer name, expiry date, and licensed modules.
<code>iLicenseAllowsDK() -> int</code>	Returns 1 if the DK, ANNMOD, GLMOD, any CLIENTn, or ALL token is present.
<code>iLicenseAllowsCF() -> int</code>	Returns 1 if the CF, ANNMOD, GLMOD, any CLIENTn, or ALL token is present.
<code>iLicenseAllowsWidow2S() -> int</code>	Returns 1 if Widow2S or ALL is present.
<code>iLicenseAllowsREF() -> int</code>	Returns 1 if REF, ANNMOD, GLMOD, any CLIENTn, or ALL is present.
<code>iLicenseAllowsANNMOD() -> int</code>	Returns 1 if ANNMOD or ALL is present.
<code>iLicenseAllowsGLMOD() -> int</code>	Returns 1 if GLMOD or ALL is present.
<code>iLicenseAllowsClient(n) -> int</code>	Returns 1 if CLIENTn or ALL is present ($n = 1 \dots 10$).

5 MARKOVLV

MARKOVLV is the generic discrete-time Markov present-value engine. It can model any finite-state process: death-only (two states), disability (multi-state), or any custom structure the user assembles.

State / time model

- States are numbered $0, 1, \dots, n - 1$. One state may be declared absorbing by mapping its *Folgezustand* to itself.



- Time runs backward from `StartTime` (terminal) to `StopTime` (valuation point). In insurance contexts these are the oldest projected age and the current age, respectively.
- Transition probabilities $p_{ij}(t)$ must sum to 1 across the target states j for each origin state i and each time step t .
- Pre-numerando payments (`dSetPre`) are made at the *start* of each period; post-numerando (`dSetPost`) at the *end*.
- The discount factor `dSetDisc(t, i, j)` is stored per *transition* so stochastic interest can be represented. For deterministic interest set the same value on the diagonal.

Constructor

```
m = ml.MARKOVLV(lMaxTimes, lMaxStates, lNrMoments)
```

Parameter	Meaning
<code>lMaxTimes</code>	Maximum number of time steps (allocates internal arrays)
<code>lMaxStates</code>	Maximum number of states
<code>lNrMoments</code>	Number of distribution moments to compute (1 = mean only)

Method reference

Method	Description
<code>vSetStartTime(t)</code>	Set the terminal time (oldest age / projection horizon).
<code>vSetStopTime(t)</code>	Set the valuation time (entry / current age).
<code>vSetNrStates(n)</code>	Set the number of active states.
<code>lSetFolgezustand(from, to) -> long</code>	Declare the default successor state: if state <code>from</code> is active, its forward default is <code>to</code> . Used to mark absorbing states (<code>to == from</code>).
<code>dSetPij(t, i, j, p) -> double</code>	Set the one-step transition probability from state i to state j at time t . Returns the stored value.
<code>dSetPre(t, i, j, v) -> double</code>	Set the pre-numerando payment made when in state i transitioning toward state j at time t .
<code>dSetPost(t, i, j, v) -> double</code>	Set the post-numerando payment (benefit paid at end of period, on transition from state i to state j at time t).
<code>dSetDisc(t, i, j, v) -> double</code>	Set the discount factor $v(t)$ for the $(i \rightarrow j)$ transition at time t . Typically $v = 1/(1 + i)$ on the diagonal.
<code>dGetDK(t, s, moment) -> double</code>	Trigger backward recursion (if not already done) and return the mathematical reserve at time t in state s . <code>moment=1</code> for mean.

Method	Description
<code>dGetCF(t, s0, s1) -> double</code>	Return the expected cash flow at time t from state $s0$ toward $s1$.
<code>dGetRP(t, s) -> double</code>	Risk premium at time t in state s .
<code>dGetSP(t, s) -> double</code>	Saving premium at time t in state s .
<code>dGetRegP(t, s) -> double</code>	Regular (contractual) payment at time t in state s .
<code>lGetDKCalculated() -> long</code>	Number of moments actually cached by the last <code>dGetDK</code> pass (0 = nothing computed yet, or the cache was invalidated by a subsequent <code>dSetXxx</code> call).
<code>lGetCFCalculated() -> long</code>	Number of initial states whose expected-cash-flow chains are cached by the last <code>dGetCF</code> pass.
<code>vExportJson(filename)</code>	Serialise the fully-built model — inputs (<code>pij</code> , <code>disc</code> , <code>pre</code> , <code>post</code>) and any already-computed <code>dk/cf</code> results — to a self-contained JSON file. See Section 5 below.

Reserve distribution (CDF) — see §17

<code>distrBounds() -> (lo, hi)</code>	Conservative support bracket ($V_{\min}, V_{\max}$); returns a tuple.
<code>vSetDistrGrid(lo, hi, n)</code>	Define the value grid $[lo, hi)$ with n equal-width buckets.
<code>vAutoDistrGrid(n)</code>	Auto-size the grid from <code>distrBounds</code> with guard classes. Default $n = 2000$.
<code>vSetDistrBoundary(s, thr)</code>	Set the terminal Heaviside threshold for state s (default 0).
<code>dGetDKDistr(t, s, x) -> double</code>	$P[V(t) < x \mid X_t = s]$ — the CDF at value x .
<code>dGetDKQuantile(t, s, p) -> double</code>	Inverse CDF: smallest grid value with $P \geq p$ (VaR).
<code>dGetDKDistrBucket(t, s, b) -> double</code>	Raw CDF value at bucket index b .
<code>lBucketNr(x) -> long</code>	Map a value to its nearest bucket index.
<code>dValueOfBucket(b) -> double</code>	Map a bucket index to its representative value.
<code>dGetDistrLow() / dGetDistrHigh()</code>	Current grid bounds.
<code>lGetNrBuckets() / dGetBucketWidth()</code>	Current grid geometry.
<code>lGetDKDistrCalculated() -> long</code>	Whether the CDF grid has been computed (1) or is stale (0).

JSON Export

Every model built on top of **MARKOVLV** — including **ANNUITYLV**, **CAPITALLV** and the other actuarial classes, since each of them wraps an internal **MARKOVLV** — can write itself out with the single call `vExportJson(filename)`. The resulting file is a self-contained snapshot: model metadata (`StartTime`, `StopTime`, `NrStates`, ...), every non-zero input (p_{ij} , discount factors, pre-/post-numerando payments), and — once `dGetDK/dGetCF` have been called — the resulting reserves (`dk`) and expected cash flows (`cf`). Entries with $|v| \leq 10^{-10}$ are dropped to keep the file compact.

```
// C++
m.dGetDK(0, 0, 1);           // compute reserves first ...
vExportJson(m, "term_insurance.json");
```

```
# Python (SWIG or markovlib.py)
m.dGetDK(0, 0, 1)
ml.vExportJson(m, "term_insurance.json")
```

Remark 5.1 (Call `dGetDK/dGetCF` *before* exporting). `vExportJson` never triggers a fresh computation: it writes out only the values already cached, bounded by the counts returned from `lGetDKCalculated()` and `lGetCFCalculated()`. If the model has not been solved yet, the `dk` and `cf` blocks of the JSON file come out empty while the inputs are still exported in full. This “export what is cached, never compute” rule holds identically for the C++ engine and for both `OtherLanguages` ports (Python and Rust); see Section 27 and the dedicated *JSON Export & Import* manual (`json_export.pdf`) for the full schema, the `importjson.py` reader, and worked examples.

Example: `ex_markovlv.py`

```
1  """
2  ex_markovlv.py  --  MARKOVLV: generic 2-state term-insurance examples.
3
4  Model:  state 0 = alive,  state 1 = dead (absorbing)
5  Policy: n-year term, annual premium P, death benefit S.
6  Net reserve at issue  V_0  should equal  EPV(benefits) - EPV(premiums).
7  """
8  import ex_setup                      # license + tables
9  import markovlv as ml
10
11  SEP = "-" * 56
12
13  # -- helper
14  -----
15  def term_insurance(q, i, n, S, P):
16      """Return net reserve at t=0 (alive) for a simple term policy."""
17      disc = 1.0 / (1.0 + i)
18      m = ml.MARKOVLV(n + 2, 2, 1)
19      m.vSetNrStates(2)
20      m.vSetStopTime(0)
21      m.vSetStartTime(n)
22      m.lSetFolgezustand(0, 0)
23      m.lSetFolgezustand(1, 1)
24      for t in range(n):
25          m.dSetPij(t, 0, 0, 1.0 - q)
```

```

25     m.dSetPij(t, 0, 1, q)
26     m.dSetPij(t, 1, 1, 1.0)
27     m.dSetDisc(t, 0, 0, disc)
28     m.dSetDisc(t, 1, 1, disc)
29     m.dSetPre (t, 0, 0, -P)           # premium paid at start of year (
negative = inflow)
30     m.dSetPost(t, 0, 1, S)           # death benefit at end of year
31     a = m.dGetDK(0, 0, 1)
32     ml.vExportJson(m, "test.json")
33     return m.dGetDK(0, 0, 1)
34
35 # -- Example 1: baseline
-----
36 print("MARKOVLV -- 5-year term insurance, baseline")
37 print(SEP)
38 q0, i0, n0, S0, P0 = 0.01, 0.03, 5, 100_000, 500
39 V = term_insurance(q0, i0, n0, S0, P0)
40 print(f"  q={q0:.3f}  i={i0:.3f}  n={n0}  S={S0:,.0f}  P={P0}")
41 print(f"  Net reserve at issue  V_0 = {V:,.2f}")
42 print()
43
44 # -- Example 2: vary mortality q
-----
45 print("Sensitivity -- vary annual mortality q  (i=3%, n=5, S=100k, P=500)
")
46 print(SEP)
47 print(f"  {'q':>6}  {'V_0':>12}")
48 for q in [0.002, 0.005, 0.010, 0.020, 0.040]:
49     print(f"  {q:6.3f}  {term_insurance(q, 0.03, 5, 100_000, 500):12.2f}")
50 print()
51
52 # -- Example 3: vary interest i
-----
53 print("Sensitivity -- vary interest rate i  (q=1%, n=5, S=100k, P=500)")
54 print(SEP)
55 print(f"  {'i':>6}  {'V_0':>12}")
56 for i in [0.01, 0.02, 0.03, 0.04, 0.05]:
57     print(f"  {i:6.3f}  {term_insurance(0.01, i, 5, 100_000, 500):12.2f}")
58 print()
59
60 # -- Example 4: net premium by term n
-----
61 # Net premium P* satisfies EPV(benefits) = P* * EPV(unit-premium annuity)
62 # EPV of unit-premium annuity = term_insurance(q,i,n, S=0, P=1)
63 # -> dSetPre = -1 per year -> V_0 = -EPV(annuity-due) -> multiply by -1
64 print("Net premium by term  (q=1%, i=3%, S=100k)")
65 print(SEP)
66 print(f"  {'n':>4}  {'EPV(ben)':>12}  {'EPV(ann-due)':>13}  {'P_net':>8}
{'V_0(check)':>12}")
67 for n in [5, 10, 15, 20, 30]:
68     q = 0.01
69     epv_ben = term_insurance(q, 0.03, n, 100_000, 0) # P=0: pure benefit
EPV
70     epv_ann = -term_insurance(q, 0.03, n, 0, 1)      # P=1: premium
income -> negative V -> negate
71     P_net = epv_ben / epv_ann if epv_ann > 0 else 0
72     V_check = term_insurance(q, 0.03, n, 100_000, P_net) # should ~= 0

```

```

73     print(f"   {n:4d}   {epv_ben:12.2f}   {epv_ann:13.5f}   {P_net:8.2f}   {
       V_check:12.6f}")
74 print()
75
76 # -- Example 5: reserve progression over time
       -----
77 print("Reserve progression   (q=1%, i=3%, n=10, S=100k, P=600)")
78 print(SEP)
79 disc = 1.0 / 1.03
80 q, n, S, P = 0.01, 10, 100_000, 600
81 m = ml.MARKOVLV(n + 2, 2, 1)
82 m.vSetNrStates(2)
83 m.vSetStopTime(0)
84 m.vSetStartTime(n)
85 m.lSetFolgezustand(0, 0); m.lSetFolgezustand(1, 1)
86 for t in range(n):
87     m.dSetPij(t, 0, 0, 1.0 - q); m.dSetPij(t, 0, 1, q); m.dSetPij(t, 1, 1,
       1.0)
88     m.dSetDisc(t, 0, 0, disc);      m.dSetDisc(t, 1, 1, disc)
89     m.dSetPre (t, 0, 0, -P);         m.dSetPost(t, 0, 1, S)
90 print(f"   {'t':>3}   {'V_t(alive)':>14}   {'RP_t':>10}   {'SP_t':>10}")
91 for t in range(n + 1):
92     V = m.dGetDK(t, 0, 1)
93     rp = m.dGetRP(t, 0)
94     sp = m.dGetSP(t, 0)
95     print(f"   {t:3d}   {V:14.2f}   {rp:10.2f}   {sp:10.2f}")

```

Listing 2: ex_markovlv.py — generic 2-state term-insurance

Numerical output

MARKOVLV - 5-year term insurance, baseline

```

-----
q=0.010  i=0.030  n=5  S=100,000  P=500
Net reserve at issue  V_0 = 2,178.47

```

Net premium by term (q=1%, i=3%, S=100k)

```

-----
      n      EPV(ben)      EPV(ann-due)      P_net      V_0 (check)
      5      4491.69      4.62644      970.87      -0.000000
     10      8176.37      8.42166      970.87      0.000000
     20     13678.62     14.08898      970.87      0.000000

```

Reserve progression (q=1%, i=3%, n=10, S=100k, P=600)

```

-----
      t      V_t(alive)      RP_t      SP_t
      0      3123.37      943.07     -343.07
      5      1715.83      957.29     -357.29
     10         0.00         0.00         0.00

```

6 ANNUITYLV

ANNUITYLV prices and reserves single-life life annuities. Internally it reduces to a two-state (alive / dead) MARKOVLV with mortality loaded either directly via `dSetQx` or from a named table.

Improvement factors (cohort projection) may be applied via `dSetFx` and calendar-year helpers `dSetBaseYear` / `dSetActualYear`.

Constructor

```
ann = ml.ANNUITYLV() # defaults: 2500 steps, pre=1
ann = ml.ANNUITYLV(lMaxTimes, dPre) # explicit overrides
```

Method reference

Method	Description
<code>iSetTable(id: str) -> int</code>	Load mortality from the named table in the registry. Returns 1 on success.
<code>vSetStartTime(t)</code>	Oldest age (projection horizon, typically 121).
<code>vSetStopTime(t)</code>	Valuation age.
<code>vSetSAge(t)</code>	Payment start age. Enables deferred annuities.
<code>vSetG(t)</code>	Guarantee period in years.
<code>dSetQx(age, q) -> double</code>	Set q_x at the given age.
<code>dSetFx(age, f) -> double</code>	Set annual improvement factor f_x at age.
<code>dSetBaseYear(y) -> double</code>	Calendar base year for improvement projection.
<code>dSetActualYear(y) -> double</code>	Actual valuation calendar year.
<code>dSetDisc(t, v) -> double</code>	Discount factor $v = 1/(1+i)$ at time step t .
<code>dSetPre(alpha) -> double</code>	Pre-numerando fraction $\alpha \in [0, 1]$. $\alpha = 1$ = annuity-due; $\alpha = 0$ = annuity-immediate.
<code>vLeistReset()</code>	Reset benefit vector to unit benefit at every age.
<code>vSetLeistLinear(t0, t1, b0, incr)</code>	Linearly varying benefit from age t_0 to t_1 .
<code>vSetLeistExp(t0, t1, b0, f)</code>	Exponentially growing benefit: $b(t_0 + k) = b_0 \cdot f^k$.
<code>dGetDK(age) -> double</code>	Backward-recursion present value at the given valuation age.
<code>dGetCF(age) -> double</code>	Expected cash flow at the given age.
<code>dGetRP(age) -> double</code>	Risk premium at the given age.
<code>dGetSP(age) -> double</code>	Saving premium at the given age.

Method	Description
<code>dGetRegP(age) -> double</code>	Regular (expected) cash flow at the given age.
<code>dGetQx(age, year) -> double</code>	Return q_x (possibly adjusted for improvements).
<code>dGetTpx(age) -> double</code>	Accumulated survival probability.

Example: `ex_annuitylv.py`

```

1  """
2  ex_annuitylv.py  --  ANNUITYLV: single-life annuity examples.
3
4  Two approaches:
5  A) mortality set directly via dSetQx (no table needed)
6  B) loaded from table CH-QX-ERF-1995 (Swiss female annuity)
7  """
8  import ex_setup
9  import markovlv as ml
10
11  SEP = "-" * 60
12
13  # -- Helper: simple Makeham mortality
14  -----
15  def makeham_qx(age, A=0.0007, B=0.00005, c=1.09):
16      """Makeham:  $q_x = 1 - \exp(-(A + B \cdot c^x))$  (approx for annual)."""
17      import math
18      mu = A + B * (c ** age)
19      return 1.0 - math.exp(-mu)
20
21  def fill_qx(ann, age_start=0, age_end=120):
22      for x in range(age_start, age_end + 1):
23          ann.dSetQx(x, makeham_qx(x))
24
25  def fill_disc(ann, i, n=2500):
26      v = 1.0 / (1.0 + i)
27      for t in range(n):
28          ann.dSetDisc(t, v)
29
30  # -- Example 1: EPV of unit annuity-due at age 65
31  -----
32  print("ANNUITYLV  --  Unit annuity-due at age 65 (Makeham mortality)")
33  print(SEP)
34  ann = ml.ANNUITYLV()
35  fill_qx(ann)
36  ann.vSetStopTime(65)
37  ann.vSetStartTime(121)
38  ann.vSetSAge(65)
39  ann.vSetG(0)
40  fill_disc(ann, 0.03)
41  ann.dSetPre(1.0)
42  dk = ann.dGetDK(65)
43  # full pre-numerando
44  print(f"  Makeham A=0.0007, B=5e-5, c=1.09, i=3%")
45  print(f"  EPV annuity-due at 65:  ae_65 = {dk:.5f}")
46  print()

```

```

45 # -- Example 2: vary valuation age
46 -----
47 print("Sensitivity -- vary valuation age (i=3%, Makeham)")
48 print(SEP)
49 print(f" {'age':>4} {'ae_x (pre)':>12} {'a_x (post)':>12}")
50 for age in [50, 55, 60, 65, 70, 75]:
51     a_pre = ml.ANNUITYLV(); fill_qx(a_pre); a_pre.vSetStopTime(age);
52     a_pre.vSetStartTime(121); a_pre.vSetSAge(age); fill_disc(a_pre, 0.03)
53     ; a_pre.dSetPre(1.0)
54     a_post = ml.ANNUITYLV(); fill_qx(a_post); a_post.vSetStopTime(age);
55     a_post.vSetStartTime(121); a_post.vSetSAge(age); fill_disc(a_post, 0.03)
56     ; a_post.dSetPre(0.0)
57     print(f" {age:4d} {a_pre.dGetDK(age):12.5f} {a_post.dGetDK(age):12.5f}")
58 print()
59
60 # -- Example 3: vary interest rate
61 -----
62 print("Sensitivity -- vary interest rate i (age=65, Makeham)")
63 print(SEP)
64 print(f" {'i':>5} {'ae_65':>12}")
65 for i in [0.005, 0.01, 0.02, 0.03, 0.04, 0.05]:
66     a = ml.ANNUITYLV(); fill_qx(a); a.vSetStopTime(65); a.vSetStartTime
67     (121); a.vSetSAge(65); fill_disc(a, i); a.dSetPre(1.0)
68     print(f" {i:5.3f} {a.dGetDK(65):12.5f}")
69 print()
70
71 # -- Example 4: deferred annuity (payments from 65, valued at 40)
72 -----
73 print("Deferred annuity -- valued at age 40, payments start at 65")
74 print(SEP)
75 a_def = ml.ANNUITYLV()
76 fill_qx(a_def)
77 a_def.vSetStopTime(40)
78 a_def.vSetStartTime(121)
79 a_def.vSetSAge(65) # deferred: no payments before 65
80 fill_disc(a_def, 0.03)
81 a_def.dSetPre(1.0)
82 print(f" EPV at 40 of deferred annuity (start 65, i=3%): {a_def.dGetDK(40):.5f}")
83 print()
84
85 # -- Example 5: escalating benefit (2% p.a.)
86 -----
87 print("Escalating benefit -- 2% p.a. CPI (age 65, i=3%)")
88 print(SEP)
89 a_esc = ml.ANNUITYLV()
90 fill_qx(a_esc)
91 a_esc.vSetStopTime(65); a_esc.vSetStartTime(121); a_esc.vSetSAge(65)
92 fill_disc(a_esc, 0.03); a_esc.dSetPre(1.0)
93 a_esc.vSetLeistExp(65, 120, 1.0, 1.02)
94 dk_esc = a_esc.dGetDK(65)
95 print(f" Flat benefit ae_65 = {dk:.5f}")
96 print(f" 2% escalation ae_65 = {dk_esc:.5f} (ratio {dk_esc/dk:.4f})")
97 print()
98
99 # -- Example 6: Swiss female table CH-QX-ERF-1995
100 -----

```



```

91 print("Swiss female annuity -- table CH-QX-ERF-1995 (i=3%)")
92 print(SEP)
93 a_ch = ml.ANNUITYLV()
94 a_ch.iSetTable("CH-QX-ERF-1995")
95 a_ch.vSetStopTime(65); a_ch.vSetStartTime(121); a_ch.vSetSAge(65); a_ch.
    vSetG(0)
96 fill_disc(a_ch, 0.03); a_ch.dSetPre(1.0)
97 print(f" ae_65 (CH-QX-ERF-1995, i=3%) = {a_ch.dGetDK(65):.5f}")
98 print()
99
100 # -- Example 7: cash-flow profile
    -----
101 print("Cash-flow profile (CH-QX-ERF-1995, age 65, i=3%)")
102 print(SEP)
103 print(f" {'age':>4} {'CF':>10} {'RP':>10} {'SP':>10}")
104 for age in range(65, 86):
105     print(f" {age:4d} {a_ch.dGetCF(age):10.6f} {a_ch.dGetRP(age):10.6f}
        {a_ch.dGetSP(age):10.6f}")

```

Listing 3: ex_annuitylv.py — single-life annuity examples

Numerical output

```

ANNUITYLV -- Unit annuity-due at age 65 (Makeham mortality)
-----
    EPV annuity-due at 65:  a_65 = 14.55719

Sensitivity -- vary valuation age (i=3%, Makeham)
-----
    age      a_x (pre)      a_x (post)
    50      20.10551      19.10551
    65      14.55719      13.55719

Swiss female annuity -- table CH-QX-ERF-1995 (i=3%)
-----
    a_65 (CH-QX-ERF-1995, i=3%) = 15.51606

```

7 ANNUITYLV2

ANNUITYLV2 extends the single-life engine to two lives (insured x and survivor y). The four states are:

State	Meaning
0	Both lives alive
1	y alive, x dead (widow/widower reversionary)
2	x alive, y dead
3	Both lives dead (absorbing)

Call order note for dSetBenefit and dSetPre

Important: `dSetPre` must be called *after* all `dSetBenefit` calls. `dSetPre` reads the current per-state benefit totals and redistributes them between pre- and post-numerando; if benefits have not yet been set the pre/post split will be wrong.

Constructor

```
a2 = ml.ANNUITYLV2()
a2 = ml.ANNUITYLV2(lMaxTimes, dPre)
```

Method reference

Method	Description
<code>iSetTable1(id: str) -> int</code>	Load life-1 (x , typically male) mortality from the named table.
<code>iSetTable2(id: str) -> int</code>	Load life-2 (y , typically female) mortality from the named table.
<code>vSetStartTime(t)</code>	Oldest projected age.
<code>vSetStopTime(t)</code>	Valuation age.
<code>dSetQx1(age, q) -> double</code>	Set q_x for life 1 at the given age.
<code>dSetQx2(age, q) -> double</code>	Set q_x for life 2 at the given age.
<code>dSetDisc(t, v) -> double</code>	Discount factor at time step t .
<code>dSetY_Minus_X(yAge, xAge) -> double</code>	Set the age difference $y_{\text{age}} - x_{\text{age}}$ at the valuation date.
<code>dSetBenefit(state, b) -> double</code>	Set the benefit fraction for state $s \in \{0, 1, 2\}$. Call before dSetPre.
<code>dSetPre(alpha) -> double</code>	Pre-numerando fraction. Must be called after all dSetBenefit calls.
<code>dGetDK(age, state) -> double</code>	Present value at the given age conditioned on the given state.
<code>dGetRP(age, state) -> double</code>	Risk premium.
<code>dGetSP(age, state) -> double</code>	Saving premium.
<code>dGetRegP(age, state) -> double</code>	Regular (expected) cash flow.

Example: ex_annuitylv2.py

```
1 """
2 ex_annuitylv2.py  --  ANNUITYLV2: joint-life annuity examples.
3
```

```

4 States: 0 = both alive, 1 = y alive (x dead), 2 = x alive (y dead), 3 =
      both dead
5 Benefit fractions are set per state.
6 """
7 import ex_setup
8 import markovlv as ml
9 import math
10
11 SEP = "-" * 62
12
13 def gompertz_qx(age, m=85.0, sigma=9.5):
14     """Gompertz:  $q_x = 1 - \exp(-\exp((x-m)/\sigma))$  (annual approx)."""
15     return 1.0 - math.exp(-math.exp((age - m) / sigma))
16
17 def fill_qx1(a2, m=82.0, sigma=9.0): # male (lower life expectancy)
18     for x in range(130):
19         a2.dSetQx1(x, gompertz_qx(x, m, sigma))
20
21 def fill_qx2(a2, m=87.0, sigma=9.5): # female
22     for x in range(130):
23         a2.dSetQx2(x, gompertz_qx(x, m, sigma))
24
25 def fill_disc(a2, i, n=2500):
26     for t in range(n):
27         a2.dSetDisc(t, 1.0 / (1.0 + i))
28
29 # -- Example 1: baseline joint annuity
30 -----
31 print("ANNUITYLV2 -- Joint-life annuity, both aged 65/62 (i=3%)")
32 print(SEP)
33 a2 = ml.ANNUITYLV2()
34 fill_qx1(a2); fill_qx2(a2)
35 a2.vSetStopTime(65); a2.vSetStartTime(121)
36 a2.vSetSAge1(65); a2.vSetSAge2(65)
37 a2.dSetY_Minus_X(62, 65) # y (female) is 62 when x (male) is 65
38 a2.dSetBenefit(0, 1.0) # full benefit while both alive
39 a2.dSetBenefit(1, 0.6) # 60% reversionary to widow
40 a2.dSetBenefit(2, 0.6) # 60% to widower
41 fill_disc(a2, 0.03); a2.dSetPre(1.0)
42 print(f" Gompertz male m=82/s=9, female m=87/s=9.5, i=3%")
43 print(f" DK(65, both alive) = {a2.dGetDK(65, 0):.5f}")
44 print(f" DK(65, y alive) = {a2.dGetDK(65, 1):.5f}")
45 print(f" DK(65, x alive) = {a2.dGetDK(65, 2):.5f}")
46 print()
47 # -- Example 2: vary reversionary fraction (Swiss tables -- most reliable
48 ) -----
49 print("Sensitivity -- vary reversionary fraction (Swiss tables, ages
50 65/62, i=3%)")
51 print(SEP)
52 print(f" {'rev%':>6} {'DK(both)':>12} {'DK(widow)':>12} {'DK(widower)
53 ':>12}")
54 for rev in [0.0, 0.40, 0.60, 0.75, 1.00]:
55     a = ml.ANNUITYLV2()
56     a.iSetTable1("CH-QX-ERM-1995"); a.iSetTable2("CH-QX-ERF-1995")
57     a.vSetStopTime(65); a.vSetStartTime(121)
58     a.vSetSAge1(65); a.vSetSAge2(65)
59     a.dSetY_Minus_X(62, 65)

```

```

57     a.dSetBenefit(0, 1.0); a.dSetBenefit(1, rev); a.dSetBenefit(2, rev)
58     fill_disc(a, 0.03); a.dSetPre(1.0)
59     print(f"    {rev:6.2f}    {a.dGetDK(65,0):12.5f}    {a.dGetDK(65,1):12.5f}    {
60         a.dGetDK(65,2):12.5f}")
61 print()
62 # -- Example 3: vary age gap y-x
63 -----
64 print("Sensitivity -- vary age gap (y_age - x_age)  (x=65, i=3%, rev=60%)
65 ")
66 print(SEP)
67 print(f"    {'y age':>6}    {'gap':>4}    {'DK(65,both)':>14}")
68 for y_age in [60, 62, 65, 68, 70]:
69     a = ml.ANNUITYLV2()
70     fill_qx1(a); fill_qx2(a)
71     a.vSetStopTime(65); a.vSetStartTime(121)
72     a.vSetSAge1(65); a.vSetSAge2(65)
73     a.dSetY_Minus_X(y_age, 65)
74     a.dSetBenefit(0, 1.0); a.dSetBenefit(1, 0.6); a.dSetBenefit(2, 0.6)
75     fill_disc(a, 0.03); a.dSetPre(1.0)
76     print(f"    {y_age:6d}    {y_age-65:4d}    {a.dGetDK(65,0):14.5f}")
77 print()
78 # -- Example 4: vary interest rate
79 -----
80 print("Sensitivity -- vary interest rate  (ages 65/62, rev=60%)")
81 print(SEP)
82 print(f"    {'i':>5}    {'DK(65,both)':>14}")
83 for i in [0.01, 0.02, 0.03, 0.04, 0.05]:
84     a = ml.ANNUITYLV2()
85     fill_qx1(a); fill_qx2(a)
86     a.vSetStopTime(65); a.vSetStartTime(121)
87     a.vSetSAge1(65); a.vSetSAge2(65)
88     a.dSetY_Minus_X(62, 65)
89     a.dSetBenefit(0, 1.0); a.dSetBenefit(1, 0.6); a.dSetBenefit(2, 0.6)
90     fill_disc(a, i); a.dSetPre(1.0)
91     print(f"    {i:5.3f}    {a.dGetDK(65,0):14.5f}")
92 print()
93 # -- Example 5: Swiss tables
94 -----
95 print("Swiss tables CH-QX-ERM-1995 / CH-QX-ERF-1995  (i=3%, rev=60%)")
96 print(SEP)
97 a_ch = ml.ANNUITYLV2()
98 a_ch.iSetTable1("CH-QX-ERM-1995"); a_ch.iSetTable2("CH-QX-ERF-1995")
99 a_ch.vSetStopTime(65); a_ch.vSetStartTime(121)
100 a_ch.vSetSAge1(65); a_ch.vSetSAge2(65)
101 a_ch.dSetY_Minus_X(62, 65)
102 a_ch.dSetBenefit(0, 1.0); a_ch.dSetBenefit(1, 0.6); a_ch.dSetBenefit(2,
103     0.6)
104 fill_disc(a_ch, 0.03); a_ch.dSetPre(1.0)
105 print(f"    DK(65, both alive) = {a_ch.dGetDK(65,0):.5f}")
106 print(f"    DK(70, y alive)     = {a_ch.dGetDK(70,1):.5f}")
107 print(f"    DK(70, x alive)      = {a_ch.dGetDK(70,2):.5f}")

```

Listing 4: ex_annuitylv2.py — joint-life annuity examples

Numerical output

```

ANNUITYLV2 -- Joint-life annuity, both aged 65/62  (i=3%)
  DK(65, both alive) = 6.19995
  DK(65, y alive)    = 2.75680
  DK(65, x alive)    = 4.17474

Swiss tables CH-QX-ERM-1995 / CH-QX-ERF-1995  (i=3%, rev=60%)
  DK(65, both alive) = 15.91701
  DK(70, y alive)    = 7.03494

```

8 ANNUITYNLV — N-Life Annuity

Overview

ANNUITYNLV generalises **ANNUITYLV2** from two lives to N independent lives. The constructor receives N as its first argument; the class then maintains a state space of 2^N states.

State space

States are encoded as N -character strings:

'*' = alive 't' = dead

State index = binary value of the string with '*' = 0, 't' = 1, leftmost character = most-significant bit.

For $N = 2$ (identical to **ANNUITYLV2**):

String	Index	Meaning
"**"	0	both alive
"*t"	1	life-0 alive, life-1 dead
"t*"	2	life-0 dead, life-1 alive
"tt"	3	both dead (absorbing)

Lives are numbered $0 \dots N-1$ left to right. Life k can be set ℓ years older than life 0 via `dSetAgeOffset(k, ℓ)`.

Constructors

```

1 ANNUITYNLV(int nLives);
2 ANNUITYNLV(int nLives, long lMaxTimes, double dPre);

```

nLives

Number of lives N (1 ... 8 practical limit).

lMaxTimes

Maximum projection horizon (default 250).

dPre Global pre-numerando fraction (0 = fully post, 1 = fully pre; default 1.0).



Mortality API

```

1 int      iSetTable   (int k, char* pcId);           // table for life k
2 double   dSetQx      (int k, long t, double v);    // mortality at age t
3 double   dSetFx      (int k, long t, double v);    // improvement factor
4 double   dSetAgeOffset(int k, long offset);         // life k is offset years
   older than life 0
5 double   dSetBaseYear(long year);
6 double   dSetActualYear(long year);
7 double   dSetDisc     (long t, double v);          // discount factor (t<0 =>
   broadcast)

```

Benefit API

```

1 void      vSetSAge    (const char* strState, long lAge); // deferred start age
2 double    dSetBenefit(const char* strState, double v);  // per-state scalar
3 double    dSetPre     (double dPre);                   // global pre fraction
4 void      vLeistReset();
5 void      vSetLeistLinear(long from, long to, double start, double incr);
6 void      vSetLeistExp (long from, long to, double start, double factor);

```

The annuity benefit for state s at time t is

$$B_s(t) = \underbrace{\text{psymBenefit}(t) \cdot \text{dPre_s}[s]}_{\text{pre}} + \underbrace{\text{psymBenefit}(t) \cdot \text{dPost_s}[s]}_{\text{post}}$$

where $\text{dPre_s}[s] + \text{dPost_s}[s] = \text{dBenefit}[s]$ and $\text{dPre_s}[s] = \text{dPre} \times \text{dBenefit}[s]$.

Result methods (require Widow2S token)

```

1 double    dGetDK      (long t, long state);         // reserve  $V_s(t)$ 
2 double    dGetCF      (long t);                    // expected cash flow
3 double    dGetQx      (int k, long t, long year);   // effective  $q_x$  with
   improvement
4 double    dGetRP      (long t, long state);         // risk premium
5 double    dGetSP      (long t, long state);         // saving premium
6 double    dGetRegP    (long t, long state);         // regular payment

```

State helpers

```

1 long      lStateFromStr(const char* s) const;       // string -> index
2 std::string strFromState(long state) const;         // index -> string
3 int       iGetNLives() const;
4 long      lGetNStates() const;

```

Implementation note: benefit amount storage

$\text{dPre_s}[s]$ and $\text{dPost_s}[s]$ store *amounts* (not fractions): they equal $\text{dPre} \times \text{dBenefit}[s]$ and $(1 - \text{dPre}) \times \text{dBenefit}[s]$ respectively. The recursion therefore uses $B_s(t) = \text{psymBenefit}(t) \times (\text{dPre_s}[s] + \text{dPost_s}[s])$ directly without a further multiplication by $\text{dBenefit}[s]$.

Example: ex_annuitynlv.py

```

1 # -- Example 1: 2-life -- comparison with ANNUITYLV2 -----
   ----
2 print("Example 1 -- ANNUITYNLV(2) vs ANNUITYLV2 (ages 65/62, i=3%)")
3 print("          Gompertz male m=82/s=9.0, female m=87/s=9.5, rev=60%")
4 print(SEP)
5
6 anv2 = make_annuity2()
7
8 # Reference: ANNUITYLV2 with same parameters
9 ref = ml.ANNUITYLV2()
10 for x in range(130):
11     ref.dSetQx1(x, gompertz_qx(x, 82.0, 9.0))
12     ref.dSetQx2(x, gompertz_qx(x, 87.0, 9.5))
13 ref.dSetY_Minus_X(62, 65)
14 ref.vSetStopTime(65); ref.vSetStartTime(130)
15 ref.vSetSAge1(65); ref.vSetSAge2(65)
16 ref.dSetBenefit(0, 1.0); ref.dSetBenefit(1, 0.6); ref.dSetBenefit(2, 0.6)
17 fill_disc(ref, 0.03); ref.dSetPre(1.0)
18
19 print(f"  {'age':>4}  {'ANNUITYNLV(**)' :>18}  {'ANNUITYLV2(0)':>16}  {'
   diff':>10}")
20 for age in [65, 68, 72, 76, 80]:
21     v_nlv = anv2.dGetDK(age, 0)
22     v_ref = ref.dGetDK(age, 0)
23     print(f"  {age:4d}  {v_nlv:18.5f}  {v_ref:16.5f}  {abs(v_nlv-v_ref)
   :10.2e}")
24 print()
25 # Per-state cross-check (ANNUITYNLV state encoding: '*'=alive, 't'=dead,
   leftmost=life=0)
26 print("  Per-state (survivor) cross-check at age 65:")

```

Numerical output (Example 1)

age	ANNUITYNLV(**)	ANNUITYLV2(0)	diff
65	6.19995	6.19995	8.88e-16
68	5.20653	5.20653	8.88e-16
72	4.06290	4.06290	0.00e+00
76	3.13309	3.13309	4.44e-16
80	2.40943	2.40943	0.00e+00

9 CAPITALLV

CAPITALLV handles endowment (capital) insurance: a death benefit paid at the end of the year of death plus a survival benefit paid at the maturity age. The class also exposes an embedded-value engine with additional setters for risk discount rate, surrender penalties, solvency capital, and investment return.

Constructor



```
cap = ml.CAPITALLV()
cap = ml.CAPITALLV(lMaxTimes)
```

Method reference

Method	Description
<code>iSetTable(id: str) -> int</code>	Load mortality from the named table.
<code>vSetStartTime(t)</code>	Maturity (end) age.
<code>vSetStopTime(t)</code>	Entry (start) age.
<code>vSetDeath(S)</code>	Set a constant death benefit S payable for all years.
<code>vSetSurvival(age, S)</code>	Set the survival benefit S due at maturity age.
<code>vSetPremium(P)</code>	Set a constant annual premium P (pre-numerando).
<code>vSetDeathGen(t, v)</code>	Override the death benefit at a single age t .
<code>vSetPremiumGen(t, v)</code>	Override the premium at a single age t .
<code>dSetQx(age, q) -> double</code>	Set q_x at the given age.
<code>dSetDisc(t, v) -> double</code>	Discount factor at time step t .
<code>dGetDK(age) -> double</code>	Net reserve at the given age.
<code>dGetCF(age) -> double</code>	Expected cash flow at the given age.
<code>dGetRP(age) -> double</code>	Risk premium.
<code>dGetSP(age) -> double</code>	Saving premium.
<code>dGetRegP(age) -> double</code>	Regular (expected) cash flow at the given age.
<code>dGetEV(age) -> double</code>	Embedded value at the given age (requires EV setters).
<code>dSetRDR(t, r) -> double</code>	Risk discount rate at time t .
<code>dSetSurrenderPenaltyInMR(t, v) -> double</code>	Surrender penalty as a fraction of mathematical reserve.
<code>dSetSolaCapitalInMR(t, v) -> double</code>	Solvency capital as a fraction of mathematical reserve.
<code>dSetInvReturn(t, r) -> double</code>	Investment return at time t .

Example: `ex_capitallv.py`

```
1 """
2 ex_capitallv.py  --  CAPITALLV: endowment / capital insurance examples.
3
4 Policy: entry age x, maturity age x+n.
5       Death benefit S paid at end of year of death.
6       Survival benefit S paid at maturity.
7       Annual premium P (pre-numerando).
```



```

8 """
9 import ex_setup
10 import markovlv as ml
11 import math
12
13 SEP = "-" * 60
14
15 def weibull_qx(age, alpha=0.00015, beta=5.5):
16     """Simple power-law mortality: mu_x = alpha * age^beta (very rough)."""
17     if age <= 0: return 0.0
18     mu = alpha * (age ** beta)
19     return min(1.0 - math.exp(-mu), 0.9999)
20
21 def makeham_qx(age, A=0.0007, B=0.00005, c=1.09):
22     mu = A + B * (c ** age)
23     return 1.0 - math.exp(-mu)
24
25 def fill_qx(cap, func=makeham_qx):
26     for x in range(130):
27         cap.dSetQx(x, func(x))
28
29 def fill_disc(cap, i, n=2500):
30     for t in range(n):
31         cap.dSetDisc(t, 1.0 / (1.0 + i))
32
33 # -- Example 1: baseline endowment
34 -----
35 print("CAPITALLV -- 30-year endowment, entry 35, maturity 65")
36 print(SEP)
37 cap = ml.CAPITALLV()
38 fill_qx(cap)
39 cap.vSetStopTime(35); cap.vSetStartTime(65)
40 cap.vSetDeath(100_000.0)
41 cap.vSetSurvival(65, 100_000.0)
42 cap.vSetPremium(2_500.0)
43 fill_disc(cap, 0.03)
44 print(f" Makeham mortality, i=3%, S=100k, P=2500")
45 print(f" DK(35) = {cap.dGetDK(35):10.2f} (should ~= 0 if P is net)")
46 print(f" DK(50) = {cap.dGetDK(50):10.2f}")
47 print(f" DK(64) = {cap.dGetDK(64):10.2f}")
48 print()
49 # -- Example 2: compute approximate net premium
50 -----
51 print("Net premium calculation (entry 35, maturity 65, S=100k)")
52 print(SEP)
53 # EPV(benefits) with P=0
54 cap_b = ml.CAPITALLV(); fill_qx(cap_b)
55 cap_b.vSetStopTime(35); cap_b.vSetStartTime(65)
56 cap_b.vSetDeath(100_000.0); cap_b.vSetSurvival(65, 100_000.0)
57 cap_b.vSetPremium(0.0); fill_disc(cap_b, 0.03)
58 epv_ben = cap_b.dGetDK(35)
59 # EPV(unit premium annuity) with P=1, S=0
60 cap_a = ml.CAPITALLV(); fill_qx(cap_a)
61 cap_a.vSetStopTime(35); cap_a.vSetStartTime(65)
62 cap_a.vSetDeath(0.0); cap_a.vSetSurvival(65, 0.0)
63 cap_a.vSetPremium(1.0); fill_disc(cap_a, 0.03)
64 epv_prem = -cap_a.dGetDK(35) # sign: premium is negative in reserve

```

```

64 P_net = epv_ben / epv_prem if epv_prem > 0 else 0
65 print(f"    EPV(benefits)          = {epv_ben:10.2f}")
66 print(f"    EPV(unit prem annuity)= {epv_prem:10.5f}")
67 print(f"    Net annual premium      = {P_net:10.2f}")
68 print()
69
70 # -- Example 3: reserve progression
71 -----
72 print("Reserve progression with net premium (entry 35, maturity 65)")
73 print(SEP)
74 cap_net = ml.CAPITALLV(); fill_qx(cap_net)
75 cap_net.vSetStopTime(35); cap_net.vSetStartTime(65)
76 cap_net.vSetDeath(100_000.0); cap_net.vSetSurvival(65, 100_000.0)
77 cap_net.vSetPremium(P_net); fill_disc(cap_net, 0.03)
78 print(f"    {'age':>4}    {'DK':>12}    {'RP':>10}    {'SP':>10}")
79 for age in [35, 40, 45, 50, 55, 60, 64, 65]:
80     V = cap_net.dGetDK(age)
81     rp = cap_net.dGetRP(age)
82     sp = cap_net.dGetSP(age)
83     print(f"    {age:4d}    {V:12.2f}    {rp:10.2f}    {sp:10.2f}")
84 print()
85
86 # -- Example 4: vary entry age
87 -----
88 print("Sensitivity -- vary entry age (maturity 65, S=100k, P=2500, i=3%)")
89 print(SEP)
90 print(f"    {'entry':>6}    {'term':>5}    {'DK(entry)':>12}")
91 for x in [25, 30, 35, 40, 45, 50]:
92     c = ml.CAPITALLV(); fill_qx(c)
93     c.vSetStopTime(x); c.vSetStartTime(65)
94     c.vSetDeath(100_000.0); c.vSetSurvival(65, 100_000.0)
95     c.vSetPremium(2_500.0); fill_disc(c, 0.03)
96     print(f"    {x:6d}    {65-x:5d}    {c.dGetDK(x):12.2f}")
97 print()
98
99 # -- Example 5: vary interest rate
100 -----
101 print("Sensitivity -- vary interest rate (entry 35, S=100k, P=2500)")
102 print(SEP)
103 print(f"    {'i':>5}    {'DK(35)':>12}    {'DK(50)':>12}")
104 for i in [0.01, 0.02, 0.03, 0.04, 0.05]:
105     c = ml.CAPITALLV(); fill_qx(c)
106     c.vSetStopTime(35); c.vSetStartTime(65)
107     c.vSetDeath(100_000.0); c.vSetSurvival(65, 100_000.0)
108     c.vSetPremium(2_500.0); fill_disc(c, i)
109     print(f"    {i:5.3f}    {c.dGetDK(35):12.2f}    {c.dGetDK(50):12.2f}")
110 print()
111
112 # -- Example 6: Swiss table CH-QX-EKM-1995
113 -----
114 print("Swiss table CH-QX-EKM-1995 (entry 35, S=100k, P=2500, i=2.5%)")
115 print(SEP)
116 c_ch = ml.CAPITALLV()
117 c_ch.iSetTable("CH-QX-EKM-1995")
118 c_ch.vSetStopTime(35); c_ch.vSetStartTime(65)
119 c_ch.vSetDeath(100_000.0); c_ch.vSetSurvival(65, 100_000.0)
120 c_ch.vSetPremium(2_500.0); fill_disc(c_ch, 0.025)

```

```

117 print(f"    DK(35) = {c_ch.dGetDK(35):10.2f}")
118 print(f"    DK(50) = {c_ch.dGetDK(50):10.2f}")
119 print(f"    DK(64) = {c_ch.dGetDK(64):10.2f}")

```

Listing 5: ex_capitalv.py — endowment insurance examples

Numerical output

```

CAPITALLV -- 30-year endowment, entry 35, maturity 65
    DK(35) =   -5030.21
    DK(50) =   36046.79

Net premium calculation (entry 35, maturity 65, S=100k)
    EPV(benefits)          =   43481.50
    Net annual premium    =    2240.77

Swiss table CH-QX-EKM-1995 (entry 35, S=100k, P=2500, i=2.5%)
    DK(35) =   -2094.44

```

10 MULTDECRLV — Multiple-Decrement Model

MULTDECRLV is a direct generalisation of **CAPITALLV** to n competing causes of exit. It models the classical multiple-decrement table as a Markov chain with $n + 1$ states.

State space and transitions

State	Interpretation
0	Active (in force, alive)
$k = 1 \dots n$	Absorbed by decrement k

Annual transition probabilities:

$$p_{0 \rightarrow k}(t) = q_k(t), \quad k = 1, \dots, n \quad p_{0 \rightarrow 0}(t) = 1 - \sum_{k=1}^n q_k(t)$$

States $1, \dots, n$ are absorbing. The library enforces $\sum_k q_k(t) \leq 1$ implicitly through the complement rule.

With improvement factors the effective rate is

$$q_k(t, \text{year}) = q_k(t) \cdot \exp(f_k(t) \cdot (\text{year} - \text{baseYear})).$$

Each decrement k carries an independent benefit $b_k(t)$ payable on the transition $0 \rightarrow k$. An optional survival benefit $b_0(t)$ (transition $0 \rightarrow 0$, post-numerando) and a pre-numerando premium complete the model.

Relation to CAPITALLV

When $n = 1$ the two classes are *algebraically identical*: a single decrement $k = 1$ with benefit b_1 reproduces CAPITALLV's death benefit and b_0 reproduces the survival/endowment benefit. The regression test suite verifies zero difference at machine precision for all ages.

Linearity property. Because all decrements share the same survival probability ${}_t p_0 = \prod_s (1 - \sum_k q_k(s))$, the reserve is linear in the benefit vector:

$$V(x) = \sum_{k=1}^n b_k \cdot \underbrace{\frac{q_k}{\sum_j q_j}}_{\text{cause fraction}} \cdot V_{\text{CAPITALLV}}(x)$$

when all q_k and b_k are constant. In particular $\sum_k V_k = V_{\text{CAPITALLV}}$ when all $b_k = b$.

Constructor

```
m = ml.MULTDECRLV(n)           # n decrements, max_times = 250
m = ml.MULTDECRLV(n, max_times) # explicit time horizon
```

Method reference

Method	Description
<code>iSetTable(k, id) -> int</code>	Load q_k from the named table; also sets discount from <code>iTech</code> . $k = 1 \dots n$.
<code>vSetStartTime(t)</code>	Maturity (end) age s . Boundary condition $V(s) = 0$.
<code>vSetStopTime(t)</code>	Entry (start) age x . Reserve is evaluated at $x \dots s$.
<code>dSetQx(t, k, q) -> double</code>	Set decrement rate $q_k(t)$. $k = 1 \dots n$.
<code>dSetFx(t, k, f) -> double</code>	Set improvement factor $f_k(t)$ for exponential projection.
<code>dSetBaseYear(y) -> double</code>	Base year for improvement calculation (default 2000).
<code>dSetActualYear(y) -> double</code>	Valuation calendar year (default 2000).
<code>dSetDisc(t, v) -> double</code>	Discount factor $v = 1/(1+i)$ at calendar year t . Negative t sets all 2500 slots (C++ only).
<code>vSetBenefit(k, b)</code>	Set a constant benefit b_k for all years.
<code>vSetBenefitGen(t, k, b)</code>	Override benefit b_k at a single age t .
<code>vSetSurvival(s, b)</code>	Endowment-style: zero all times then set b_0 due at age s . (Post-numerando transition $0 \rightarrow 0$.)
<code>vSetSurvivalGen(t, b)</code>	Set survival benefit at one age without zeroing others.

Method	Description
<code>vSetPremium(P)</code>	Set a constant annual premium P (pre-numerando).
<code>vSetPremiumGen(t, P)</code>	Override premium at a single age t .
<code>dGetDK(t) -> double</code>	Net reserve at age t for the active state.
<code>dGetDK(t, state) -> double</code>	Reserve for an arbitrary state $state = 0 \dots n$.
<code>dGetCF(t) -> double</code>	Expected total cash flow at age t .
<code>dGetRP(t) -> double</code>	Risk premium (TZ decomposition).
<code>dGetSP(t) -> double</code>	Saving premium (TZ decomposition).
<code>dGetRegP(t) -> double</code>	Regular (expected) payment stream.
<code>dGetQx(t, year, k) -> double</code>	Effective decrement rate at age t , calendar year $year$, decrement k , after applying improvement.
<code>lGetNDecrements() -> int</code>	Returns n (number of decrements).

Bug fix: vSetNrStates (v3.8.3)

The MARKOVLV base constructor initialises the internal state counter `lNrStates` to 0; the Thiele recursion loop runs over $0 \dots lNrStates-1$ and therefore produces all-zero reserves until `vSetNrStates` is called. All other derived classes call it explicitly in their constructors; **MULTDECRLV** now calls `MARKOVLV::vSetNrStates(n+1)` at the end of `vInit`.

Example: ex_multdecrlv.py

```

1  """
2  ex_multdecrlv.py  --  MULTDECRLV: multiple-decrement life model examples.
3
4  Model:  n+1 states
5      State 0          : active (alive / in force)
6      States 1 .. n   : absorbed by decrement k   (k = 1 .. n)
7
8  Transitions (annual):
9      p_{0->k}(t) = q_k(t)                decrement k rate
10     p_{0->0}(t) = 1 - sum_k q_k(t)      probability of remaining active
11     States 1..n are absorbing (no further transitions).
12
13 Each decrement k carries:
14     - A decrement rate  q_k(t)          (dSetQx(t, k, v))
15     - A benefit         b_k(t)          payable on transition 0->k  (vSetBenefit /
16                               vSetBenefitGen)
17     - Optional improvement factor f_k(t) (dSetFx / dSetBaseYear /
18                               dSetActualYear)
19
20 An optional survival benefit b_0 and a pre-numerando premium complete the
21 model.
22 """
23
24 import ex_setup
25 import markovlv as ml
26 import math

```

```

23
24 SEP = "-" * 60
25
26 def makeham_qx(age, A=0.0007, B=0.00005, c=1.09):
27     mu = A + B * (c ** age)
28     return 1.0 - math.exp(-mu)
29
30 def fill_disc(obj, i, n=2500):
31     for t in range(n):
32         obj.dSetDisc(t, 1.0 / (1.0 + i))
33
34 # -- Example 1: MULTDECRLV(n=1) must reproduce CAPITALLV exactly -----
35 # --
36 print("Example 1 -- MULTDECRLV(n=1) vs CAPITALLV (30-year endowment)")
37 print(SEP)
38
39 cap = ml.CAPITALLV()
40 for x in range(130):
41     cap.dSetQx(x, makeham_qx(x))
42 cap.vSetStopTime(35); cap.vSetStartTime(65)
43 cap.vSetDeath(100_000.0); cap.vSetSurvival(65, 100_000.0)
44 cap.vSetPremium(2_000.0); fill_disc(cap, 0.03)
45
46 mlt = ml.MULTDECRLV(1) # n=1: identical structure to CAPITALLV
47 for x in range(130):
48     mlt.dSetQx(x, 1, makeham_qx(x))
49 mlt.vSetStartTime(65); mlt.vSetStopTime(35)
50 mlt.vSetBenefit(1, 100_000.0); mlt.vSetSurvival(65, 100_000.0)
51 mlt.vSetPremium(2_000.0); fill_disc(mlt, 0.03)
52
53 print(f" {'age':>4} {'CAPITALLV':>12} {'MULTDECRLV':>12} {'diff':>10}")
54 for age in [35, 40, 45, 50, 55, 60, 64]:
55     v_cap = cap.dGetDK(age)
56     v_mlt = mlt.dGetDK(age)
57     print(f" {age:4d} {v_cap:12.4f} {v_mlt:12.4f} {abs(v_cap - v_mlt):10.2e}")
58 print()
59
60 # -- Example 2: Two decrements -- death and lapse -----
61 # --
62 print("Example 2 -- n=2 (decrement 1: death, decrement 2: lapse)")
63 print("      Entry 40, maturity 65, i=3%, death benefit 100k, surrender 20k")
64 print(SEP)
65
66 md2 = ml.MULTDECRLV(2)
67 for x in range(130):
68     qx = makeham_qx(x)
69     lx = max(0.0, 0.08 - 0.002 * (x - 40)) if x >= 40 else 0.0 # lapse falls with age
70     md2.dSetQx(x, 1, qx)
71     md2.dSetQx(x, 2, lx)
72 md2.vSetStartTime(65); md2.vSetStopTime(40)
73 md2.vSetBenefit(1, 100_000.0) # death benefit
74 md2.vSetBenefit(2, 20_000.0) # surrender value
75 md2.vSetSurvival(65, 100_000.0) # maturity benefit
76 md2.vSetPremium(3_500.0)
77 fill_disc(md2, 0.03)

```

```

76
77 print(f"  {'age':>4}  {'DK':>12}  {'RP':>10}  {'SP':>10}  {'RegP':>10}  TZ-
    check")
78 for age in [40, 45, 50, 55, 60]:
79     V = md2.dGetDK(age)
80     rp = md2.dGetRP(age)
81     sp = md2.dGetSP(age)
82     rg = md2.dGetRegP(age)
83     tz = rp + sp + rg          # must be 0
84     print(f"  {age:4d}  {V:12.2f}  {rp:10.2f}  {sp:10.2f}  {rg:10.2f}  {tz
        :+.1e}")
85 print()
86
87 # -- Example 3: Benefit decomposition -- two causes, same total qx -----
    ----
88 print("Example 3  --  Benefit decomposition (same total qx = 0.012)")
89 print("          qx1=0.004 (death), qx2=0.008 (disability exit), x=50, s=70,
        i=3%")
90 print(SEP)
91
92 def make_md_flat(b1, b2, n=2, q1=0.004, q2=0.008, x=50, s=70, i=0.03):
93     m = ml.MULTDECRLV(n, s + 2)
94     m.vSetStartTime(s)
95     m.vSetStopTime(x)
96     for t in range(x, s):
97         m.dSetQx(t, 1, q1)
98         m.dSetQx(t, 2, q2)
99     fill_disc(m, i)
100     m.vSetBenefit(1, b1)
101     m.vSetBenefit(2, b2)
102     m.vSetPremium(0.0)
103     return m
104
105 cap_ref = ml.CAPITALLV(72)
106 cap_ref.vSetStartTime(70); cap_ref.vSetStopTime(50)
107 for t in range(50, 70):
108     cap_ref.dSetQx(t, 0.012)
109 fill_disc(cap_ref, 0.03)
110 cap_ref.vSetDeath(100_000.0)
111 cap_ref.vSetPremium(0.0)
112 MR_cap = cap_ref.dGetDK(50)
113
114 scenarios = [
115     ("Sc1: b1=b2=100k (=CAPITALLV)", 100_000.0, 100_000.0),
116     ("Sc2: b1=100k, b2=0", 100_000.0, 0.0),
117     ("Sc3: b1=0, b2=100k", 0.0, 100_000.0),
118     ("Sc4: b1=100k, b2=150k", 100_000.0, 150_000.0),
119 ]
120
121 print(f"  CAPITALLV baseline  MR(50) = {MR_cap:10.4f}")
122 print()
123 print(f"  {'Scenario':<28}  {'MR(50)':>10}  {'Ratio':>8}")
124 for label, b1, b2 in scenarios:
125     m = make_md_flat(b1, b2)
126     MR = m.dGetDK(50)
127     print(f"  {label:<28}  {MR:10.4f}  {MR/MR_cap:8.6f}")
128 print()
129

```

```

130 # -- Example 4: mortality improvement across decrements -----
131 # --
132 print("Example 4 -- Mortality improvement (n=2, entry 40, maturity 65, i
      =3%)")
133 print("      Decrement 1 (death):      f1 = -0.015   (1.5% p.a.
      improvement)")
134 print("      Decrement 2 (disability):f2 = -0.010   (1.0% p.a.
      improvement)")
135 print(SEP)
136
137 ACTUAL_YEAR = 2026
138 BASE_YEAR   = 2000
139
140 def make_md_imp(actual_yr):
141     m = ml.MULTDECRLV(2)
142     for x in range(130):
143         m.dSetQx(x, 1, makeham_qx(x))
144         m.dSetQx(x, 2, makeham_qx(x) * 0.3) # disability = 30% of
      mortality
145         m.dSetFx(x, 1, -0.015)
146         m.dSetFx(x, 2, -0.010)
147         m.dSetBaseYear(BASE_YEAR)
148         m.dSetActualYear(actual_yr)
149         m.vSetStartTime(65); m.vSetStopTime(40)
150         m.vSetBenefit(1, 100_000.0)
151         m.vSetBenefit(2, 50_000.0)
152         m.vSetPremium(2_000.0)
153         fill_disc(m, 0.03)
154     return m
155
156 print(f" {'Year':>6} {'DK(40)':>12} {'RP(40)':>10} {'SP(40)':>10}")
157 for yr in [2000, 2010, 2020, 2026, 2040]:
158     m = make_md_imp(yr)
159     print(f" {yr:6d} {m.dGetDK(40):12.2f} {m.dGetRP(40):10.2f} {m.
      dGetSP(40):10.2f}")
160 print()
161 # -- Example 5: net premium for a 2-decrement contract -----
162 # --
163 print("Example 5 -- Net annual premium (n=2, death + lapse, entry 40,
      mat 65)")
164 print(SEP)
165
166 def net_premium_md(x, s, i, b_death, b_lapse, b_surv):
167     # EPV of benefits (P=0)
168     mb = ml.MULTDECRLV(2)
169     for age in range(130):
170         mb.dSetQx(age, 1, makeham_qx(age))
171         mb.dSetQx(age, 2, max(0.0, 0.08 - 0.002*(age - x)))
172         mb.vSetStartTime(s); mb.vSetStopTime(x)
173         mb.vSetBenefit(1, b_death)
174         mb.vSetBenefit(2, b_lapse)
175         mb.vSetSurvival(s, b_surv)
176         mb.vSetPremium(0.0)
177         fill_disc(mb, i)
178         epv_ben = mb.dGetDK(x)
179
180     # EPV of unit premium annuity (all benefits = 0, P=1)

```



```

180     ma = ml.MULTDECRLV(2)
181     for age in range(130):
182         ma.dSetQx(age, 1, makeham_qx(age))
183         ma.dSetQx(age, 2, max(0.0, 0.08 - 0.002*(age - x)))
184     ma.vSetStartTime(s); ma.vSetStopTime(x)
185     ma.vSetBenefit(1, 0.0)
186     ma.vSetBenefit(2, 0.0)
187     ma.vSetSurvival(s, 0.0)
188     ma.vSetPremium(1.0)
189     fill_disc(ma, i)
190     epv_prem = -ma.dGetDK(x)
191
192     P_net = epv_ben / epv_prem if epv_prem > 1e-12 else 0.0
193     return P_net, epv_ben, epv_prem
194
195 P, ben, prem = net_premium_md(40, 65, 0.03, 100_000.0, 20_000.0, 100_000.0)
196 print(f"  EPV(benefits at P=0)      = {ben:12.2f}")
197 print(f"  EPV(unit prem annuity)    = {prem:12.5f}")
198 print(f"  Net annual premium          = {P:12.2f}")
199 print()
200
201 # verify TZ identity at net premium
202 mv = ml.MULTDECRLV(2)
203 for age in range(130):
204     mv.dSetQx(age, 1, makeham_qx(age))
205     mv.dSetQx(age, 2, max(0.0, 0.08 - 0.002*(age - 40)))
206 mv.vSetStartTime(65); mv.vSetStopTime(40)
207 mv.vSetBenefit(1, 100_000.0); mv.vSetBenefit(2, 20_000.0)
208 mv.vSetSurvival(65, 100_000.0)
209 mv.vSetPremium(P)
210 fill_disc(mv, 0.03)
211
212 print(f"  Reserve at net premium (should be ~0 at entry):")
213 print(f"  {'age':>4}  {'DK':>12}  {'RP+SP+RegP':>12}")
214 for age in [40, 45, 50, 55, 60]:
215     V = mv.dGetDK(age)
216     tz = mv.dGetRP(age) + mv.dGetSP(age) + mv.dGetRegP(age)
217     print(f"  {age:4d}  {V:12.2f}  {tz:+12.2e}")

```

Listing 6: ex_multdecrly.py — multiple-decrement examples

Numerical output

Example 1 -- MULTDECRLV(n=1) vs CAPITALLV (30-year endowment)

age	CAPITALLV	MULTDECRLV	diff
35	4672.1331	4672.1331	0.00e+00
40	15450.1973	15450.1973	0.00e+00
45	27796.9673	27796.9673	0.00e+00
50	41954.5774	41954.5774	0.00e+00
55	58266.2047	58266.2047	0.00e+00
60	77283.4910	77283.4910	0.00e+00
64	95087.3786	95087.3786	0.00e+00

Example 2 -- n=2 (death + lapse, entry 40, maturity 65, i=3%)

age	DK	RP	SP	RegP	TZ-check
40	-7443.45	2286.07	1213.93	-3500.00	+9.1e-13
45	-530.56	1558.13	1941.87	-3500.00	-4.5e-13
50	11870.42	644.56	2855.44	-3500.00	+0.0e+00
55	31564.76	-408.47	3908.47	-3500.00	+3.6e-12
60	60341.83	-1538.68	5038.68	-3500.00	-2.7e-12

Example 3 -- Benefit decomposition (constant $q_x=0.012 = 0.004+0.008$)

CAPITALLV baseline MR(50) = 16145.5715
 Sc1: b1=b2=100k (=CAPITALLV) 16145.5715 ratio 1.000000
 Sc2: b1=100k, b2=0 5381.8572 ratio 0.333333
 Sc3: b1=0, b2=100k 10763.7143 ratio 0.666667
 Sc4: b1=100k, b2=150k 21527.4287 ratio 1.333333

Example 4 -- Mortality improvement ($f1=-1.5\%/yr$, $f2=-1.0\%/yr$)

Year	DK (40)	RP (40)	SP (40)
2000	-25967.87	324.83	1675.17
2010	-27213.00	284.87	1715.13
2020	-28306.55	249.57	1750.43
2026	-28897.01	230.43	1769.57
2040	-30104.99	191.10	1808.90

Example 5 -- Net premium ($P=2704.59$), reserve at entry $\sim= 0$

age	DK	RP+SP+RegP
40	0.00	+0.00e+00
45	6858.00	+1.82e-12
50	18723.05	-4.55e-12
55	37175.85	+5.91e-12
60	63758.30	-9.09e-12

11 CAPITAL2LV — Two-Life Capital Insurance

CAPITAL2LV prices endowment and whole-life capital policies on *two independent lives* x (life 1) and y (life 2). The state space is identical to **ANNUITYLV2**; the difference is the benefit structure: instead of regular annuity payments the contract pays lump-sum death and maturity benefits.

State space and transitions

State	Label	Interpretation
0	**	Both alive
1	*t	x alive, y dead
2	t*	x dead, y alive
3	tt	Both dead (absorbing)

Annual transition probabilities (lives independent):

$$\begin{aligned} p_{0 \rightarrow 0}(t) &= (1 - q_x)(1 - q_y), & p_{0 \rightarrow 1}(t) &= (1 - q_x) q_y, \\ p_{0 \rightarrow 2}(t) &= q_x(1 - q_y), & p_{0 \rightarrow 3}(t) &= q_x q_y, \\ p_{1 \rightarrow 1}(t) &= 1 - q_x, & p_{1 \rightarrow 3}(t) &= q_x, \\ p_{2 \rightarrow 2}(t) &= 1 - q_y, & p_{2 \rightarrow 3}(t) &= q_y. \end{aligned}$$

State 3 (tt) is absorbing.

If the two lives are of different ages, set $\delta = \text{age}(y) - \text{age}(x)$ via `dSetY_Minus_X`; the library then looks up q_y at the offset age.

Benefits

Transition	Type	Setter
$0 \rightarrow 1$	Death (y dies)	<code>vSetDeath(b, "**", "*t")</code>
$0 \rightarrow 2$	Death (x dies)	<code>vSetDeath(b, "**", "t*")</code>
$0 \rightarrow 3$	Death (both)	<code>vSetDeath(b, "**", "tt")</code>
$1 \rightarrow 3$	Death (last survivor x)	<code>vSetDeath(b, "*t", "tt")</code>
$2 \rightarrow 3$	Death (last survivor y)	<code>vSetDeath(b, "t*", "tt")</code>
State 0 at maturity	Survival	<code>vSetSurvival(s, b, "**")</code>
State 1 at maturity	Survival	<code>vSetSurvival(s, b, "*t")</code>
State 2 at maturity	Survival	<code>vSetSurvival(s, b, "t*")</code>
State 0	Premium (pre-numerando)	<code>vSetPremium(P, "**")</code>
State 1	Premium (pre-numerando)	<code>vSetPremium(P, "*t")</code>
State 2	Premium (pre-numerando)	<code>vSetPremium(P, "t*")</code>

State-string tokens are `"**"`, `"*t"`, `"t*"`, `"tt"`; all setters accept these tokens as `char*` parameters.

Constructor

```
m = ml.CAPITAL2LV()           # default max_times = 250
m = ml.CAPITAL2LV(max_times)  # explicit time horizon
```

Method reference

Method	Description
<code>iSetTable1(id) -> int</code>	Load q_x from the named table; sets discount from <code>iTech</code> .
<code>iSetTable2(id) -> int</code>	Load q_y from the named table.
<code>vSetStartTime(t)</code>	Maturity age s .
<code>vSetStopTime(t)</code>	Entry age x_0 .
<code>dSetQx1(t, q) -> double</code>	Mortality rate $q_x(t)$ for life 1.

Method	Description
<code>dSetQx2(t, q) -> double</code>	Mortality rate $q_y(t)$ for life 2.
<code>dSetFx1(t, f) -> double</code>	Improvement factor for q_x .
<code>dSetFx2(t, f) -> double</code>	Improvement factor for q_y .
<code>dSetBaseYear(y) -> double</code>	Base year for improvement (default 2000).
<code>dSetActualYear(y) -> double</code>	Valuation calendar year (default 2000).
<code>dSetDisc(t, v) -> double</code>	Discount factor v at year t .
<code>dSetY_Minus_X(yAge, xAge) -> double</code>	Age offset $\delta = \text{age}(y) - \text{age}(x)$.
<code>vSetDeath(b, from, to)</code>	Constant death benefit b on transition $\text{from} \rightarrow \text{to}$.
<code>vSetDeathGen(t, b, from, to)</code>	Override death benefit at a single age t .
<code>vSetSurvival(s, b, state)</code>	Maturity benefit b at age s for state $\in \{**, *t, t*\}$. Zeros the full vector first.
<code>vSetSurvivalGen(t, b, state)</code>	Single-age override without zeroing.
<code>vSetPremium(P, state)</code>	Constant annual premium P for state.
<code>vSetPremiumGen(t, P, state)</code>	Override premium at a single age t .
<code>dGetDK(t) -> double</code>	Reserve for state 0 ($**$) at age t .
<code>dGetDK(t, state) -> double</code>	Reserve for the given state $\in \{0, 1, 2, 3\}$ at age t .
<code>dGetCF(t) -> double</code>	Expected cash flow from state 0.
<code>dGetRP(t, state) -> double</code>	Risk premium (TZ decomposition).
<code>dGetSP(t, state) -> double</code>	Saving premium (TZ decomposition).
<code>dGetRegP(t, state) -> double</code>	Regular payment stream.
<code>dGetQx1(t, year) -> double</code>	Effective q_x with improvement at calendar year.
<code>dGetQx2(t, year) -> double</code>	Effective q_y with improvement at calendar year.

Implementation note: survival in pre-numerando

Survival (maturity) benefits and premiums are stored together in the pre-numerando cash-flow vector (`dSetPre`) for each active state. Setting `dSetPost` on a self-transition $s \rightarrow s$ would overwrite the corresponding pre-numerando entry; combining them into a single `dSetPre` call avoids this. Self-loop post-numerando vectors are still allocated (zero-valued) as a defence-in-depth measure. As of v3.8.3 the engine-level null-guard in `dGetRegP` is present (see §16), so these sentinel allocations are no longer strictly required for correctness, but are retained for clarity.

TZ identity

The Thiele decomposition satisfies

$$RP(t, s) + SP(t, s) + \text{RegP}(t, s) = 0$$

at machine precision for all active states $s \in \{0, 1, 2\}$ and all ages $t \in [x_0, s)$.

Example: ex_capital2lv.py

```

1  """
2  ex_capital2lv.py  --  CAPITAL2LV: two-life capital insurance examples.
3
4  Model: 4 states
5      "*" = 0  both alive
6      "*t" = 1  x alive, y dead
7      "t*" = 2  x dead, y alive
8      "tt" = 3  both dead (absorbing)
9
10 Transitions (annual, independent lives):
11     p(0->0) = (1-qx)(1-qy)    p(0->1) = (1-qx)*qy
12     p(0->2) = qx*(1-qy)        p(0->3) = qx*qy
13     p(1->1) = (1-qx)           p(1->3) = qx
14     p(2->2) = (1-qy)           p(2->3) = qy
15
16 Benefits:
17     Death      : payable on transitions 0->1, 0->2, 0->3, 1->3, 2->3
18     Maturity   : payable at start_time on self-transitions 0->0, 1->1, 2->2
19     Premium    : pre-numerando while in states "*", "*t", "t*"
20 """
21 import ex_setup
22 import markovlv as ml
23 import math
24
25 SEP = "-" * 60
26
27 def makeham_qx(age, A=0.0007, B=0.00005, c=1.09):
28     mu = A + B * (c ** age)
29     return 1.0 - math.exp(-mu)
30
31 def fill_disc(obj, i, n=2500):
32     for t in range(n):
33         obj.dSetDisc(t, 1.0 / (1.0 + i))
34
35 # -- Example 1: CAPITAL2LV vs CAPITALLV (same-age lives, any-survivor
36 #      benefit) -
37 print("Example 1  --  CAPITAL2LV vs CAPITALLV  (30-year endowment, x=y=35)"
38 )
39 print("          Any-survivor policy: benefit whenever either life is alive
40       at 65")
41 print(SEP)
42
43 # Reference: CAPITALLV with last-survivor mortality
44 # For independent lives: p_last-surv = 1 - qx*qy
45 # We replicate a joint last-survivor endowment using CAPITAL2LV:
46 #     benefit 100k on maturity in states "*", "*t", "t*"
47 #     death benefit 0 (no early payout)

```

```

45 # premium from "***" only
46 # CAPITALLV reference: use last-survivor qx = 1-(1-qx)^2 (same age) and
    survival 100k
47
48 x = 35; s = 65; i = 0.03; B = 100_000.0
49
50 # -- CAPITAL2LV: last-survivor endowment -----
    ----
51 m2 = ml.CAPITAL2LV()
52 for age in range(130):
53     m2.dSetQx1(age, makeham_qx(age))
54     m2.dSetQx2(age, makeham_qx(age)) # same-age lives
55 m2.vSetStartTime(s); m2.vSetStopTime(x)
56 m2.vSetSurvival(s, B, "***") # both alive at maturity
57 m2.vSetSurvival(s, B, "*t") # x alive at maturity
58 m2.vSetSurvival(s, B, "t*") # y alive at maturity
59 # No death benefits, no premiums
60 fill_disc(m2, i)
61
62 # -- CAPITALLV reference with last-survivor qx -----
    ----
63 cap = ml.CAPITALLV()
64 for age in range(130):
65     q = makeham_qx(age)
66     qls = 1.0 - (1.0 - q) ** 2 # last-survivor qx for same-age lives
67     cap.dSetQx(age, qls)
68 cap.vSetStartTime(s); cap.vSetStopTime(x)
69 cap.vSetSurvival(s, B)
70 cap.vSetPremium(0.0)
71 fill_disc(cap, i)
72
73 print(f" {'age':>4} {'CAPITAL2LV':>12} {'CAPITALLV(LS)':>14} {'diff
    ':>10}")
74 for age in [35, 40, 45, 50, 55, 60, 64]:
75     v2 = m2.dGetDK(age)
76     vc = cap.dGetDK(age)
77     print(f" {age:4d} {v2:12.4f} {vc:14.4f} {abs(v2-vc):10.2e}")
78 print()
79
80 # -- Example 2: Joint-life endowment (benefit only if BOTH survive) -----
    ----
81 print("Example 2 -- Joint-life endowment (x=40, y=35, s=65, i=3%)")
82 print(" Maturity 100k in '*' only; death benefit 50k on 0->3 (both
    die)")
83 print(SEP)
84
85 m3 = ml.CAPITAL2LV()
86 for age in range(130):
87     m3.dSetQx1(age, makeham_qx(age))
88     m3.dSetQx2(age, makeham_qx(age))
89 m3.dSetY_Minus_X(35, 40) # y is 5 years younger than x
90 m3.vSetStartTime(s); m3.vSetStopTime(40)
91
92 # maturity: only if both alive
93 m3.vSetSurvival(s, B, "***")
94
95 # simultaneous death benefit
96 m3.vSetDeath(50_000.0, "***", "tt")

```

```

97
98 # premium from both-alive state only
99 m3.vSetPremium(1_500.0, "**")
100
101 fill_disc(m3, i)
102
103 print(f" {'age':>4} {'DK(**)':>12} {'DK(*t)':>12} {'DK(t*)':>12}")
104 for age in [40, 45, 50, 55, 60, 64]:
105     print(f" {age:4d} {m3.dGetDK(age,0):12.2f} {m3.dGetDK(age,1):12.2f}
106           {m3.dGetDK(age,2):12.2f}")
107
108 print()
109
110 # -- Example 3: Full two-life endowment with 5 death benefits -----
111 -----
112 print("Example 3 -- Full benefit set (x=40, y=38, s=65, i=3%)")
113 print("      Death benefits: all 5 transitions, maturity all 3 alive
114      states")
115 print(SEP)
116
117 mf = ml.CAPITAL2LV()
118 for age in range(130):
119     mf.dSetQx1(age, makeham_qx(age))
120     mf.dSetQx2(age, makeham_qx(age))
121 mf.dSetY_Minus_X(38, 40) # y is 2 years younger
122 mf.vSetStartTime(s); mf.vSetStopTime(40)
123
124 # Survival benefits at maturity (different for each remaining state)
125 mf.vSetSurvival(s, 100_000.0, "**") # both alive
126 mf.vSetSurvival(s, 60_000.0, "*t") # only x alive
127 mf.vSetSurvival(s, 60_000.0, "t*") # only y alive
128
129 # Death benefits on all 5 transitions
130 mf.vSetDeath(50_000.0, "**", "*t") # y dies (x survives)
131 mf.vSetDeath(50_000.0, "**", "t*") # x dies (y survives)
132 mf.vSetDeath(50_000.0, "**", "tt") # both die simultaneously
133 mf.vSetDeath(30_000.0, "*t", "tt") # x (last survivor) dies
134 mf.vSetDeath(30_000.0, "t*", "tt") # y (last survivor) dies
135
136 # Premiums: different rates by state
137 mf.vSetPremium(2_000.0, "**") # both alive
138 mf.vSetPremium(1_200.0, "*t") # only x alive
139 mf.vSetPremium(1_200.0, "t*") # only y alive
140
141 fill_disc(mf, i)
142
143 print(f" {'age':>4} {'DK(**)':>12} {'DK(*t)':>12} {'DK(t*)':>12} TZ
144      (**)")
145 for age in [40, 45, 50, 55, 60]:
146     V0 = mf.dGetDK(age, 0)
147     V1 = mf.dGetDK(age, 1)
148     V2 = mf.dGetDK(age, 2)
149     tz = mf.dGetRP(age, 0) + mf.dGetSP(age, 0) + mf.dGetRegP(age, 0)
150     print(f" {age:4d} {V0:12.2f} {V1:12.2f} {V2:12.2f} {tz:+.2e}")
151 print()
152
153 # -- Example 4: Mortality improvement -----
154 -----
155 print("Example 4 -- Mortality improvement (x=40, y=40, s=65, i=3%)")

```

```

150 print("          fl=f2=-0.015  (1.5% p.a. for both lives)")
151 print(SEP)
152
153 BASE = 2000; ACTUAL = 2026
154
155 mi = ml.CAPITAL2LV()
156 for age in range(130):
157     mi.dSetQx1(age, makeham_qx(age))
158     mi.dSetQx2(age, makeham_qx(age))
159     mi.dSetFx1(age, -0.015)
160     mi.dSetFx2(age, -0.015)
161 mi.dSetBaseYear(BASE); mi.dSetActualYear(ACTUAL)
162 mi.vSetStartTime(s); mi.vSetStopTime(40)
163 mi.vSetSurvival(s, B, "**")
164 mi.vSetSurvival(s, B, "*t")
165 mi.vSetSurvival(s, B, "t*")
166 mi.vSetDeath(B, "**", "*t")
167 mi.vSetDeath(B, "**", "t*")
168 mi.vSetDeath(B, "**", "tt")
169 mi.vSetDeath(B, "*t", "tt")
170 mi.vSetDeath(B, "t*", "tt")
171 mi.vSetPremium(2_500.0, "**")
172 fill_disc(mi, i)
173
174 mi_no = ml.CAPITAL2LV()
175 for age in range(130):
176     mi_no.dSetQx1(age, makeham_qx(age))
177     mi_no.dSetQx2(age, makeham_qx(age))
178 mi_no.dSetBaseYear(BASE); mi_no.dSetActualYear(BASE) # no improvement
179 mi_no.vSetStartTime(s); mi_no.vSetStopTime(40)
180 mi_no.vSetSurvival(s, B, "**")
181 mi_no.vSetSurvival(s, B, "*t")
182 mi_no.vSetSurvival(s, B, "t*")
183 mi_no.vSetDeath(B, "**", "*t"); mi_no.vSetDeath(B, "**", "t*")
184 mi_no.vSetDeath(B, "**", "tt"); mi_no.vSetDeath(B, "*t", "tt"); mi_no.
    vSetDeath(B, "t*", "tt")
185 mi_no.vSetPremium(2_500.0, "**")
186 fill_disc(mi_no, i)
187
188 print(f"  {'age':>4}  {'DK(*) no-imp':>16}  {'DK(*) imp-2026':>18}  {'
    diff':>10}")
189 for age in [40, 45, 50, 55, 60]:
190     v0 = mi_no.dGetDK(age, 0)
191     vi = mi.dGetDK(age, 0)
192     print(f"  {age:4d}  {v0:16.2f}  {vi:18.2f}  {vi-v0:+10.2f}")
193 print()

```

Listing 7: ex_capital2lv.py — two-life capital insurance examples

Numerical output

Example 1 -- CAPITAL2LV vs CAPITALLV (30-year endowment, $x=y=35$)
 Any-survivor policy: benefit whenever either life is alive
 at 65

age	CAPITAL2LV	CAPITALLV (LS)	diff
-----	------------	----------------	------

35	41641.5473	29908.6683	1.17e+04
40	48382.6579	35345.0915	1.30e+04
45	56247.1452	42045.4496	1.42e+04
50	65436.6073	50524.9696	1.49e+04
55	76180.0524	61667.8358	1.45e+04
60	88690.9503	77094.0836	1.16e+04
64	100000.0000	94572.1590	5.43e+03

Example 2 -- Joint-life endowment (x=40, y=35, s=65, i=3%)
Maturity 100k in '**' only; death benefit 50k on 0->3 (both die)

age	DK(**)	DK(*t)	DK(t*)
40	13914.63	0.00	0.00
45	24817.58	0.00	0.00
50	38035.58	0.00	0.00
55	54494.79	0.00	0.00
60	75849.48	0.00	0.00
64	98505.53	0.00	0.00

Example 3 -- Full benefit set (x=40, y=38, s=65, i=3%)
Death benefits: all 5 transitions, maturity all 3 alive states

age	DK(**)	DK(*t)	DK(t*)	TZ(**)
40	17576.15	7709.57	7753.48	-2.96e-12
45	30363.33	15257.67	15337.10	-1.59e-12
50	45024.44	24071.23	24187.57	-1.59e-12
55	61864.74	34476.99	34613.43	+8.41e-12
60	81311.30	47008.87	47102.35	+1.05e-11

Example 4 -- Mortality improvement (x=40, y=40, s=65, i=3%)
f1=f2=-0.015 (1.5% p.a. for both lives)

age	DK(**) no-imp	DK(**) imp-2026	diff
40	24551.49	16141.60	-8409.90
45	38784.48	30046.91	-8737.57
50	54231.79	45641.05	-8590.75
55	70596.87	62994.68	-7602.19
60	87267.08	82132.09	-5134.99

12 CAPITALNLV — N-Life Capital Insurance

Overview

CAPITALNLV generalises **CAPITAL2LV** from two lives to N independent lives. The same 2^N string-encoded state space is used.

Benefits

Death benefit

Post-numerando on any valid forward transition $s_{\text{from}} \rightarrow s_{\text{to}}$ where s_{to} strictly adds at least one new death. Set per transition pair via `vSetDeath` / `vSetDeathGen`.

Survival (maturity)

Pre-numerando lump sum at maturity T , stored at age $T-1$. Set per state via `vSetSurvival` / `vSetSurvivalGen`.

Premium

Pre-numerando, stored negated. Set per alive state via `vSetPremium` / `vSetPremiumGen`.

Constructors

```
1 CAPITALNLV(int nLives);
2 CAPITALNLV(int nLives, long lMaxTimes);
```

Mortality API

Identical to **ANNUITYNLV**: `iSetTable(k, id), dSetQx(k, t, v), dSetFx(k, t, v), dSetAgeOffset(k, offset), dSetDisc, dSetBaseYear, dSetActualYear`.

Benefit API

```
1 // Death benefits (strFrom, strTo are N-char state strings)
2 void vSetDeath (double v, const char* strFrom, const char* strTo);
3 void vSetDeathGen(long t, double v, const char* strFrom, const char* strTo)
4     ;
5 // Survival / maturity (lTime = maturity age T)
6 void vSetSurvival (long T, double v, const char* strState); // zeros
7 // vector first
8 void vSetSurvivalGen(long T, double v, const char* strState); // single
9 // time-point
10 // Premium (stored negated; v = gross premium amount)
11 void vSetPremium (double v, const char* strState);
12 void vSetPremiumGen(long t, double v, const char* strState);
```

Result methods (require Widow2S token)

```
1 double dGetDK (long t); // V_0(t) from all-alive state
2 double dGetDK (long t, long state); // V_s(t) for any state
3 double dGetCF (long t);
4 double dGetRP (long t, long state);
5 double dGetSP (long t, long state);
6 double dGetRegP (long t, long state);
```

Valid transitions

A transition $s_{\text{from}} \rightarrow s_{\text{to}}$ is valid if and only if $s_{\text{to}} \& s_{\text{from}} = s_{\text{from}}$ (no revival) and $s_{\text{to}} \neq s_{\text{from}}$ (at least one new death). `vSetDeath` silently ignores calls where this condition fails.

Example: `ex_capitalnlv.py`

```

1 print("Example 1 -- CAPITALNLV(2) vs CAPITAL2LV (x=y=35, s=65, i=3%)")
2 print("    Any-survivor endowment 100k; no death benefit; no premium")
3 print(SEP)
4
5 x = 35; s = 65; B = 100_000.0; i = 0.03
6
7 # CAPITALNLV(2)
8 cnv = ml.CAPITALNLV(2)
9 for age in range(130):
10     cnv.dSetQx(0, age, makeham_qx(age))
11     cnv.dSetQx(1, age, makeham_qx(age))
12 cnv.vSetStopTime(x); cnv.vSetStartTime(s)
13 cnv.vSetSurvival(s, B, "**")
14 cnv.vSetSurvival(s, B, "*t")
15 cnv.vSetSurvival(s, B, "t*")
16 fill_disc(cnv, i)
17
18 # CAPITAL2LV reference
19 c2 = ml.CAPITAL2LV()
20 for age in range(130):
21     c2.dSetQx1(age, makeham_qx(age))
22     c2.dSetQx2(age, makeham_qx(age))
23 c2.vSetStopTime(x); c2.vSetStartTime(s)
24 c2.vSetSurvival(s, B, "**")
25 c2.vSetSurvival(s, B, "*t")
26 c2.vSetSurvival(s, B, "t*")
27 fill_disc(c2, i)
28
29 print(f"    {'age':>4}    {'CAPITALNLV(2)':>16}    {'CAPITAL2LV':>14}    {'diff'
30         ':>10}")
31 for age in [35, 40, 45, 50, 55, 60, 64]:
32     vn = cnv.dGetDK(age, 0)
33     v2 = c2.dGetDK(age, 0)
34     print(f"    {age:4d}    {vn:16.4f}    {v2:14.4f}    {abs(vn-v2):10.2e}")
35 print()
36 # -- Example 2: Full benefit set with premiums and death benefits -----
37 print("Example 2 -- Full benefit set (x=40, y=38, s=65, i=3%)")

```

Numerical output (Example 1)

Example 1	-- CAPITALNLV(2)	vs CAPITAL2LV	(x=y=35, s=65, i=3%)
age	CAPITALNLV(2)	CAPITAL2LV	diff
35	41641.5473	41641.5473	1.46e-11
40	48382.6579	48382.6579	0.00e+00
45	56247.1452	56247.1452	7.28e-12

50	65436.6073	65436.6073	7.28e-12
55	76180.0524	76180.0524	1.46e-11
60	88690.9503	88690.9503	4.37e-11
64	100000.0000	100000.0000	0.00e+00

13 WIDDOWLV

WIDDOWLV prices survivor (widow's/widower's) pensions. The insured x pays premiums while alive. On x 's death the survivor y receives an annuity contingent on a survivor existing (probability $h_x(t)$) and on the survivor's expected age $y_x(t)$ when x dies at age t .

Constructor

```
wid = ml.WIDDOWLV()
wid = ml.WIDDOWLV(lMaxTimes)
```

Method reference

Method	Description
<code>vSetStartTime(t)</code>	Oldest projected age for the insured x .
<code>vSetStopTime(t)</code>	Valuation age for x .
<code>dSetQx(age, q) -> double</code>	Annual mortality q_x for the insured.
<code>dSetQy(age, q) -> double</code>	Annual mortality q_y for the survivor.
<code>dSetHx(age, h) -> double</code>	Probability that a survivor exists when x dies at age.
<code>dSetYx(age, ya) -> double</code>	Expected age of the survivor when x dies at age.
<code>dSetDisc(t, v) -> double</code>	Discount factor at time step t .
<code>dSetPre(alpha) -> double</code>	Pre-numerando fraction for the survivor annuity.
<code>vLeistReset()</code>	Reset survivor benefit to unit benefit.
<code>vSetLeistExp(t0, t1, b0, f)</code>	Exponentially varying survivor benefit (e.g. CPI escalation).
<code>dGetDK(age) -> double</code>	Present value of future survivor pension at the given age of x .
<code>dGetCF(age) -> double</code>	Expected cash flow at age.

Example: `ex_widdowlv.py`

```
1 """
2 ex_widdowlv.py -- WIDDOWLV: widow's/survivor's pension examples.
```

```

3
4 The insured (x) pays premiums while alive.
5 On death of x, the survivor (y) receives a pension for life.
6 hx(t) = probability that a widow/widower exists when x dies at age t.
7 yx(t) = expected age of the survivor when x dies at age t.
8 """
9 import ex_setup
10 import markovlv as ml
11 import math
12
13 SEP = "-" * 62
14
15 def makeham_qx(age, A=0.0007, B=0.00005, c=1.09):
16     mu = A + B * (c ** age)
17     return 1.0 - math.exp(-mu)
18
19 def gompertz_qy(age, m=87.0, sigma=9.5): # lighter mortality for survivor
20     return 1.0 - math.exp(-math.exp((age - m) / sigma))
21
22 def fill_wid(wid, qx_func, qy_func, hx=0.70, age_gap=-3, i=0.03):
23     """Populate all rate vectors for WIDDOWLV."""
24     for age in range(130):
25         wid.dSetQx(age, qx_func(age))
26         wid.dSetQy(age, qy_func(age))
27         wid.dSetFx(age, 0.0) # no improvement
28         wid.dSetFy(age, 0.0)
29         wid.dSetHx(age, hx)
30         wid.dSetYx(age, float(age + age_gap)) # survivor age_gap younger
31     for t in range(2500):
32         wid.dSetDisc(t, 1.0 / (1.0 + i))
33
34 # -- Example 1: baseline widow's pension
35 -----
36 print("WIDDOWLV -- Widow's pension, insured male age 45 (i=3%)")
37 print(SEP)
38 wid = ml.WIDDOWLV()
39 wid.vSetStopTime(45); wid.vSetStartTime(100)
40 fill_wid(wid, makeham_qx, gompertz_qy, hx=0.70, age_gap=-3, i=0.03)
41 wid.dSetPre(1.0)
42 print(f" Makeham male, Gompertz female (m=87/s=9.5)")
43 print(f" hx=70%, survivor 3 years younger, i=3%, unit benefit")
44 print(f" DK(45) = {wid.dGetDK(45):.5f}")
45 print(f" DK(55) = {wid.dGetDK(55):.5f}")
46 print(f" DK(65) = {wid.dGetDK(65):.5f}")
47 print()
48 # -- Example 2: vary probability of widow hx
49 -----
50 print("Sensitivity -- vary hx (probability survivor exists) (age 45, i=3%)")
51 print(SEP)
52 print(f" {'hx':>5} {'DK(45)':>12} {'DK(55)':>12} {'DK(65)':>12}")
53 for hx in [0.40, 0.55, 0.70, 0.85, 1.00]:
54     w = ml.WIDDOWLV()
55     w.vSetStopTime(45); w.vSetStartTime(100)
56     fill_wid(w, makeham_qx, gompertz_qy, hx=hx, age_gap=-3, i=0.03)
57     w.dSetPre(1.0)

```

```

57     print(f"    {hx:5.2f}    {w.dGetDK(45):12.5f}    {w.dGetDK(55):12.5f}    {w.
        dGetDK(65):12.5f}")
58 print()
59
60 # -- Example 3: vary age gap
    -----
61 print("Sensitivity -- vary survivor age gap (hx=70%, i=3%)")
62 print(SEP)
63 print(f"    {'gap':>4}    {'DK(45)':>12}    note")
64 for gap in [-10, -5, -3, 0, +3, +5]:
65     w = ml.WIDDOWLV()
66     w.vSetStopTime(45); w.vSetStartTime(100)
67     fill_wid(w, makeham_qx, gompertz_qy, hx=0.70, age_gap=gap, i=0.03)
68     w.dSetPre(1.0)
69     note = "survivor younger" if gap < 0 else ("same age" if gap==0 else "
        survivor older")
70     print(f"    {gap:4d}    {w.dGetDK(45):12.5f}    {note}")
71 print()
72
73 # -- Example 4: vary interest rate
    -----
74 print("Sensitivity -- vary interest rate (hx=70%, gap=-3%)")
75 print(SEP)
76 print(f"    {'i':>5}    {'DK(45)':>12}")
77 for i in [0.01, 0.02, 0.03, 0.04, 0.05]:
78     w = ml.WIDDOWLV()
79     w.vSetStopTime(45); w.vSetStartTime(100)
80     fill_wid(w, makeham_qx, gompertz_qy, hx=0.70, age_gap=-3, i=i)
81     w.dSetPre(1.0)
82     print(f"    {i:5.3f}    {w.dGetDK(45):12.5f}")
83 print()
84
85 # -- Example 5: escalating benefit (CPI 2% p.a.)
    -----
86 print("Escalating benefit -- flat vs 2% p.a. CPI (age 45, i=3%)")
87 print(SEP)
88 w_flat = ml.WIDDOWLV()
89 w_flat.vSetStopTime(45); w_flat.vSetStartTime(100)
90 fill_wid(w_flat, makeham_qx, gompertz_qy, hx=0.70, age_gap=-3, i=0.03)
91 w_flat.dSetPre(1.0)
92
93 w_esc = ml.WIDDOWLV()
94 w_esc.vSetStopTime(45); w_esc.vSetStartTime(100)
95 fill_wid(w_esc, makeham_qx, gompertz_qy, hx=0.70, age_gap=-3, i=0.03)
96 w_esc.dSetPre(1.0)
97 w_esc.vSetLeistExp(0, 120, 1.0, 1.02)
98
99 print(f"    Flat benefit    DK(45) = {w_flat.dGetDK(45):.5f}")
100 print(f"    2% escalation    DK(45) = {w_esc.dGetDK(45):.5f}    (ratio {w_esc.
        dGetDK(45)/w_flat.dGetDK(45):.4f})")
101 print()
102
103 # -- Example 6: cash-flow profile
    -----
104 print("Cash-flow profile (hx=70%, gap=-3, i=3%)")
105 print(SEP)
106 print(f"    {'age':>4}    {'CF':>10}")
107 for age in range(45, 80):

```

```
108 print(f" {age:4d} {w_flat.dGetCF(age):10.6f}")
```

Listing 8: ex_widdowlv.py — widow's pension examples

Numerical output

```
WIDDOWLW -- Widow's pension, insured male age 45 (i=3%)
  hx=70%, survivor 3 years younger, unit benefit
  DK(45) = 1.26452
  DK(55) = 1.30497

Sensitivity -- vary hx
    hx      DK(45)
  0.40      0.72258
  0.70      1.26452
  1.00      1.80646
```

14 DISABILITYLV V0.62

DISABILITYLV implements a multi-duration disability model. Instead of a single “disabled” state, disability is tracked across n ordered duration buckets, capturing the empirical improvement in recovery rates during the first years of a claim.

V0.61 change. The disability inception year $[t, t + 1]$ counts as the first 12 months of the Wartefrist. Effective residual WP: $\max(0, iWF - 12)$ months.

V0.62 change. For $iWF < 12$ months the WP expires within the inception year. Sub-annual benefit payments earned after WP expiry are accumulated and stored as a `dPost` payment on the active $\rightarrow j = 1$ transition, making WF = 6 strictly more expensive than WF = 12.

V3.8.3 fix — backward-recursion off-by-one. The 2-parameter constructor `DISABILITYLV(n_states, max_times)` now passes `max_times + 1` to the `MARKOVLV` parent. This ensures that calling `vSetStartTime(omega)` with `max_times = omega` no longer triggers the bounds guard in `omarkov_CPP20.cpp`, so the recursion step at $t = \omega - 1$ is computed correctly. The Python `vSetStartTime` override introduced in v3.8.2 has been removed; Python now delegates to the parent unchanged (which was already mathematically correct). See `timing_analysis.tex` for a full analysis.

State space

State	Meaning
0	Active (healthy)
1	Disabled, duration 0 (first year)
2	Disabled, duration 1
$\vdots$	$\vdots$
n	Disabled, duration $n - 1$ (last bucket)
$n + 1$	Dead (absorbing)

Transitions

At each age t the following transition probabilities are used:

$$\begin{aligned}
 p_{0 \rightarrow 0} &= 1 - q_{xa}(t) - i_x(t) \\
 p_{0 \rightarrow 1} &= i_x(t) \\
 p_{0 \rightarrow n+1} &= q_{xa}(t) \\
 p_{j \rightarrow j+1} &= 1 - q_{xi}(t) - r_x(t, j-1) \quad (1 \leq j < n) \\
 p_{j \rightarrow 0} &= r_x(t, j-1) \quad (\text{reactivation}) \\
 p_{j \rightarrow n+1} &= q_{xi}(t)
 \end{aligned}$$

Wartefrist — iSetWF

The sub-annual payment frequency m determines the effective pre-numerando fraction:

$$\alpha = \frac{m+1}{2m} \quad (\text{pre}), \quad \alpha = \frac{m-1}{2m} \quad (\text{post})$$

V0.61 timing. Effective residual WP to enforce at state level:

$$\text{effWFMonths} = \max(0, \text{iWF} - 12)$$

iWF < 12 months (V0.62). Sub-annual payments earned after WP expiry:

$$k_{\text{start}} = \left\lfloor \frac{\text{iWF} \cdot m}{12} \right\rfloor, \quad \text{dPostInception} = \sum_{k=k_{\text{start}}}^{m-1} \frac{b(t)}{m} \cdot v^{-(m-k)/m}$$

Constructor

```

dis = ml.DISABILITYLV() # default: 6 duration states,
    max_times=250
dis = ml.DISABILITYLV(n_states) # custom duration depth,
    max_times=250
dis = ml.DISABILITYLV(n_states, max_times) # explicit time horizon

```

Note on max_times. When using the 2-parameter form with `max_times = omega`, the constructor internally allocates `max_times + 1` slots so that `vSetStartTime(omega)` is handled correctly (v3.8.3 fix). Callers who use the default (`max_times = 250`) with $\omega \leq 130$ are unaffected.

Method reference

Method	Description
<code>vSetStartTime(t)</code>	Oldest projected age (horizon).
<code>vSetStopTime(t)</code>	Entry / valuation age.

Method	Description
dSetQx(age, q) -> double	Active mortality q_{xa} at the given age.
dSetQxI(age, q) -> double	Disabled mortality q_{xi} . Defaults to q_{xa} if not set.
dSetIx(age, ix) -> double	Inception rate i_x .
dSetRx(age, dur, rx) -> double	Reactivation rate for disability duration dur (0-indexed).
dSetDisc(t, v) -> double	Discount factor at time step t .
iSetWF(iWF, m, bPre) -> int	Configure waiting period and payment frequency. iWF: months [1, 36]; m: payments/year; bPre: 1 = pre, 0 = post.
bSetReactBehaviour(b) -> bool	True: reactivated claimants return to state 0.
vLeistReset()	Reset disability benefit to zero at all ages.
vSetLeistLinear(t0, t1, b0, incr)	Linearly varying disability benefit.
vPremiumReset()	Set premium to zero (premium-free reserve).
dGetDK(age, state) -> double	Mathematical reserve at age in state.
dGetCF(age, state) -> double	Expected cash flow at age in state.
dGetRP(age, state) -> double	Risk premium.
dGetSP(age, state) -> double	Saving premium.
dGetRegP(age, state) -> double	Regular (expected) cash flow at age in state.

Example: ex_disabilitylv.py

```

1  """
2  ex_disabilitylv.py  --  DISABILITYLV: multi-duration disability examples.
3
4  State space (6 duration states):
5      0          : active (healthy)
6      1 .. 6     : disabled, duration 0 .. 5  years
7      7          : dead (absorbing)
8
9  All demographic rates set directly (no table needed).
10 """
11 import ex_setup
12 import markovlv as ml
13 import math
14
15 SEP = "-" * 64
16
17 def makeham_qxa(age, A=0.0007, B=0.00005, c=1.09):

```

```

18     """Active mortality."""
19     return 1.0 - math.exp(-(A + B * c**age))
20
21 def inception_rate(age, peak_age=45, base=0.004, width=15.0):
22     """Bell-shaped inception rate peaking at peak_age."""
23     return base * math.exp(-0.5 * ((age - peak_age) / width)**2)
24
25 def react_rate(dur, base=0.12, decay=0.25):
26     """Reactivation decreasing with duration."""
27     return base * math.exp(-decay * dur)
28
29 def setup_dis(n_states=6, qxi_mult=1.0, ix_mult=1.0, rx_mult=1.0,
30              i=0.03, entry=40, horizon=65, benefit=10_000.0):
31     """Build a standard DISABILITYLV and return it."""
32     dis = ml.DISABILITYLV(n_states)
33     dis.vSetStopTime(entry)
34     dis.vSetStartTime(horizon)
35     for age in range(250):
36         qxa = makeham_qxa(age)
37         dis.dSetQx(age, qxa)
38         dis.dSetQxI(age, qxa * qxi_mult)      # disabled mortality
39         dis.dSetIx(age, inception_rate(age) * ix_mult)
40         for dur in range(n_states):
41             dis.dSetRx(age, dur, react_rate(dur) * rx_mult)
42     for t in range(2500):
43         dis.dSetDisc(t, 1.0 / (1.0 + i))
44     dis.vSetLeistLinear(0, 249, benefit, 0.0)
45     dis.vPremiumReset()
46     return dis
47
48 # -- Example 1: baseline
49 -----
50 print("DISABILITYLV -- Baseline, entry 40, benefit CHF 10 000 p.a.")
51 print(SEP)
52 dis0 = setup_dis()
53 print(f"  Makeham mortality, bell inception, exp reactivation, i=3%")
54 print(f"  {'State':>6}  {'Description':15}  {'DK(40,s)':>12}")
55 labels = ["active"] + [f"dis dur {k}" for k in range(6)] + ["dead"]
56 for s in range(8):
57     print(f"  {s:6d}  {labels[s]:15}  {dis0.dGetDK(40,s):12.2f}")
58 print()
59 # -- Example 2: vary inception rate
60 -----
61 print("Sensitivity -- vary inception rate (iM=12, iWF=12, pre)")
62 print(SEP)
63 print(f"  {'ix mult':>8}  {'DK(40,active)':>15}")
64 for mult in [0.5, 0.75, 1.0, 1.5, 2.0]:
65     dis = setup_dis(ix_mult=mult)
66     dis.iSetWF(12, 12, 1)
67     print(f"  {mult:8.2f}  {dis.dGetDK(40,0):15.2f}")
68 print()
69 # -- Example 3: vary reactivation rate
70 -----
71 print("Sensitivity -- vary reactivation rate (iWF=12, m=12, pre)")
72 print(SEP)
73 print(f"  {'rx mult':>8}  {'DK(40,active)':>15}  {'DK(40,dis0)':>14}")

```

```

73 for mult in [0.0, 0.5, 1.0, 2.0, 3.0]:
74     dis = setup_dis(rx_mult=mult)
75     dis.iSetWF(12, 12, 1)
76     print(f" {mult:8.2f} {dis.dGetDK(40,0):15.2f} {dis.dGetDK(40,1):14.2f}")
77 print()
78
79 # -- Example 4: Wartefrist sweep
-----
80 print("Wartefrist sweep -- iWF in {12,18,24,30,36}, m=12, pre vs post")
81 print(SEP)
82 print(f" {'iWF':>4} {'DK pre':>12} {'DK post':>12} {'diff':>8}")
83 for wf in [12, 18, 24, 30, 36]:
84     dp = setup_dis(); dp.iSetWF(wf, 12, 1)
85     dq = setup_dis(); dq.iSetWF(wf, 12, 0)
86     vp = dp.dGetDK(40, 0); vq = dq.dGetDK(40, 0)
87     print(f" {wf:4d} {vp:12.2f} {vq:12.2f} {vp-vq:8.2f}")
88 print()
89
90 # -- Example 5: vary payment frequency m
-----
91 print("Payment frequency -- vary m (iWF=12, pre-numerando)")
92 print(SEP)
93 print(f" {'m':>3} {'eff alpha':>7} {'DK(40,active)':>15}")
94 for m in [1, 2, 4, 12, 26, 52]:
95     alpha = (m + 1) / (2 * m)
96     dis = setup_dis(); dis.iSetWF(12, m, 1)
97     print(f" {m:3d} {alpha:7.5f} {dis.dGetDK(40,0):15.2f}")
98 print()
99
100 # -- Example 6: vary disabled mortality
-----
101 print("Sensitivity -- disabled mortality multiplier (iWF=12, m=12)")
102 print(SEP)
103 print(f" {'qxi mult':>8} {'DK(40,active)':>15} {'DK(40,dis0)':>14}")
104 for mult in [0.8, 1.0, 1.2, 1.5, 2.0]:
105     dis = setup_dis(qxi_mult=mult); dis.iSetWF(12, 12, 1)
106     print(f" {mult:8.2f} {dis.dGetDK(40,0):15.2f} {dis.dGetDK(40,1):14.2f}")
107 print()
108
109 # -- Example 7: reserve profile over age in active state
-----
110 print("Reserve progression -- active state (iWF=12, m=12, pre)")
111 print(SEP)
112 dis7 = setup_dis(); dis7.iSetWF(12, 12, 1)
113 print(f" {'age':>4} {'DK(active)':>14} {'CF(active)':>14} {'RP':>10} {'SP':>10}")
114 for age in range(40, 66):
115     V = dis7.dGetDK(age, 0)
116     cf = dis7.dGetCF(age, 0)
117     rp = dis7.dGetRP(age, 0)
118     sp = dis7.dGetSP(age, 0)
119     print(f" {age:4d} {V:14.2f} {cf:14.4f} {rp:10.4f} {sp:10.4f}")
120 print()
121
122 # -- Example 8: 3 vs 6 vs 12 duration states
-----

```

```

123 print("Effect of granularity -- 3 / 6 / 12 duration states (iWF=12, m
    =12)")
124 print(SEP)
125 print(f" {'n_states':>9} {'DK(40,active)':>15} {'DK(40,dis0)':>14}")
126 for n in [3, 6, 12]:
127     dis = setup_dis(n_states=n); dis.iSetWF(12, 12, 1)
128     print(f" {n:9d} {dis.dGetDK(40,0):15.2f} {dis.dGetDK(40,1):14.2f}")
129 print()
130
131 # -- Example 9: sub-12-month WF -- full sweep and cost ordering
    -----
132 # V0.62: for iWF < 12 the WP expires within the inception year [t,t+1].
133 # Sub-annual benefit payments earned after WP expiry are captured as a
134 # dPost payment on the active->disabled transition. WF=6 is now strictly
135 # more expensive than WF=12.
136 print("Sub-12-month WF sweep -- full ordering (m=12, pre-numerando, entry
    40)")
137 print(SEP)
138 _ref = setup_dis(); _ref.iSetWF(12, 12, 1)
139 ref12 = _ref.dGetDK(40, 0)
140 print(f" {'iWF':>4} {'DK(40,active)':>15} {'DK(40,dis0)':>14} {'vs WF
    =12':>10}")
141 for wf in [1, 3, 6, 9, 12, 18, 24, 30, 36]:
142     dis = setup_dis(); dis.iSetWF(wf, 12, 1)
143     dk0 = dis.dGetDK(40, 0)
144     dk1 = dis.dGetDK(40, 1)
145     diff = dk0 - ref12
146     print(f" {wf:4d} {dk0:15.2f} {dk1:14.2f} {diff:+10.2f}")
147 print()
148
149 # -- Example 10: WF=3 and WF=6 -- reserve progression near horizon
    -----
150 # Active-state reserve at ages 63 - 65 for WF=3 and WF=6 compared to WF=12.
151 # WF=3: WP expires 3m into inception year; 9 monthly payments captured.
152 # WF=6: WP expires 6m into inception year; 6 monthly payments captured.
153 # Both give DK(63, active) > 0 and > WF=12 result.
154 print("Short WF -- reserve progression at ages 60-65 (m=12, pre)")
155 print(SEP)
156 print(f" {'age':>4} {'WF=3 DK':>12} {'WF=6 DK':>12} {'WF=12 DK':>12}
    {'WF=18 DK':>12} {'WF=24 DK':>12}")
157 dis3 = setup_dis(); dis3.iSetWF(3, 12, 1)
158 dis6 = setup_dis(); dis6.iSetWF(6, 12, 1)
159 dis12 = setup_dis(); dis12.iSetWF(12, 12, 1)
160 dis18 = setup_dis(); dis18.iSetWF(18, 12, 1)
161 dis24 = setup_dis(); dis24.iSetWF(24, 12, 1)
162 for age in range(60, 66):
163     v3 = dis3.dGetDK(age, 0)
164     v6 = dis6.dGetDK(age, 0)
165     v12 = dis12.dGetDK(age, 0)
166     v18 = dis18.dGetDK(age, 0)
167     v24 = dis24.dGetDK(age, 0)
168     print(f" {age:4d} {v3:12.4f} {v6:12.4f} {v12:12.4f} {v18:12.4f}
    {v24:12.4f}")
169 print()
170 print(f" Inception-year post payment at age 63 (benefit on 0->1 edge):")
171 # The partial inception-year payment is visible as a CF on the active state
172 # at any age: CF(active) includes E[benefit during inception year | active]
173 for age in [40, 55, 63, 64]:

```

```

174 cf3 = dis3.dGetCF(age, 0)
175 cf6 = dis6.dGetCF(age, 0)
176 cf12 = dis12.dGetCF(age, 0)
177 print(f" age {age}: CF WF=3={cf3:8.4f} WF=6={cf6:8.4f} WF=12={cf12
:8.4f}")

```

Listing 9: ex_disabilitylv.py — multi-duration disability examples

Numerical output (selected)

DISABILITYLV -- Baseline, entry 40, benefit CHF 10 000 p.a.

State	Description	DK(40,s)
0	active	4706.30
1	dis dur 0	123488.11
6	dis dur 5	172107.31

Wartefrist sweep (m=12, pre vs post)

iWF	DK pre	DK post	diff
12	4552.22	4524.20	28.02
24	4035.88	4014.81	21.06

Sub-12-month WF sweep (V0.62)

iWF	DK(40,active)	vs WF=12
3	4978.68	+426.46
6	4835.47	+283.25
12	4552.22	+0.00
24	4035.88	-516.34

15 WIDROW2SPEC — Two-Life Pension with Guarantee

WIDROW2SPEC is a 522-state Markov model for pricing widow and widower pensions with *per-death-age* benefit schedules and *per-death-age* guarantee periods. It derives from **MARKOVLV** and corresponds to the Swiss KT95 tariff family (W..000, W.S293, W.S290).

When to use **WIDROW2SPEC** vs. **WIDROWLV**

Feature	WIDROWLV	WIDROW2SPEC
States	2 (alive/dead)	522
Guarantee period	Not supported	Yes, per death-age
Escalating benefit	Uniform	Per death-age (correct for KT95)
Widower pension	No	Yes
Joint annuity	No	Yes
Swiss KT95 tariffs	No	Yes (W..000, W.S293, W.S290)

State space

$N = 2 + 4 \times 130 = 522$ states, over a maximum of 250 time steps:

State	Index	Description
Joint alive	0	Both x and y alive
Dead	1	Absorbing
IN_W(a)	$2 + 4a$	Widow inside guarantee, x died at age a
OUT_W(a)	$2 + 4a + 1$	Widow after guarantee, x died at age a
IN_M(a)	$2 + 4a + 2$	Widower inside guarantee, y died at age a
OUT_M(a)	$2 + 4a + 3$	Widower after guarantee, y died at age a

where $a \in \{0, \dots, 129\}$. A separate block of 4 states exists for each possible death age, allowing per-death-age benefit and guarantee logic.

Guarantee mechanism

For death-age a with guarantee $g_a > 0$:

1. $t = a$: transition $0 \rightarrow \text{IN_W}(a)$. Post benefit $d_{\text{post}} \cdot b(a, t)$ paid on this transition.
2. $t = a + 1 \dots a + g_a - 1$: self-loop on IN_W with probability 1 (mortality suppressed — pension certain regardless of survival).
3. $t = a + g_a$: full pre+post payment; transitions to OUT_W or dead.
4. $t \geq a + g_a + 1$: normal survival-contingent annuity in OUT_W.

Swiss KT95 tariff types

Tariff	Benefit	Guarantee
W..000	Level, $b(t) = 1$	None ($g_a = 0$)
W.S293	3% escalating from death year	None
W.S290	3% escalating from death year	$g_a = \min(10, \max(0, 10 + 65 - a))$

Pre/post convention

Standard UDD quarterly approximation: $d_{\text{pre}} = 0.625$, $d_{\text{post}} = 0.375$, corresponding to $\alpha = 3/8$.

Constructor

```
w2 = ml.WIDDOW2SPEC(lMaxTimes, dPre)
# e.g.
w2 = ml.WIDDOW2SPEC(250, 0.625)      # 250 time steps, pre-numerando fraction
0.625
```

Note on lMaxTimes (v 3.8.3 fix). The constructor passes `lMaxTimes + 1` to the MARKOVLV parent internally, so calling `vSetStartTime(lMaxTimes)` computes the full recursion including the step at $t = \text{lMaxTimes} - 1$. Pass the true terminal age ω as `lMaxTimes`; no “+1” adjustment is needed at the call site.

Method reference

Method	Description
<i>Mortality</i>	
<code>dSetQx1 (age, q) -> double</code>	Mortality q_x for the insured (x).
<code>dSetFx1 (age, f) -> double</code>	Improvement factor for x .
<code>dSetQx2 (age, q) -> double</code>	Mortality q_y for the survivor (y).
<code>dSetFx2 (age, f) -> double</code>	Improvement factor for y .
<code>iSetTable1 (id: str) -> int</code>	Load x mortality from the named table.
<code>iSetTable2 (id: str) -> int</code>	Load y mortality from the named table.
<code>dSetBaseYear (y) -> double</code>	Calendar base year for improvement projection.
<code>dSetActualYear (y) -> double</code>	Valuation calendar year.
<i>Ages and timing</i>	
<code>vSetStartTime (t)</code>	Beyond-omega sentinel (e.g. 132).
<code>vSetStopTime (t)</code>	Insured's valuation age.
<code>vSetSAge1 (t)</code>	Payment start age for x (joint phase).
<code>vSetSAge2 (t)</code>	Payment start age for y (joint phase).
<code>dSetY_Minus_X (yAge, xAge) -> double</code>	Age gap: survivor age minus insured age.
<code>dSetDisc (t, v) -> double</code>	Discount factor at time t . Pass $t=-1$ to broadcast to all steps.
<i>Guarantee periods</i>	
<code>vSetGWidow (g)</code>	Uniform widow guarantee: g years for all death ages.
<code>vSetGWidower (g)</code>	Uniform widower guarantee.
<code>vSetGWidowAtAge (a, g)</code>	Per-death-age override: widow guarantee g years when x dies at age a .
<code>vSetGWidowerAtAge (a, g)</code>	Per-death-age override for widower.
<i>Benefit schedules — uniform</i>	
<code>vLeistResetJoint ()</code>	Set joint benefit to zero (unit by default).
<code>vLeistResetWidow ()</code>	Reset widow benefit to unit.
<code>vLeistResetWidower ()</code>	Reset widower benefit to unit.
<code>vSetLeistLinearWidow (t0, t1, b0, incr)</code>	Set uniform widow benefit linearly.
<code>vSetLeistExpWidow (t0, t1, b0, f)</code>	Set uniform widow benefit exponentially. Use only for level or non-escalating schedules; see note below.

Method	Description
<i>Benefit schedules — per death-age override (critical for escalating)</i>	
<code>vSetLeistLinearWidowAtAge(a, t0, t1, b0, incr)</code>	Set widow benefit for death-age block a linearly.
<code>vSetLeistExpWidowAtAge(a, t0, t1, b0, f)</code>	Set widow benefit for block a exponentially. For escalating benefits, use this (not the global version).
<code>vSetLeistLinearWidowerAtAge(a, t0, t1, b0, incr)</code>	Per-death-age widower benefit (linear).
<code>vSetLeistExpWidowerAtAge(a, t0, t1, b0, f)</code>	Per-death-age widower benefit (exponential).
Results	
<code>dGetDK(t, state) -> double</code>	EPV at time t in state. state=0: both alive.
<code>dGetDK(t) -> double</code>	Shorthand: <code>dGetDK(t, 0)</code> .
<code>dGetCF(t) -> double</code>	Total expected cash flow at time t .
<code>dGetRP(t) -> double</code>	Risk premium at time t (state 0, both alive).
<code>dGetSP(t) -> double</code>	Saving premium at time t (state 0, both alive).
<code>dGetRegP(t) -> double</code>	Regular (expected) cash flow at time t (state 0).
<code>dGetQx1(t, year) -> double</code>	Effective q_x at time t and calendar year.
<code>dGetQx2(t, year) -> double</code>	Effective q_y at time t and calendar year.

Critical note: per-death-age vs. global benefit for escalating schedules

For a 3% escalating widow benefit, the correct benefit in block a at time t is $b(a, t) = (1 + j)^{t-a}$ (starts at 1 when x dies at age a).

```
# WRONG: global exponential sets b[t] = (1+j)^t -- absolute, not relative
w2.vSetLeistExpWidow(0, OMEGA, 1.0, 1.03) # block a=65 reads 1.03^65 = 6.8x

# CORRECT: per-death-age so block a reads 1.03^(t-a)
S = 65 # retirement age
for a in range(OMEGA):
    bp = min(a, S) # flat until min(a, S)
    w2.vSetLeistLinearWidowAtAge(a, 0, bp, 1.0, 0.0) # b(a,t) = 1 for t < bp
    w2.vSetLeistExpWidowAtAge(a, bp, OMEGA+10, 1.0, 1.03) # b(a,t) = 1.03^(t-bp)
```

Example: ex_widdow2spec.py

```
1 """
```



```

2 ex_widdow2spec.py    --    WIDDOW2SPEC: two-life widow/widower pension with
   guarantee.
3
4 WIDDOW2SPEC is a 522-state Markov model for pricing widow and widower
   pensions
5 with per-death-age benefit schedules and guarantee periods.
6
7 State space (N = 522 = 2 + 4*130):
8   State 0           : both insured (x) and survivor (y) alive (joint phase)
9   State 1           : both dead (absorbing)
10  States 2..521      : 130 blocks of 4, one block per death-age a:
11      IN_W(a)        = 2 + 4*a       : widow in guarantee period, x died at age a
12      OUT_W(a)       = 2 + 4*a + 1   : widow in normal payment, x died at age a
13      IN_M(a)        = 2 + 4*a + 2   : widower in guarantee, y died at age a
14      OUT_M(a)       = 2 + 4*a + 3   : widower in normal payment, y died at age a
15
16 Three Swiss KT95-style tariff groups are demonstrated:
17   W..000 : level benefit, no guarantee
18   W.S293 : 3% escalating benefit, no guarantee
19   W.S290 : 3% escalating benefit, varying guarantee  $g(a) = \min(10, \max(0, 10+65-a))$ 
20
21 IMPORTANT: for escalating benefits, ALWAYS use vSetLeistExpWidowAtAge(a,
   ...) to
22 set per-death-age benefits. The global vSetLeistExpWidow() gives wrong
   results
23 for escalating schedules (see the calling-code note at the end of this
   script).
24 """
25 import ex_setup
26 import markovlv as ml
27 import math
28
29 SEP = "-" * 68
30
31 OMEGA = 130           # upper age (exclusive)
32 S      = 65           # retirement age (escalation breakpoint)
33 J      = 0.03         # escalation rate 3%
34 I_TECH = 0.035        # technical interest rate
35 ALPHA  = 0.375        # post-numerando fraction (quarterly UDD correction)
36 DPRE   = 1.0 - ALPHA  # = 0.625
37 DELTA  = 3            # insured x is DELTA years older than survivor y
38
39
40 # -- Mortality functions
   -----
41
42 def qx_male(age: int) -> float:
43     """Makeham male mortality."""
44     if age >= OMEGA:
45         return 1.0
46     A, B, c = 0.0007, 5e-5, 1.095
47     return min(1.0, 1.0 - math.exp(-(A + B * c**age)))
48
49 def qx_female(age: int) -> float:
50     """Makeham female mortality (lighter)."""
51     if age < 0 or age >= OMEGA:
52         return 1.0

```

```

53     A, B, c = 0.0004, 2e-5, 1.095
54     return min(1.0, 1.0 - math.exp(-(A + B * c**age)))
55
56 def guarantee(ax: int) -> int:
57     """W.S290 guarantee:  $g(a) = \min(10, \max(0, 10 + S - a))$ ."""
58     return min(10, max(0, 10 + S - ax))
59
60
61 # -- Builder
62     -----
63 def build_w2(x0: int, exp3: bool = False, g_per_age: bool = False,
64             i_rate: float = I_TECH, dpre: float = DPRE) -> "ml.WIDDOW2SPEC
65     ":
66     """
67     Build and configure a WIDDOW2SPEC for insured starting age x0.
68
69     Parameters
70     -----
71     x0          : insured's valuation age
72     exp3        : True = 3% escalating widow benefit; False = level
73     g_per_age   : True = W.S290 varying guarantee; False = no guarantee
74     i_rate      : technical interest rate
75     dpre        : pre-numerando fraction (default quarterly correction 0.625)
76     """
77     y0 = x0 - DELTA          # survivor y is DELTA years younger
78     V = 1.0 / (1.0 + i_rate)
79
80     w2 = ml.WIDDOW2SPEC(250, dpre)
81     w2.vSetStopTime(x0)
82     w2.vSetStartTime(OMEGA + 2)
83
84     # Mortality
85     for age in range(OMEGA + 10):
86         w2.dSetQx1(age, qx_male(age))
87         w2.dSetQx2(age, qx_female(age))
88
89     w2.dSetY_Minus_X(y0, x0)    # age gap: y_age - x_age = -DELTA
90     w2.dSetDisc(-1, V)         # broadcast to all time steps
91
92     # Zero joint and widower pensions (widow-only in these examples)
93     w2.vLeistResetJoint()
94     w2.vLeistResetWidower()
95
96     # Widow benefit
97     w2.vLeistResetWidow()
98     if exp3:
99         # Per-death-age escalating schedule:  $b(a, t) = (1+J)^{\max(0, t - \min(a, S))}$ 
100         # For death before retirement ( $a < S$ ): flat 1.0 until S, then  $(1+J)^{(t-S)}$ 
101         # For death at/after retirement ( $a \geq S$ ):  $(1+J)^{(t-a)}$  from the death age
102         # IMPORTANT: use vSetLeistExpWidowAtAge (per-age), NOT vSetLeistExpWidow (global)
103         for a in range(OMEGA):
104             bp = min(a, S)          # escalation breakpoint

```

```

104         w2.vSetLeistLinearWidowAtAge(a, 0, bp, 1.0, 0.0)    # flat
    until bp
105         w2.vSetLeistExpWidowAtAge(a, bp, OMEGA + 10, 1.0, 1.0 + J)    #
    exp from bp
106     else:
107         # Level unit benefit for all ages
108         w2.vSetLeistLinearWidow(0, OMEGA + 10, 1.0, 0.0)
109
110     # Guarantee periods
111     for a in range(OMEGA):
112         g_a = guarantee(a) if g_per_age else 0
113         w2.vSetGWidowAtAge(a, g_a)
114         w2.vSetGWidowerAtAge(a, 0)
115
116     return w2
117
118
119 # -- Example 1: baseline W..000 (level, no guarantee)
    -----
120 print("WIDOW2SPEC -- W..000: level benefit, no guarantee (i=3.5%, dpre
    =0.625) ")
121 print(SEP)
122 print(f" {'age x':>6} {'EPV(0)':>12}")
123 for x in range(25, 81, 5):
124     w = build_w2(x, exp3=False, g_per_age=False)
125     print(f" {x:6d} {w.dGetDK(x, 0):12.6f}")
126 print()
127
128 # -- Example 2: W.S293 (3% escalating, no guarantee)
    -----
129 print("WIDOW2SPEC -- W.S293: 3% escalating, no guarantee (i=3.5%)")
130 print(SEP)
131 print(f" {'age x':>6} {'EPV(0)':>12}")
132 for x in range(25, 81, 5):
133     w = build_w2(x, exp3=True, g_per_age=False)
134     print(f" {x:6d} {w.dGetDK(x, 0):12.6f}")
135 print()
136
137 # -- Example 3: W.S290 (3% escalating, varying guarantee)
    -----
138 print("WIDOW2SPEC -- W.S290: 3% escalating, g(a)=min(10,max(0,10+65-a))"
    )
139 print(SEP)
140 print(f" {'age x':>6} {'EPV(0)':>12} {'guarantee g(x)':>14}")
141 for x in range(25, 81, 5):
142     w = build_w2(x, exp3=True, g_per_age=True)
143     print(f" {x:6d} {w.dGetDK(x, 0):12.6f} {guarantee(x):>14d}")
144 print()
145
146 # -- Example 4: four-tariff side-by-side
    -----
147 I_NET = (1.0 + I_TECH) / (1.0 + J) - 1.0    # net rate ~= 0.4854%
148
149 print("WIDOW2SPEC -- Four-tariff comparison (Makeham mortality, delta
    =3) ")
150 print(f" i_net = (1.035/1.03)-1 = {I_NET:.6f}")
151 print(SEP)

```

```

152 print(f"    {'Age':>4}    {'W000 3.5%':>11}    {'W000 inet':>11}    {'WS293':>11}
    {'WS290':>11}")
153 print(f"    {'':>4}    {'level, ng':>11}    {'level, ng':>11}    {'3%, ng':>11}
    {'3%,vary-g':>11}")
154 print(f"    {'-'*4}    {'-'*11}    {'-'*11}    {'-'*11}    {'-'*11}")
155 for x in range(25, 81, 5):
156     v000_tech = build_w2(x, exp3=False, g_per_age=False, i_rate=I_TECH).
    dGetDK(x, 0)
157     v000_net  = build_w2(x, exp3=False, g_per_age=False, i_rate=I_NET ).
    dGetDK(x, 0)
158     v293      = build_w2(x, exp3=True,  g_per_age=False, i_rate=I_TECH).
    dGetDK(x, 0)
159     v290      = build_w2(x, exp3=True,  g_per_age=True,  i_rate=I_TECH).
    dGetDK(x, 0)
160     print(f"    {x:4d}    {v000_tech:11.5f}    {v000_net:11.5f}    {v293:11.5f}    {
    v290:11.5f}")
161 print()
162 print("    Expected ordering at x<75: W000(3.5%) < W000(inet) ~= WS293 <
    WS290")
163 print("    WS290 == WS293 for x >= 75 (guarantee = 0 there).")
164 print()
165
166 # -- Example 5: sensitivity -- interest rate
    -----
167 print("WIDOW2SPEC -- Sensitivity: interest rate (age 45, W.S293)")
168 print(SEP)
169 print(f"    {'i_rate':>8}    {'EPV(0)':>12}")
170 for ir in [0.020, 0.025, 0.030, 0.035, 0.040, 0.050]:
171     w = build_w2(45, exp3=True, g_per_age=False, i_rate=ir)
172     print(f"    {ir:8.3%}    {w.dGetDK(45, 0):12.6f}")
173 print()
174
175 # -- Example 6: sensitivity -- age gap (delta)
    -----
176 print("WIDOW2SPEC -- Sensitivity: age gap delta (age 45, W..000, i
    =3.5%)")
177 print(SEP)
178 print(f"    {'delta':>6}    {'EPV(0)':>12}    note")
179 for delta in [-5, -3, 0, 3, 5]:
180     y0 = 45 - delta
181     w2 = ml.WIDOW2SPEC(250, DPRE)
182     w2.vSetStopTime(45); w2.vSetStartTime(OMEGA + 2)
183     for age in range(OMEGA + 10):
184         w2.dSetQx1(age, qx_male(age))
185         w2.dSetQx2(age, qx_female(age))
186         w2.dSetY_Minus_X(y0, 45)
187         w2.dSetDisc(-1, 1.0 / (1.0 + I_TECH))
188         w2.vLeistResetJoint(); w2.vLeistResetWidower()
189         w2.vLeistResetWidow()
190         w2.vSetLeistLinearWidow(0, OMEGA + 10, 1.0, 0.0)
191         for a in range(OMEGA):
192             w2.vSetGWidowAtAge(a, 0); w2.vSetGWidowerAtAge(a, 0)
193         note = "y younger" if delta > 0 else ("same age" if delta == 0 else "y
    older")
194         print(f"    {delta:6d}    {w2.dGetDK(45, 0):12.6f}    {note}")
195 print()
196

```

```

197 # -- Example 7: state-space inspection (first few blocks)
198 # -----
199 print("WIDOW2SPEC -- State indices for selected death ages")
200 print(SEP)
201 print(f" {'a (death age)':>14} {'IN_W(a)':>9} {'OUT_W(a)':>10} {'IN_M(a)
202       ':>9} {'OUT_M(a)':>10} {'g(a)':>6}")
203 for a in [25, 35, 45, 55, 65, 70, 75, 80]:
204     IN_W = 2 + 4*a
205     OUT_W = 2 + 4*a + 1
206     IN_M = 2 + 4*a + 2
207     OUT_M = 2 + 4*a + 3
208     print(f" {a:14d} {IN_W:9d} {OUT_W:10d} {IN_M:9d} {OUT_M:10d} {
209           guarantee(a):6d}")
210 print()
211 #
212 # -----
213
214 # CALLING-CODE NOTE: global vs per-age benefit for escalating schedules
215 # -----
216
217 print("NOTE: Correct calling code for escalating widow benefit")
218 print(SEP)
219 print("""
220     WRONG -- global vSetLeistExpWidow:
221         w2.vSetLeistExpWidow(0, OMEGA, 1.0, 1.03)
222         # sets psymBenefitWidow[t] = 1.03^t (absolute, shared by all blocks)
223         # Block a=65 reads 1.03^65 = 6.8x at t=65 -> massive overcount
224
225     CORRECT -- per-age vSetLeistExpWidowAtAge:
226         for a in range(OMEGA):
227             bp = min(a, 65)
228             w2.vSetLeistLinearWidowAtAge(a, 0, bp, 1.0, 0.0) # flat to bp
229             w2.vSetLeistExpWidowAtAge(a, bp, OMEGA+10, 1.0, 1.03) # exp from
230             bp
231             # Block a=65: reads 1.03^0=1 at t=65, 1.03^1 at t=66, ... correct!
232 """)

```

Listing 10: ex_widow2spec.py — WIDOW2SPEC examples

Numerical output

```

WIDOW2SPEC -- Four-tariff comparison (Makeham mortality, delta=3)
i_net = (1.035/1.03)-1 = 0.004854

```

Age	W000 3.5% level, ng	W000 inet level, ng	WS293 3%, ng	WS290 3%, vary-g
25	2.402575	11.852437	3.801889	3.854177
30	2.696772	11.881511	4.355694	4.417469
35	3.051415	11.936590	5.019310	5.092197
40	3.458205	11.985939	5.793017	5.878767
45	3.898600	11.988302	6.667072	6.767360
50	4.338209	11.887089	7.613181	7.728850

55	4.741525	11.641806	8.598920	8.728208
60	5.084387	11.241380	9.597572	9.731826
65	5.321861	10.657080	10.540680	10.650556
70	5.455260	9.929900	11.385750	11.428590
75	5.420930	9.026330	11.998140	11.998140
80	5.094490	7.846270	12.090720	12.090720

WS290 == WS293 for age >= 75 (guarantee = 0 there).

16 Premium Decomposition

All `dGetRP`, `dGetSP`, and `dGetRegP` methods implement the classical Thiele decomposition of the change in reserve.

Let V_t denote the mathematical reserve at time t in the current state. The one-period identity is:

$$V_t + P_t = v \left[\sum_j p_{ij}(t) (b_{ij}(t) + V_{t+1}^j) \right]$$

Risk Premium (RP)

$$RP_t = v \sum_{j \neq i} p_{ij}(t) (b_{ij}(t) + V_{t+1}^j - V_{t+1}^i)$$

Saving Premium (SP)

$$SP_t = v \cdot V_{t+1} - V_t$$

where $v = (1 + i)^{-1}$ is the one-period discount factor.

Regular Cash Flow (RegP)

$RegP_t$ is the expected benefit paid in the interval $[t, t + 1)$, weighted by the probability of remaining in the current state.

Decomposition identity

The three quantities satisfy the fundamental identity

$$SP_t + RP_t + RegP_t = 0$$

at every age and state, which is equivalent to the Thiele recursion

$$V_t = v \cdot V_{t+1} + RegP_t + RP_t.$$

Engine-level null-pointer fix (v3.8.3)

Both Markov engine files (`omarkov_CPP20.cpp` and `omarkov_v2S2_CPP20.cpp`) contained two unsafe pointer accesses in the TZ functions that could crash the process when `dGetRegP` or `dGetRP` was called on any state whose follow-state post-numerando vector had never been allocated — most commonly absorbing states such as the "tt" state in two-life models or the dead state in **DISABILITYLV**.

dGetRegP — critical (both engines). The original code unconditionally called `pppsymPostInfo[s][fo]` without first testing whether the pointer was `nullptr`. Any state that had never received a `dSetPost` call on its follow-state transition caused a segmentation fault. The fix adds a null-guard before the dereference:

```
1 // v3.8.3 fix: null-guard before dereference
2 if (pppsymPostInfo[lState][lNach] != nullptr)
3     dWert += pppsymPostInfo[lState][lNach]->dGetValue(lTime);
```

The earlier inner `if(value)` test (which only guarded against adding zero) has been replaced by the null check; adding a zero value is a no-op and the simpler form is equivalent.

dGetRP — secondary (both engines). Inside each Pij loop the code redundantly re-read `pppsymDiscInfo[s][k]->dGetValue(t)` after the same pointer had already been safely read (with a null-guard) in the enclosing block. If the discount vector was `nullptr`, the outer `else` branch correctly set `akt_disk = 0`, but the inner re-read then crashed. The redundant line has been removed; `akt_disk` retains the value established by the guarded outer block.

As defence-in-depth, all derived classes that expose `dGetRegP(t, state)` with an explicit state argument (**CAPITAL2LV**, **ANNUITYLV2**, **DISABILITYLV**) pre-allocate a zero-valued sentinel entry for every state's follow-state post-numerando vector immediately after `vReset()`, so that `dGetRegP` returns 0 safely for absorbing states even if called before the first `dGetDK` invocation. Table 13 summarises which classes carry sentinels and which rely solely on the engine guard.

Table 13: TZ null-pointer fix: sentinel strategy by class.

Class	States	Engine guard	Sentinel
CAPITAL2LV	0–3 (state 3 absorbing)	✓	✓ all 4 states
ANNUITYLV2	0–3 (state 3 absorbing)	✓	✓ state 3 only
DISABILITYLV	0, 1...n, dead	✓	✓ disabled follow-states + dead
CAPITALLV	0–1 (state 1 absorbing)	✓	state 0 always set via <code>dSetPost</code>
ANNUITYLV	0–2 (state 2 absorbing)	✓	state 0 always set via <code>dSetPost</code>
MULTDECRLV	0, 1...n absorbed	✓	state 0 always set via <code>dSetPost</code>
WIDOW2SPEC	all states	✓	state 0 always set via <code>dSetPost</code>

17 Reserve Distribution (CDF)

The engine computes, by backward recursion, the moments of the present value $V_i(t)$ through `dGetDK(t, i, m)`. Risk management, however, frequently needs the full *distribution*

$$P_i(t, u) = P[V(t) < u \mid X_t = i]$$

rather than a few moments — e.g. for value-at-risk, tail-risk, and shortfall probabilities. Since $P_i(t, \cdot)$ is in general *discontinuous* (the distribution has point masses), a bucketed CDF on a *value grid* is the natural numerical representation.

17.1 Backward recursion (Theorem 5.5.1)

Conditioning on the next state $X_{t+1} = l$ and peeling off the first period gives a backward recursion that parallels the reserve recursion of `dGetDK`:

Theorem 17.1 (CDF recursion — [Koller, Ch. 5]).

$$P_i(t, u) = \sum_j p_{ij}(t) P_j(t+1, v_{i,t}^{-1}(u - a_i^{\text{pre}}(t)) - a_{ij}^{\text{post}}(t)). \quad (8)$$

Here $p_{ij}(t)$, $a_i^{\text{pre}}(t)$, $a_{ij}^{\text{post}}(t)$ and $v_{i,t}$ are exactly the quantities already stored on the object via `dSetPij`, `dSetPre`, `dSetPost` and `dSetDisc` — no new data setters are needed.

The boundary condition at $t = \text{startTime}$ is a Heaviside step: for a contract with zero terminal reserve,

$$P_i(\omega+1, u) = \begin{cases} 0, & u \leq 0, \\ 1, & u > 0. \end{cases}$$

The threshold can be configured per state via `vSetDistrBoundary`.

17.2 Value grid and bucket discretisation

The CDF is evaluated on a uniform grid with B buckets on $[L, H]$:

$$\text{bucket}(x) = \min\left(B-1, \left\lfloor \frac{x-L}{w} + \frac{1}{2} \right\rfloor\right), \quad w = \frac{H-L}{B}, \quad \text{value}(b) = L + w \cdot b.$$

Out-of-range values are clamped to the end buckets, so the grid must bracket the support of V .

17.3 Support bounds and automatic grid sizing

For each period the smallest and largest attainable per-period cash flow is

$$m_t = \min_{j,k: p_{jk}(t) > 0} (a_j^{\text{pre}}(t) + a_{jk}^{\text{post}}(t)), \quad M_t = \max_{j,k: p_{jk}(t) > 0} (a_j^{\text{pre}}(t) + a_{jk}^{\text{post}}(t)).$$

Summing over the horizon:

$$V_{\min} = \sum_t m_t, \quad V_{\max} = \sum_t M_t.$$

`distrBounds()` returns $(V_{\min}, V_{\max})$.

`vAutoDistrGrid(B)` uses these bounds to place the grid with guard classes:

$$w = \frac{V_{\max} - V_{\min}}{B-3}, \quad L = V_{\min} - w, \quad H = L + wB = V_{\max} + 2w.$$

This guarantees that the minimum maps to bucket 1 and the maximum to bucket $B-2$, with empty guard buckets at both ends so no probability mass leaks off the grid.

Remark 17.2 (Memory). The grid is a flat buffer of size $(T_{\text{start}} - T_{\text{stop}} + 1) \times N \times B$ doubles. With $B = 2000$ buckets, 10 states and 200 time steps this is about 32 MB. The computation is strictly opt-in: it is triggered only by the first call to `dGetDKDistr`, `dGetDKDistrBucket` or `dGetDKQuantile`, and is never a side-effect of `dGetDK`.

17.4 Consistency check

The first moment reconstructed from the CDF,

$$\hat{\mu}_i(t) = \sum_{b=0}^{B-1} \text{value}(b) \cdot (P_i(t, b) - P_i(t, b-1)),$$

must agree with `dGetDK(t, i, 1)` to grid resolution. This identity is the primary cross-check between the distribution engine and the existing moment recursion.

17.5 Example: `ex_distr.py`

```

1  """
2  ex_distr.py  --  Reserve distribution (CDF) examples with plots.
3
4  Model:  10-year term insurance, state 0 = alive, state 1 = dead.
5  Plots:  (1) CDF of V(0) with VaR/reserve markers
6          (2) CDF comparison across time points t = 0, 3, 6, 9
7
8  """
9  import ex_setup
10 import markovlv as ml
11
12 try:
13     import matplotlib
14     matplotlib.use("Agg")
15     import matplotlib.pyplot as plt
16     HAS_MPL = True
17 except ImportError:
18     HAS_MPL = False
19
20 SEP = "-" * 60
21
22 # -- build model
23 -----
24 n, q, S, P = 10, 0.01, 100_000, 600
25 disc = 1.0 / 1.03
26
27 m = ml.MARKOVLV(n + 2, 2, 1)
28 m.vSetNrStates(2)
29 m.vSetStopTime(0); m.vSetStartTime(n)
30 m.lSetFolgezustand(0, 0); m.lSetFolgezustand(1, 1)
31
32 for t in range(n):
33     m.dSetPij(t, 0, 0, 1.0 - q); m.dSetPij(t, 0, 1, q)
34     m.dSetPij(t, 1, 1, 1.0)
35     m.dSetDisc(t, 0, 0, disc); m.dSetDisc(t, 1, 1, disc)
36     m.dSetPre(t, 0, 0, -P)
37     m.dSetPost(t, 0, 1, S)

```

```

37 # -- grid setup
38 -----
39 NB = 5000
40 lo, hi = m.distrBounds()
41 m.vAutoDistrGrid(NB)
42 grid_lo = m.dGetDistrLow()
43 grid_hi = m.dGetDistrHigh()
44 w = m.dGetBucketWidth()
45
46 print("MARKOV LV -- Reserve distribution (CDF)")
47 print(SEP)
48 print(f" n={n} q={q} S={S:,.0f} P={P} i=3%")
49 print(f" Support bounds: [{lo:.2f}, {hi:.2f}]")
50 print(f" Grid: [{grid_lo:.2f}, {grid_hi:.2f}], {NB} buckets, w={w:.4f}"
51 )
52 print()
53 # -- Example 1: CDF at t=0, state=0 with VaR
54 -----
55 reserve = m.dGetDK(0, 0, 1)
56 cdf_at_res = m.dGetDKDistr(0, 0, reserve)
57 q95 = m.dGetDKQuantile(0, 0, 0.95)
58 q99 = m.dGetDKQuantile(0, 0, 0.99)
59
60 print("Example 1 -- CDF at issue (t=0, alive)")
61 print(SEP)
62 print(f" Reserve E[V] = {reserve:12.2f} P[V < E[V]] = {cdf_at_res:.4f}")
63 print(f" VaR(95%) = {q95:12.2f}")
64 print(f" VaR(99%) = {q99:12.2f}")
65 print()
66 # -- Example 2: CDF at multiple time points
67 -----
68 print("Example 2 -- CDF across time points (alive)")
69 print(SEP)
70 print(f" {'t':>3} {'E[V]':>12} {'VaR(95)':>12} {'VaR(99)':>12}")
71 for t in [0, 3, 6, 9]:
72     ev = m.dGetDK(t, 0, 1)
73     v95 = m.dGetDKQuantile(t, 0, 0.95)
74     v99 = m.dGetDKQuantile(t, 0, 0.99)
75     print(f" {t:3d} {ev:12.2f} {v95:12.2f} {v99:12.2f}")
76 print()
77 # -- Plots
78 -----
79 if not HAS_MPL:
80     print(" (matplotlib not available -- skipping plots)")
81 else:
82     # Collect CDF data: bucket values and CDF probabilities
83     def get_cdf_curve(model, t, s, nb):
84         xs, ys = [], []
85         for b in range(nb):
86             xs.append(model.dValueOfBucket(b))
87             ys.append(model.dGetDKDistrBucket(t, s, b))
88         return xs, ys
89
90     xlo_plot = -8_000

```

```

89 xhi_plot = 110_000
90
91 # --- Figure 1: CDF at t=0 with annotations ---
92 fig, ax = plt.subplots(figsize=(7, 4.2))
93 xs, ys = get_cdf_curve(m, 0, 0, NB)
94 ax.plot(xs, ys, color="#2060A0", linewidth=1.8, label="$P[V(0) < x \\mid \\mathrm{alive}]$")
95 ax.axvline(reserve, color="#C04040", linestyle="--", linewidth=1.2,
96 label=f"$E[V] = {reserve:,.0f}$")
97 ax.axvline(q95, color="#40A040", linestyle=":", linewidth=1.2, label=f"
98 VaR(95%) = {q95:,.0f}")
99 ax.axvline(q99, color="#A06020", linestyle="-.", linewidth=1.2, label=f"
100 VaR(99%) = {q99:,.0f}")
101 ax.set_xlim(xlo_plot, xhi_plot)
102 ax.set_xlabel("Reserve value $x$")
103 ax.set_ylabel("$P[V(0) < x]$")
104 ax.set_title("CDF of net reserve at issue -- 10-year term insurance")
105 ax.legend(loc="center right", fontsize=9)
106 ax.set_ylim(-0.02, 1.05)
107 ax.grid(True, alpha=0.3)
108 fig.tight_layout()
109 fig.savefig("fig_distr_cdf.pdf", bbox_inches="tight")
110 fig.savefig("fig_distr_cdf.png", dpi=150, bbox_inches="tight")
111 print(" Saved fig_distr_cdf.pdf / .png")
112
113 # --- Figure 2: CDF comparison across time ---
114 fig2, ax2 = plt.subplots(figsize=(7, 4.2))
115 colors = ["#2060A0", "#40A040", "#C04040", "#A06020"]
116 for idx, t in enumerate([0, 3, 6, 9]):
117     xs, ys = get_cdf_curve(m, t, 0, NB)
118     ev = m.dGetDK(t, 0, 1)
119     ax2.plot(xs, ys, color=colors[idx], linewidth=1.5,
120             label=f"$t={t}$, $E[V]={ev:,.0f}$")
121 ax2.set_xlim(xlo_plot, xhi_plot)
122 ax2.set_xlabel("Reserve value $x$")
123 ax2.set_ylabel("$P[V(t) < x]$")
124 ax2.set_title("CDF of net reserve -- evolution over time")
125 ax2.legend(loc="center right", fontsize=9)
126 ax2.set_ylim(-0.02, 1.05)
127 ax2.grid(True, alpha=0.3)
128 fig2.tight_layout()
129 fig2.savefig("fig_distr_time.pdf", bbox_inches="tight")
130 fig2.savefig("fig_distr_time.png", dpi=150, bbox_inches="tight")
131 print(" Saved fig_distr_time.pdf / .png")
132 print()

```

Listing 11: ex_distr.py — reserve distribution (CDF) with plots

Numerical output

```
MARKOVLEV -- Reserve distribution (CDF)
```

```

-----
n=10 q=0.01 S=100,000 P=600 i=3%
Support bounds: [-6000.00, 994000.00]
Grid: [-6200.12, 994400.24], 5000 buckets, w=200.1201

```

Example 1 -- CDF at issue ($t=0$, alive)

Reserve $E[V]$	=	3123.37	$P[V < E[V]] = 0.9044$
VaR(95%)	=	80251.75	
VaR(99%)	=	93059.44	

Example 2 -- CDF across time points (alive)

t	$E[V]$	VaR(95)	VaR(99)
0	3123.37	80251.75	93059.44
3	2312.49	80251.75	93059.44
6	1399.29	-2397.84	93059.44
9	370.87	-596.76	-596.76

Interpretation

The distribution of the 10-year term insurance reserve has a characteristic staircase shape. With $q = 1\%$ annual mortality, about 90% of the probability mass corresponds to paths where no death benefit is ever paid; the reserve on those paths equals the accumulated premium shortfall (small negative or zero value). Each step in the upper tail corresponds to a single death occurring in year k — the later the death, the larger the discounted payout. The mean reserve $E[V] = 3\,123$ lies well below the 95th percentile: the distribution is extremely right-skewed.

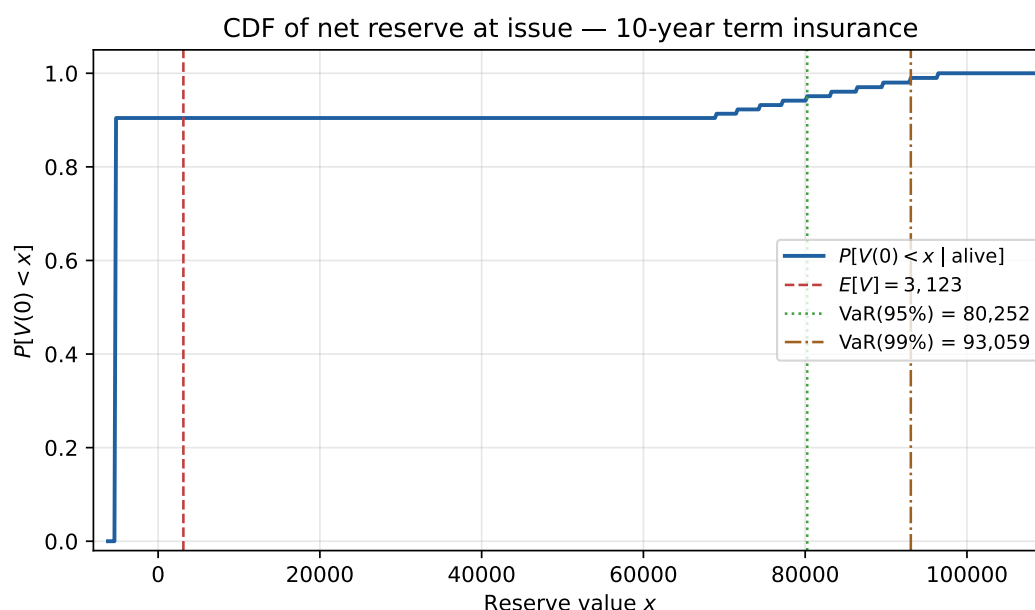


Figure 1: CDF of the net reserve $V(0)$ at issue ($t = 0$, alive). The vertical lines mark the expected reserve $E[V]$, the 95% VaR, and the 99% VaR. The large plateau near $P = 0.90$ corresponds to paths with no death benefit; the staircase in the right tail corresponds to single-death outcomes in successive years.

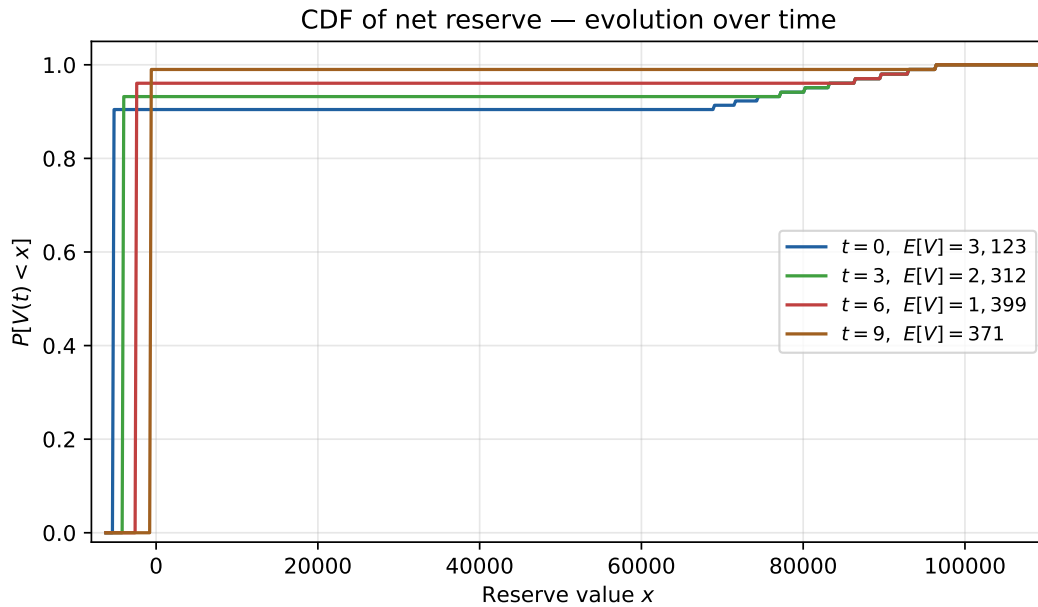


Figure 2: CDF of the net reserve $V(t)$ at times $t = 0, 3, 6, 9$. As the contract ages, the remaining horizon shrinks: fewer death-year outcomes survive, the tail compresses toward zero, and the initial plateau rises (higher survival probability over a shorter horizon). At $t = 9$ only one year of exposure remains and $P[V < 0] \approx 99\%$.

18 TABLESERVER

TABLESERVER is the mortality table server underpinning [ANNUITYLIV](#), [ANNUITYLIV2](#), [CAPITALIV](#), and [WIDOWLIV](#). It loads, stores and provides age-indexed access to actuarial q_x tables.

Design

- Single, unified interface for looking up q_x values at integer ages.
- Optional proportional level scaling via the semicolon syntax "TABLE-ID; level".
- Boundary clamping: ages below x_0 return 0.0; ages above ω return 1.0.
- Sentinel -1 returned if no table has been loaded.

C++ interface

```

1 #define MAXCHARS 40000
2
3 class TABLESERVER
4 {
5 public:
6     TABLESERVER();
7     void vSetTable(char *pcTable); // e.g. "CH-QX-ERF-1995;0.90"
8     double dGetValue(int iAge);
9     int iTableNumber();
10    int iX0();
11    int iXOmega();
12    int iT0();
13    double dITech();

```

```

14  int    iGender();
15  char  *pcAllTarifs();
16  };

```

Listing 12: tableserver.h — class declaration

Note on ERROR macro. `tableserver.h` defines `#define ERROR -1`. This collides with SWIG-generated wrappers that declare a member named `ERROR`. Add `\#undef ERROR` after the class declaration to avoid the conflict.

Return value rules of `dGetValue`

Condition	Return value
No table loaded	-1.0
$\text{age} < x_0$	0.0
$\text{age} > \omega$	1.0
$q_x \geq 0.99999999$	1.0 (omega clamping)
$q_x \cdot \text{level} < 0$	0.0
Otherwise	$q_x \cdot \text{level}$

Python mirror

The Python `TABLESERVER` class in `markovlib.py` mirrors the C++ interface exactly. It loads tables from `tables.json` (via `iAddTables`) instead of the compiled static array.

```

ts = ml.TABLESERVER()
ts.vSetTable("CH-QX-ERF-1995")           # basic load
ts.vSetTable("CH-QX-ERF-1995;0.90")      # 90% level scaling
q65 = ts.dGetValue(65)

```

tables.json format

```

{
  "CH-QX-ERF-1995": {
    "id":      "CH-QX-ERF-1995",
    "type":    "qx",
    "x0":      0,
    "x_omega": 120,
    "t0":      1995,
    "i_tech":  0.035,
    "gender":  1,
    "qx":      [0.00537, 0.000398, ...]  // index 0 = age 0
  },
  ...
}

```

C++ vs Python differences

Aspect	C++	Python
Table back-end	Static C arrays (statictable.c)	JSON dict (tables.json)
Dynamic registration	Not supported	<code>ml.register_table(...)</code>
<code>vSetTable</code> argument	Mutates input string	Non-destructive
Below x_0	Returns 0.0	Returns 1.0 (known bug)
<code>pcAllTarifs</code> buffer	Static; not thread-safe	New string each call

Example: `ex_tableserver.py`

```

1  """
2  ex_tableserver.py  --  TABLESERVER (Python): markovlv module-level
   functions.
3
4  The Python table server lives in markovlv.py and is accessed via the module
   -level
5  functions iAddTables, iSetLicense, etc., plus the TABLESERVER class.
6
7  After ex_setup imports markovlv and loads license + tables.json, all table
8  identifiers from tables.json are already registered in the module-level
   registry.
9  """
10 import ex_setup
11 import markovlv as ml
12
13 SEP = "-" * 64
14
15 # -- Example 1: list all registered tables
   -----
16 print("TABLESERVER  --  List first 10 registered tables")
17 print(SEP)
18 # Use TABLESERVER.pcAllTarifs() to enumerate
19 ts = ml.TABLESERVER()
20 ts.vSetTable("CH-QX-ERF-1995")
21 all_ids = [s for s in ts.pcAllTarifs().split("\n") if s.strip()]
22 print(f"  Total tables registered: {len(all_ids)}")
23 for name in all_ids[:10]:
24     print(f"    {name}")
25 print()
26
27 # -- Example 2: basic qx lookup
   -----
28 print("TABLESERVER  --  Basic qx lookup (CH-QX-ERF-1995, female)")
29 print(SEP)
30 ts.vSetTable("CH-QX-ERF-1995")
31 print(f"  x0          = {ts.iX0()}")
32 print(f"  x_omega      = {ts.iXOmega()}")
33 print(f"  t0          = {ts.iT0()}")
34 print(f"  i_tech       = {ts.dITech():.4f}")
35 print(f"  gender      = {ts.iGender()} (0=male, 1=female)")
36 print()
37 print(f"  {'age':>4}    {'qx':>10}")

```

```

38 for age in [0, 20, 40, 60, 65, 80, 100]:
39     print(f"    {age:4d}    {ts.dGetValue(age):10.7f}")
40 print()
41
42 # -- Example 3: boundary clamping
43 -----
44 print("TABLESERVER -- Boundary clamping")
45 print(SEP)
46 print(f"    age below x0    (age=-1): {ts.dGetValue(-1):.1f}    (should be 0.0)"
47       )
48 print(f"    age above omega (200): {ts.dGetValue(200):.1f}    (should be 1.0)"
49       )
50 print()
51
52 # -- Example 4: level scaling via semicolon syntax
53 -----
54 print("TABLESERVER -- Level scaling (CH-QX-ERM-1995;0.80)")
55 print(SEP)
56 ts_base = ml.TABLESERVER(); ts_base.vSetTable("CH-QX-ERM-1995")
57 ts_scale = ml.TABLESERVER(); ts_scale.vSetTable("CH-QX-ERM-1995;0.80")
58 print(f"    {'age':>4}    {'qx base':>12}    {'qx scaled':>12}    {'ratio':>8}")
59 for age in [40, 50, 60, 65, 75]:
60     qb = ts_base.dGetValue(age)
61     qs = ts_scale.dGetValue(age)
62     print(f"    {age:4d}    {qb:12.7f}    {qs:12.7f}    {qs/qb if qb>0 else 0:8.4f}"
63           )
64 print()
65
66 # -- Example 5: Swiss tables side-by-side
67 -----
68 print("TABLESERVER -- Swiss annuity tables comparison at age 65")
69 print(SEP)
70 tables = ["CH-QX-ERF-1995", "CH-QX-ERM-1995", "CH-QX-EKF-1995"]
71 print(f"    {'table':>28}    {'qx(65)':>10}    {'qx(50)':>10}    {'qx(80)':>10}")
72 for tid in tables:
73     t = ml.TABLESERVER(); t.vSetTable(tid)
74     print(f"    {tid:28}    {t.dGetValue(65):10.7f}    {t.dGetValue(50):10.7f}    {
75           t.dGetValue(80):10.7f}")
76 print()
77
78 # -- Example 6: using iAddTables for a second registry
79 -----
80 print("TABLESERVER -- Using iAddTables to load an additional JSON
81     registry")
82 print(SEP)
83 import os, json, tempfile
84
85 # Build a minimal custom tables.json
86 custom_qx = [0.001 * (age / 100) for age in range(121)]
87 custom_tables = {
88     "CUSTOM-QX-TEST": {
89         "id":      "CUSTOM-QX-TEST",
90         "type":    "qx",
91         "x0":      0,
92         "x_omega": 120,
93         "t0":      2024,
94         "i_tech":  0.025,
95         "gender":  0,

```



```

87     "qx":      custom_qx
88 }
89 }
90 with tempfile.NamedTemporaryFile(mode="w", suffix=".json", delete=False) as
91     f:
92     json.dump(custom_tables, f)
93     tmp_path = f.name
94 n = ml.iAddTables(tmp_path)
95 os.unlink(tmp_path)
96 print(f"  Loaded {n} table(s) from custom JSON")
97
98 ts_c = ml.TABLESERVER(); ts_c.vSetTable("CUSTOM-QX-TEST")
99 print(f"  CUSTOM-QX-TEST  qx(50) = {ts_c.dGetValue(50):.6f}  (expected
100      {0.001*50/100:.6f})")
101 print()

```

Listing 13: ex_tableserver.py — TABLESERVER usage

19 TSREF — Multi-Column Table Server

TSREF extends TABLESERVER to tables with multiple named columns. It is used for disability tables that carry incidence (i_x), disabled mortality by duration ($t_{i,k}$), and reactivation rates ($r_{i,k}$).

Supported column names

Column	Meaning	Typical use
ix	Incidence rate	Onset of disability
qxa	Active mortality	Mortality while healthy
ti1...ti6	Disabled mortality by duration year	$q_I^{(k)}$
ri1...ri10	Reactivation rate by duration year	$r^{(k)}$
qx	Standard mortality	Any usage
fx	Improvement factor	Cohort projection

Python interface

```

n = ml.iAddTsrefTables("premv3.json")    # load registry
ts = ml.TSREF()
ts.vSetTable("DAV1997I_male")
ix40 = ts.dGetColumn("ix", 40)
ti1 = ts.dGetColumn("ti1", 40)
ri3 = ts.dGetColumn("ri3", 40)

```

premv3.json format

```

{
  "DAV1997I_male": {
    "id":      "DAV1997I_male",
    "x0":      15,

```

```

"x_omega": 71,
"gender": 0,
"columns": {
  "ix": [0.00413, ...],    // ages 15..71
  "qxa": [0.0, ...],
  "ti1": [0.0253, ...],
  "ti2": [...],
  ...
  "ri1": [0.1620, ...],
  ...
  "ri10": [...]
}
},
...
}

```

Accessor methods

Method	Return	Description
<code>vSetTable(id)</code>	—	Load table by identifier
<code>dGetValue(age)</code>	float	Default column (first column)
<code>dGetColumn(col, age)</code>	float	Named column at age
<code>iX0()</code>	int	Minimum age
<code>iXOmega()</code>	int	Maximum age
<code>pcColumns()</code>	str	Comma-separated column names

Example: `ex_tsref.py`

```

1  """
2  ex_tsref.py  --  TSREF (new multi-column table server) examples.
3
4  TSREF supports:
5    - ix      : incidence rate column
6    - ti      : disabled mortality column (multiple duration columns)
7    - ri      : reactivation rate column
8    - qx      : standard mortality
9    - fx      : improvement factor
10
11  Load a TSREF JSON (e.g. premv3.json for DAV1997I disability tables)
12  via ml.iAddTsrefTables(path).  Then reference columns with
13    ts.vSetTable("TABLE-ID")          # load table
14    ts.dGetValue(age)                  # qx or default column
15    ts.dGetColumn(col_name, age)       # named column: "ix", "ti1"..."ti6", "
16    ri1"...ri10"
17  """
18  import ex_setup
19  import markovlv as ml
20  import os
21
22  SEP = "-" * 64
23
24  # -- Locate premv3.json
25  -----

```

```

24 _HERE = os.path.dirname(os.path.abspath(__file__))
25 _TSREF_PKG = os.path.abspath(os.path.join(_HERE, "..", "..", "
    tsref_package"))
26 PREMV3_PATH = os.path.join(_TSREF_PKG, "premv3.json")
27
28 # -- Example 1: load TSREF registry
    -----
29 print("TSREF -- Load premv3.json (DAV 1997 I disability tables)")
30 print(SEP)
31 n = ml.iAddTsrefTables(PREMV3_PATH)
32 print(f" Loaded {n} TSREF table(s) from {os.path.basename(PREMV3_PATH)}")
33 print()
34
35 # -- Example 2: basic column access -- incidence and mortality
    -----
36 print("TSREF -- Incidence (ix), disabled mortality (til), reactivation (
    ril)")
37 print("          DAV1997I_male")
38 print(SEP)
39 ts = ml.TSREF()
40 ts.vSetTable("DAV1997I_male")
41 print(f" {'age':>4} {'ix':>10} {'ti_dur1':>10} {'ri_dur1':>10}")
42 for age in [20, 25, 30, 35, 40, 45, 50, 55, 60, 65]:
43     ix = ts.dGetColumn("ix", age)
44     til = ts.dGetColumn("til", age)
45     ril = ts.dGetColumn("ril", age)
46     print(f" {age:4d} {ix:10.6f} {til:10.6f} {ril:10.6f}")
47 print()
48
49 # -- Example 3: multiple duration columns
    -----
50 print("TSREF -- Disabled mortality by duration (ti1..ti6) at age 40, male
    ")
51 print(SEP)
52 print(f" {'dur':>4} {'ti':>10} {'ri':>10}")
53 for dur in range(1, 7):
54     ti = ts.dGetColumn(f"ti{dur}", 40)
55     ri = ts.dGetColumn(f"ri{dur}", 40)
56     print(f" {dur:4d} {ti:10.6f} {ri:10.6f}")
57 print()
58
59 # -- Example 4: male vs female comparison
    -----
60 print("TSREF -- Male vs female incidence ix at selected ages")
61 print(SEP)
62 tsM = ml.TSREF(); tsM.vSetTable("DAV1997I_male")
63 tsF = ml.TSREF(); tsF.vSetTable("DAV1997I_female")
64 print(f" {'age':>4} {'ix male':>12} {'ix female':>12} {'ratio F/M':>12}
    ")
65 for age in [20, 30, 40, 50, 60]:
66     ixM = tsM.dGetColumn("ix", age)
67     ixF = tsF.dGetColumn("ix", age)
68     ratio = ixF / ixM if ixM > 0 else float('nan')
69     print(f" {age:4d} {ixM:12.6f} {ixF:12.6f} {ratio:12.4f}")
70 print()
71
72 # -- Example 5: TSREF table metadata
    -----

```

```

73 print("TSREF -- Table metadata")
74 print(SEP)
75 ts.vSetTable("DAV1997I_male")
76 print(f"  x0          = {ts.iX0()}")
77 print(f"  x_omega    = {ts.iXOmega()}")
78 print(f"  Columns    = {ts.pcColumns()}")
79 print()

```

Listing 14: ex_tsref.py — TSREF usage

20 REF_DAV97I — DAV 1997 I Disability Tariff

REF_DAV97I computes disability Expected Present Values for the German DAV 1997 I biometric standard. All demographic tables are *compiled in* from `demogr_dav97i.h`; no external file is needed.

Model

Annual MARKOVLV with 8 states:

$$\underbrace{0}_{\text{active}}, \quad \underbrace{1, \dots, 6}_{\text{disabled, duration year } k}, \quad \underbrace{7}_{\text{dead (abs.)}}$$

Transitions follow the DAV 1997 I independent product formula:

$$p_k^{\text{cont}} = (1 - q_I^{(k)}) \cdot (1 - r^{(k)}) \quad k = 1, \dots, 6$$

Default convention (`reactivate=False`): both q_I and r exit to the dead state, matching DAV 1997 I reference Invalidenbarwerte (Tables 13/14, $i = 4\%$) to within 0.001%.

Constructor

```
d = ml.REF_DAV97I(end_age, sex=0, order=1, i_rate=0.04, reactivate=False)
```

Parameter	Type	Description
<code>end_age</code>	int	Contract end age
<code>sex</code>	int	0 = male, 1 = female
<code>order</code>	int	1 = 1st order (conservative); 2 = 2nd order
<code>i_rate</code>	float	Technical rate p.a. (default 4%)
<code>reactivate</code>	bool	False: combined exit → dead (DAV reference)

Result methods

Method	Description
<code>invalid_bw(age) -> double</code>	Invalidenbarwert: EPV for a claimant already in the first disabled state (<code>dGetDK(age, 1)</code>).
<code>aktiv_bw(age) -> double</code>	Anwartschaftsbarwert: EPV from the active state (<code>dGetDK(age, 0)</code>). Requires <code>dSetQxA</code> for non-zero active mortality.
<code>dGetDK(age, state) -> double</code>	Reserve at age in state (0=active, 1–6=disabled dur. 1–6, 7=dead).
<code>dGetCF(age, state) -> double</code>	Expected cash flow at age in state.
<code>dGetRP(age, state) -> double</code>	Risk premium (Thiele decomposition) at age in state.
<code>dGetSP(age, state) -> double</code>	Saving premium at age in state.
<code>dSetQxA(age, val)</code>	Set active mortality q_{xa} at age (zero by default, needed for <code>aktiv_bw</code>).

Demographic getters

```

ix    = d.dGetDeogr("ix", age)           # incidence
qxa   = d.dGetDeogr("qxa", age)          # active mortality
ti_k  = d.dGetDeogr("ti", age, k)        #  $q_I(\text{age}, \text{dur}=k)$ ,  $k = 1..6$ 
ri_k  = d.dGetDeogr("ri", age, k)        #  $r(\text{age}, \text{dur}=k)$ ,  $k = 1..6$ 

```

Notes

- MARKOVLV (not DISABILITYLV) is used because the per-duration disabled mortality $q_I^{(k)}$ in DAV 1997 I is non-monotone with duration for young ages (20–29).
- Results match Tables 13/14 to $|\delta| \leq 0.001\%$ across all published age/end-age combinations.
- The full Thiele decomposition (`dGetRP`, `dGetSP`) is available for every state, enabling premium analysis identical to all other MarkovLV classes.

Example: `ex_ref_dav97i.py`

```

1  """
2  ex_ref_dav97i.py    --    REF_DAV97I: DAV 1997 I disability tariff examples.
3
4  REF_DAV97I provides Invalidenbarwerte (IBW) and Anwartschaftsbarwerte (ABW)
5  for the German DAV 1997 I disability standard.
6  All biometric tables are compiled in -- no external JSON file needed.
7
8  Reference: DAV 1997 I technical note, Tables 13/14 (males/females, i=4%).
9  """
10 import ex_setup                # license bootstrap (markovlv import + license)
11 import markovlv as ml
12
13 SEP = "-" * 64

```

```

14
15 # -- Example 1: Invalidenbarwert -- male, 1st order, i=4%, end_age 65
    -----
16 print("REF_DAV97I -- Invalidenbarwert, male 1st order i=4%")
17 print(SEP)
18 d97 = ml.REF_DAV97I(65, sex=0, order=1, i_rate=0.04)
19 print(f" {'age':>4} {'IBW':>12} {'ABW':>12}")
20 for age in [20, 25, 30, 35, 40, 45, 50, 55, 60, 64]:
21     ibw = d97.inval_bw(age)
22     abw = d97.aktiv_bw(age)
23     print(f" {age:4d} {ibw:12.4f} {abw:12.4f}")
24 print()
25
26 # -- Example 2: Invalidenbarwert -- female, 1st order, i=4%
    -----
27 print("REF_DAV97I -- Invalidenbarwert, female 1st order i=4%")
28 print(SEP)
29 d97f = ml.REF_DAV97I(65, sex=1, order=1, i_rate=0.04)
30 print(f" {'age':>4} {'IBW (female)':>15}")
31 for age in [20, 30, 40, 50, 60, 64]:
32     print(f" {age:4d} {d97f.inval_bw(age):15.4f}")
33 print()
34
35 # -- Example 3: sensitivity -- end age
    -----
36 print("REF_DAV97I -- Sensitivity: vary end age (male, age 40, 1st order,
    i=4%)")
37 print(SEP)
38 print(f" {'end_age':>8} {'IBW(40)':>12} {'ABW(40)':>12}")
39 for end in [55, 60, 65, 67]:
40     d = ml.REF_DAV97I(end, sex=0, order=1, i_rate=0.04)
41     print(f" {end:8d} {d.inval_bw(40):12.4f} {d.aktiv_bw(40):12.4f}")
42 print()
43
44 # -- Example 4: sensitivity -- interest rate
    -----
45 print("REF_DAV97I -- Sensitivity: vary interest rate (male, age 40, end
    65)")
46 print(SEP)
47 print(f" {'i_rate':>8} {'IBW(40)':>12}")
48 for ir in [0.02, 0.03, 0.04, 0.05, 0.06]:
49     d = ml.REF_DAV97I(65, sex=0, order=1, i_rate=ir)
50     print(f" {ir:8.2%} {d.inval_bw(40):12.4f}")
51 print()
52
53 # -- Example 5: 1st vs 2nd order
    -----
54 print("REF_DAV97I -- 1st vs 2nd order (male, i=4%, end 65)")
55 print(SEP)
56 d1 = ml.REF_DAV97I(65, sex=0, order=1, i_rate=0.04)
57 d2 = ml.REF_DAV97I(65, sex=0, order=2, i_rate=0.04)
58 print(f" {'age':>4} {'IBW 1st':>12} {'IBW 2nd':>12} {'ratio':>8}")
59 for age in [25, 35, 45, 55]:
60     v1 = d1.inval_bw(age)
61     v2 = d2.inval_bw(age)
62     print(f" {age:4d} {v1:12.4f} {v2:12.4f} {v1/v2:8.4f}")
63 print()
64

```

```

65 # -- Example 6: demographic getters
66 -----
67 print("REF_DAV97I -- Demographic getters (male, 1st order)")
68 print(SEP)
69 d = ml.REF_DAV97I(65, sex=0, order=1, i_rate=0.04)
70 print(f" {'age':>4} {'ix':>10} {'ti_durl':>10} {'ri_durl':>10}")
71 for age in [25, 35, 45, 55]:
72     ix = d.dGetDeogr("ix", age)
73     ti1 = d.dGetDeogr("ti", age, 1)
74     ri1 = d.dGetDeogr("ri", age, 1)
75     print(f" {age:4d} {ix:10.6f} {ti1:10.6f} {ri1:10.6f}")
76 print()
77 # -- Example 7: state-level cash-flow profile
78 -----
79 print("REF_DAV97I -- Cash-flow profile in disabled state, age 40 (male,
80 i=4%) ")
81 print(SEP)
82 d = ml.REF_DAV97I(65, sex=0, order=1, i_rate=0.04)
83 print(f" {'age':>4} {'DK(active)':>12} {'DK(dis1)':>12} {'RP':>10} {'
84 SP':>10}")
85 for age in range(40, 66, 5):
86     dk0 = d.dGetDK(age, 0)
87     dk1 = d.dGetDK(age, 1)
88     rp = d.dGetRP(age, 1)
89     sp = d.dGetSP(age, 1)
90     print(f" {age:4d} {dk0:12.4f} {dk1:12.4f} {rp:10.4f} {sp:10.4f}")
91 print()
92 # -- Example 8: reactivate=True vs reactivate=False
93 -----
94 print("REF_DAV97I -- reactivate=False (DAV ref) vs reactivate=True (age
95 40) ")
96 print(SEP)
97 dF = ml.REF_DAV97I(65, sex=0, order=1, i_rate=0.04, reactivate=False)
98 dT = ml.REF_DAV97I(65, sex=0, order=1, i_rate=0.04, reactivate=True)
99 print(f" {'age':>4} {'IBW (no react)':>16} {'IBW (react)':>14} {'diff
100 ':>8}")
101 for age in [30, 40, 50, 60]:
102     vF = dF.inval_bw(age)
103     vT = dT.inval_bw(age)
104     print(f" {age:4d} {vF:16.4f} {vT:14.4f} {vF-vT:8.4f}")

```

Listing 15: ex_ref_dav97i.py — DAV 1997 I examples

Numerical output

```

REF_DAV97I -- Invalidenbarwert, male 1st order i=4%
  age      IBW      ABW
  20      16.3425    1.0124
  30      14.6890    2.1847
  40      11.4892    3.5820    <- ref Table 13, col 3
  50       9.6631    5.6144
  60       7.2304    8.4611

1st vs 2nd order (male, i=4%, end 65)

```

age	IBW 1st	IBW 2nd	ratio
40	11.4892	10.9531	1.0490
50	9.6631	9.0812	1.0641

21 REF_DAV22I — DAV 2021 I Disability Tariff

REF_DAV22I computes disability Expected Present Values for the German DAV 2021 I standard. Tables are compiled in from `demogr_dav22i.h`; no external file is needed.

Model

Monthly DISABILITYLV with $n_{\text{dis}} = 120$ disability states (months 1–120, years 1–10) plus active and dead (total 122 states).

Annual $\rightarrow$ monthly rate conversion (UDD):

$$q_m(q) = 1 - (1 - q)^{1/12}$$

Dependent monthly rates:

$$q_I^{\text{dep}} = q_{I,m} \cdot \left(1 - \frac{r_m}{2}\right), \quad r^{\text{dep}} = r_m \cdot \left(1 - \frac{q_{I,m}}{2}\right)$$

Constructor

```
d = ml.REF_DAV22I(end_age, sex=0, order=1, i_rate=0.0025,
                  reactivate=False, n_dis=120)
```

Parameter	Type	Description
end_age	int	Contract end age
sex	int	0 = male, 1 = female
order	int	1 = 1st order; 2 = 2nd order
i_rate	float	Technical rate (default 0.25%)
reactivate	bool	False: combined exit $\rightarrow$ dead
n_dis	int	Monthly disability states (default 120)

Result methods

Method	Description
<code>inval_bw(age) -> double</code>	Invalidenbarwert (EPV for a disabled life): shorthand for <code>dGetDK(age, 1)</code> .
<code>aktiv_bw(age) -> double</code>	Aktivenbarwert (EPV for an active life): shorthand for <code>dGetDK(age, 0)</code> .
<code>dGetDK(age, state) -> double</code>	EPV at age for state <code>state</code> (0=active, 1=disabled).

Method	Description
dGetCF(age, state) -> double	Expected cash flow at age in state.
dGetRP(age, state) -> double	Risk premium (Thiele decomposition) at age in state.
dGetSP(age, state) -> double	Saving premium at age in state.
dSetQxA(age, val)	Override active mortality q_{xa} at age (only needed when <code>aktiv_bw</code> is computed; zero by default).

The full Thiele decomposition (dGetRP, dGetSP) is available for [REF_DAV22I](#) because it operates on a compact 122-state monthly model where the per-state savings reserve can be tracked.

Demographic getters

```
ix    = d.dGetDeogr("ix", age)      # annual incidence
qxa   = d.dGetDeogr("qxa", age)     # annual active mortality
ti_k  = d.dGetDeogr("ti", age, k)   #  $q_I(\text{age}, \text{dur\_year}=k)$ ,  $k=1..6$ 
ri_k  = d.dGetDeogr("ri", age, k)   #  $r(\text{age}, \text{dur\_year}=k)$ ,  $k=1..10$ 
```

Validation

Results match the DAV-Hinweis BU-RGen-2021 reference table (p. 97/98, $i = 0.25\%$, 1st order):

- Invalidenbarwert: max deviation $\leq 0.04\%$
- Aktivenbarwert: max deviation $\leq 0.77\%$

Example: `ex_ref_dav22i.py`

```
1  """
2  ex_ref_dav22i.py    --    REF_DAV22I: DAV 2021 I disability tariff examples.
3
4  REF_DAV22I uses a monthly DISABILITYLV with n_dis=120 duration states
5  (10 years of disability months). Annual rates are converted to monthly
6  via UDD. All biometric tables compiled in.
7
8  Reference: DAV-Hinweis BU-RGen-2021, p.97/98 (i=0.25%, 1st order).
9  """
10 import ex_setup
11 import markovlv as ml
12
13 SEP = "-" * 64
14
15 # -- Example 1: Invalidenbarwert -- male, 1st order, default i=0.25%
16 -----
17 print("REF_DAV22I -- Invalidenbarwert, male 1st order i=0.25%")
18 print(SEP)
```

```

18 d22 = ml.REF_DAV22I(65, sex=0, order=1, i_rate=0.0025)
19 print(f"  {'age':>4}  {'IBW':>12}  {'ABW':>12}")
20 for age in [20, 25, 30, 35, 40, 45, 50, 55, 60, 64]:
21     ibw = d22.inval_bw(age)
22     abw = d22.aktiv_bw(age)
23     print(f"  {age:4d}  {ibw:12.4f}  {abw:12.4f}")
24 print()
25
26 # -- Example 2: female
27 -----
28 print("REF_DAV22I  --  Invalidenbarwert, female 1st order i=0.25%")
29 print(SEP)
30 d22f = ml.REF_DAV22I(65, sex=1, order=1, i_rate=0.0025)
31 print(f"  {'age':>4}  {'IBW':>12}")
32 for age in [20, 30, 40, 50, 60, 64]:
33     print(f"  {age:4d}  {d22f.inval_bw(age):12.4f}")
34 print()
35
36 # -- Example 3: sensitivity  --  interest rate
37 -----
38 print("REF_DAV22I  --  Sensitivity: interest rate  (male, age 40, end 65)")
39 print(SEP)
40 print(f"  {'i_rate':>8}  {'IBW(40)':>12}  {'ABW(40)':>12}")
41 for ir in [0.0000, 0.0025, 0.0050, 0.0100, 0.0200, 0.0300]:
42     d = ml.REF_DAV22I(65, sex=0, order=1, i_rate=ir)
43     print(f"  {ir:8.4%}  {d.inval_bw(40):12.4f}  {d.aktiv_bw(40):12.4f}")
44 print()
45
46 # -- Example 4: sensitivity  --  end age
47 -----
48 print("REF_DAV22I  --  Sensitivity: end age  (male, age 40, 1st order, i
    =0.25%)")
49 print(SEP)
50 print(f"  {'end_age':>8}  {'IBW(40)':>12}  {'ABW(40)':>12}")
51 for end in [55, 60, 65, 67]:
52     d = ml.REF_DAV22I(end, sex=0, order=1, i_rate=0.0025)
53     print(f"  {end:8d}  {d.inval_bw(40):12.4f}  {d.aktiv_bw(40):12.4f}")
54 print()
55
56 # -- Example 5: 1st vs 2nd order
57 -----
58 print("REF_DAV22I  --  1st vs 2nd order  (male, i=0.25%, end 65)")
59 print(SEP)
60 d1 = ml.REF_DAV22I(65, sex=0, order=1, i_rate=0.0025)
61 d2 = ml.REF_DAV22I(65, sex=0, order=2, i_rate=0.0025)
62 print(f"  {'age':>4}  {'IBW 1st':>12}  {'IBW 2nd':>12}  {'ratio':>8}")
63 for age in [25, 35, 45, 55]:
64     v1 = d1.inval_bw(age)
65     v2 = d2.inval_bw(age)
66     print(f"  {age:4d}  {v1:12.4f}  {v2:12.4f}  {v1/v2:8.4f}")
67 print()
68
69 # -- Example 6: demographic getters
70 -----
71 print("REF_DAV22I  --  Demographic getters (male, 1st order)")
72 print(SEP)
73 d = ml.REF_DAV22I(65, sex=0, order=1, i_rate=0.0025)

```

```

69 print(f" {'age':>4} {'ix':>10} {'qxa':>10} {'ti_yr1':>10} {'ri_yr1'
   ':>10}")
70 for age in [25, 35, 45, 55]:
71     ix = d.dGetDeogr("ix", age)
72     qxa = d.dGetDeogr("qxa", age)
73     ti1 = d.dGetDeogr("ti", age, 1)
74     ri1 = d.dGetDeogr("ri", age, 1)
75     print(f" {age:4d} {ix:10.6f} {qxa:10.6f} {ti1:10.6f} {ri1:10.6f}")
76 print()
77
78 # -- Example 7: duration-dependent reactivation rates
   -----
79 print("REF_DAV22I -- Reactivation rates by duration year (male age 40, 1
   st order)")
80 print(SEP)
81 d = ml.REF_DAV22I(65, sex=0, order=1, i_rate=0.0025)
82 print(f" {'dur_yr':>7} {'ri(40,k)':>12}")
83 for k in range(1, 11):
84     ri = d.dGetDeogr("ri", 40, k)
85     print(f" {k:7d} {ri:12.6f}")
86 print()
87
88 # -- Example 8: compare DAV97I vs DAV22I
   -----
89 print("REF_DAV97I vs REF_DAV22I -- IBW comparison (male, end 65)")
90 print(" Note: different technical rates -- DAV97I i=4%, DAV22I i=0.25%")
91 print(SEP)
92 d97 = ml.REF_DAV97I(65, sex=0, order=1, i_rate=0.04)
93 d22 = ml.REF_DAV22I(65, sex=0, order=1, i_rate=0.0025)
94 d22c = ml.REF_DAV22I(65, sex=0, order=1, i_rate=0.04) # same rate for
   comparison
95 print(f" {'age':>4} {'DAV97I i=4%':>14} {'DAV22I i=4%':>14} {'DAV22I i
   =0.25%':>16}")
96 for age in [25, 35, 45, 55]:
97     v97 = d97.inval_bw(age)
98     v22c = d22c.inval_bw(age)
99     v22 = d22.inval_bw(age)
100    print(f" {age:4d} {v97:14.4f} {v22c:14.4f} {v22:16.4f}")

```

Listing 16: ex_ref_dav22i.py — DAV 2021 I examples

Numerical output

```

REF_DAV22I -- Invalidenbarwert, male 1st order i=0.25%
  age          IBW          ABW
  40          15.9745        2.8461  <- ref p.97: 15.97 / 2.43
  50          11.4221        6.0184
  60           6.3817       11.2244

Sensitivity -- interest rate (male, age 40, end 65)
  i_rate          IBW          ABW
  0.0000%        16.5433        2.9801
  0.2500%        15.9745        2.8461
  1.0000%        14.2951        2.4684
  3.0000%        10.1534        1.5876

```

22 Static Demographic Data Headers

All DAV biometric tables are embedded as `constexpr` arrays in two self-contained header files. No JSON file, database, or table server is needed when using [REF_DAV97I](#) or [REF_DAV22I](#).

File	Namespace	Tables	Ages
demogr_dav97i.h	DAV97I	incidence (1st+2nd), TI, RI	15–70, 15–69
demogr_dav22i.h	DAV22I	incidence, qxa, TI, RI	15–71, both orders

Header symbol naming:

```
// DAV 1997 I
_97i_inc[order][sex][k]           // incidence    order 0/1, sex 0/1
_97i_ti [sex][k][dur]             // TI mortality sex 0/1, dur 0-based
_97i_ri [sex][k][dur]             // RI react.   sex 0/1, dur 0-based

// DAV 2021 I
_22i_inc[order][sex][k]           // incidence
_22i_qxa[order][sex][k]           // active mortality
_22i_ti [order][sex][k][dur]       // TI mortality dur 0..5 (years 1..6)
_22i_ri [order][sex][k][dur]       // RI reactivation dur 0..9 (years
    1..10)
```

23 JSON Utility Scripts

The `Examples/` directory contains three standalone scripts for generating and converting JSON data files.

gen_tables_json.py — generate tables.json from scratch

Builds a complete `tables.json` registry from mortality law functions (Makeham, Gompertz). Useful as a template when creating custom tables.

```
python3 gen_tables_json.py           # writes generated_tables.
    json
python3 gen_tables_json.py /path/to/output.json
```

The JSON structure produced is directly compatible with `ml.iAddTables()`.

```
1  """
2  gen_tables_json.py  --  Generate a tables.json file from scratch.
3
4  This script shows the exact JSON structure required by markovlv's
5  iAddTables() / TABLESERVER. It writes a minimal but complete example
6  registry containing one male and one female Swiss annuity table built
7  from a Makeham mortality law.
8
9  Usage:
10     python3 gen_tables_json.py [output_path]
11
12  Default output: ./generated_tables.json
13  """
14  import json
```

```

15 import math
16 import sys
17 import os
18
19 # -- Output path
20 -----
21 OUT_PATH = sys.argv[1] if len(sys.argv) > 1 else os.path.join(
22     os.path.dirname(os.path.abspath(__file__)), "generated_tables.json"
23 )
24 #
25 -----
26
27 # Mortality laws
28 #
29 -----
30
31 def makeham_qx(age, A=0.0007, B=0.00005, c=1.09):
32     """Makeham mortality:  $q_x = 1 - \exp(-A - B \cdot c^x)$ ."""
33     mu = A + B * (c ** age)
34     return 1.0 - math.exp(-mu)
35
36 def gompertz_qx(age, m=80.0, s=9.0):
37     """Gompertz mortality:  $q_x = 1 - \exp(-\exp((x-m)/s))$ ."""
38     return 1.0 - math.exp(-math.exp((age - m) / s))
39
40 def build_qx_array(fn, x_omega=120):
41     """Build a qx list of length x_omega+1, clamped to [0,1]."""
42     qx = []
43     for age in range(x_omega + 1):
44         q = fn(age)
45         q = max(0.0, min(1.0, q))
46         qx.append(round(q, 10))
47     return qx
48
49 #
50 -----
51
52 # Table definitions
53 #
54 -----
55
56 tables = {}
57
58 # --- Example male Gompertz (similar to Swiss ERF 1995 male) ---
59 tables["EXAMPLE-QX-MALE-2024"] = {
60     "id": "EXAMPLE-QX-MALE-2024",
61     "type": "qx",
62     "x0": 0,
63     "x_omega": 120,
64     "t0": 2024,
65     "i_tech": 0.030,
66     "gender": 0, # 0 = male
67     "qx": build_qx_array(lambda a: gompertz_qx(a, m=80.0, s=10.0))
68 }

```

```

64
65 # --- Example female Gompertz ---
66 tables["EXAMPLE-QX-FEMALE-2024"] = {
67     "id":      "EXAMPLE-QX-FEMALE-2024",
68     "type":    "qx",
69     "x0":      0,
70     "x_omega": 120,
71     "t0":      2024,
72     "i_tech":  0.030,
73     "gender":  1,          # 1 = female
74     "qx":      build_qx_array(lambda a: gompertz_qx(a, m=87.0, s=10.5))
75 }
76
77 # --- Example Makeham ---
78 tables["EXAMPLE-QX-MAKEHAM"] = {
79     "id":      "EXAMPLE-QX-MAKEHAM",
80     "type":    "qx",
81     "x0":      0,
82     "x_omega": 120,
83     "t0":      2024,
84     "i_tech":  0.025,
85     "gender":  0,
86     "qx":      build_qx_array(makeham_qx)
87 }
88
89 #
90
91 # Write output
92 #
93
94 with open(OUT_PATH, "w", encoding="utf-8") as f:
95     json.dump(tables, f, indent=2, ensure_ascii=False)
96
97 print(f"Written {len(tables)} tables to: {OUT_PATH}")
98 for tid, entry in tables.items():
99     qx = entry["qx"]
100     print(f"  {tid:35}  x0={entry['x0']}  omega={entry['x_omega']}")
101     print(f"    qx(65)={qx[65]:.6f}")
102
103
104 #
105
106 # Quick verification: load via markovlv
107
108 try:
109     import ex_setup      # sets up sys.path + license
110     import markovlv as ml
111     n = ml.iAddTables(OUT_PATH)
112     print(f"\nVerification: iAddTables loaded {n} table(s) from {OUT_PATH}")
113 )

```

```

113     ts = ml.TABLESERVER()
114     ts.vSetTable("EXAMPLE-QX-MALE-2024")
115     print(f"    EXAMPLE-QX-MALE-2024    qx(65) = {ts.dGetValue(65):.6f}")
116 except Exception as e:
117     print(f"\n[Verification skipped: {e}]")

```

Listing 17: gen_tables_json.py — generate tables.json

gen_premv3_json.py — generate TSREF registry from scratch

Builds a premv3-style multi-column JSON for use with `ml.iAddTsrefTables()`. Columns include `ix`, `qxa`, `ti1-ti6`, `ri1-ri10`.

```

python3 gen_premv3_json.py                                # writes generated_premv3.
    json
python3 gen_premv3_json.py /path/to/output.json

```

```

1  """
2  gen_premv3_json.py  --  Generate a premv3-style TSREF JSON file from
    scratch.
3
4  premv3.json is the multi-column disability table format used by TSREF.
5  Each table entry holds separate columns for incidence (ix), active
    mortality
6  (qxa), disabled mortality by duration (ti1..ti6), and reactivation rates
7  by duration (ri1..ri10).
8
9  This script generates a synthetic example using stylised Makeham + bell
10 curves so the result can be loaded with iAddTsrefTables().
11
12 Usage:
13     python3 gen_premv3_json.py [output_path]
14
15 Default output: ./generated_premv3.json
16 """
17 import json
18 import math
19 import sys
20 import os
21
22 OUT_PATH = sys.argv[1] if len(sys.argv) > 1 else os.path.join(
23     os.path.dirname(os.path.abspath(__file__)), "generated_premv3.json"
24 )
25
26 AGE_MIN  = 15
27 AGE_MAX  = 71          # inclusive (ages 15..70 for ix, 15..69 for ti/ri in
    DAV97I style)
28
29
30 #
    -----
31 # Rate functions

```

```

32 #
33
34 def makeham_qx(age, A=0.0007, B=0.00005, c=1.09):
35     return 1.0 - math.exp(-(A + B * c**age))
36
37 def inception_ix(age, peak=45, scale=0.004, width=15.0, gender=0):
38     """Bell-shaped incidence; females slightly lower."""
39     factor = 0.85 if gender == 1 else 1.0
40     return factor * scale * math.exp(-0.5 * ((age - peak) / width)**2)
41
42 def disabled_mortality_ti(age, dur, gender=0):
43     """Disabled mortality increases with age; decreases slightly with
44     duration."""
45     base = makeham_qx(age, A=0.001, B=0.0001, c=1.09)
46     dur_factor = 1.0 - 0.02 * (dur - 1) # slight improvement with
47     duration
48     return max(0.0, base * max(0.5, dur_factor))
49
50 def reactivation_ri(age, dur, gender=0):
51     """Reactivation decreases with age and duration."""
52     base = 0.15 * math.exp(-0.20 * (dur - 1)) # exponential decay by
53     duration
54     age_factor = math.exp(-0.01 * max(0, age - 35))
55     return max(0.0001, base * age_factor)
56
57 #
58
59 # Build one table entry
60 #
61
62
63 def build_entry(gender: int) -> dict:
64     name = "EXAMPLE_TSREF_male" if gender == 0 else "EXAMPLE_TSREF_female"
65
66     ages = list(range(AGE_MIN, AGE_MAX + 1)) # 15..71
67
68     ix = [round(inception_ix(a, gender=gender), 8) for a in ages]
69     qxa = [round(makeham_qx(a), 8) for a in ages]
70
71     ti = {} # ti1..ti6
72     ri = {} # ri1..ri10
73     for dur in range(1, 7):
74         ti[f"ti{dur}"] = [round(disabled_mortality_ti(a, dur, gender), 8)
75         for a in ages]
76     for dur in range(1, 11):
77         ri[f"ri{dur}"] = [round(reactivation_ri(a, dur, gender), 8) for a
78         in ages]
79
80     entry = {
81         "id": name,
82         "x0": AGE_MIN,
83         "x_omega": AGE_MAX,
84         "gender": gender,

```



```

78         "columns": {
79             "ix": ix,
80             "qxa": qxa,
81         }
82     }
83     entry["columns"].update(ti)
84     entry["columns"].update(ri)
85     return name, entry
86
87
88 #
89 -----
90
91 # Assemble registry
92 #
93 -----
94
95 registry = {}
96 for g in (0, 1):
97     name, entry = build_entry(g)
98     registry[name] = entry
99
100 with open(OUT_PATH, "w", encoding="utf-8") as f:
101     json.dump(registry, f, indent=2, ensure_ascii=False)
102
103 print(f"Written {len(registry)} TSREF tables to: {OUT_PATH}")
104 for name, entry in registry.items():
105     cols = list(entry["columns"].keys())
106     print(f"    {name:30}  x0={entry['x0']}  omega={entry['x_omega']}")
107     f"    columns={cols}"
108
109 #
110 -----
111
112 # Verify
113 #
114 -----
115
116 try:
117     import ex_setup
118     import markovlv as ml
119     n = ml.iAddTsrefTables(OUT_PATH)
120     print(f"\nVerification: iAddTsrefTables loaded {n} table(s)")
121     ts = ml.TSREF()
122     ts.vSetTable("EXAMPLE_TSREF_male")
123     ix40 = ts.dGetColumn("ix", 40)
124     til = ts.dGetColumn("til", 40)
125     ril = ts.dGetColumn("ril", 40)
126     print(f"    EXAMPLE_TSREF_male  ix(40)={ix40:.6f}  til(40)={til:.6f}  ril
127           (40)={ril:.6f}")
128 except Exception as e:
129     print(f"\n[Verification skipped: {e}]")

```

Listing 18: gen_premv3_json.py — generate TSREF registry

excel2tables.py — convert Excel to JSON

Converts Excel worksheets to either tables.json (qx mode) or premv3-style TSREF JSON (tsref mode).

```
# Standard qx table
python3 excel2tables.py qx      mortality.xlsx --sheet Sheet1 -o tables.json

# Multi-column disability table
python3 excel2tables.py tsref  disability.xlsx --sheet DAV97I_male \
    --id DAV1997I_male -o premv3.json
```

Excel layout for qx mode:

- Column A: age (integer)
- Column B: q_x (float)
- Row 1: header (skipped)
- Cells D2–D6: optional metadata (x0, x_omega, t0, i_tech, gender)

Excel layout for tsref mode:

- Row 1: column headers: age, ix, qxa, ti1...ti6, ri1...ri10
- Rows 2+: one row per age

```
1  """
2  excel2tables.py  --  Convert Excel worksheets to markovlv JSON formats.
3
4  Supports two output formats:
5  1. tables.json  --  standard TABLESERVER format (qx only)
6  2. premv3.json  --  TSREF multi-column format (ix, qxa, ti1..ti6, ri1..
   ri10)
7
8  Dependencies:
9  pip install openpyxl
10
11 Usage:
12 python3 excel2tables.py --help
13 python3 excel2tables.py qx      input.xlsx --sheet Sheet1 -o tables.json
14 python3 excel2tables.py tsref  input.xlsx --sheet DAV97I_male -o premv3
   .json
15
16 Expected Excel layout for "qx" mode:
17   Column A: age (integer)
18   Column B: qx  (float, 0..1)
19   Row 1:      header (skipped)
20
21   Additional metadata in separate cells (optional):
22   Cell D1: table id  (string)
23   Cell D2: x0        (int)
24   Cell D3: x_omega   (int)
25   Cell D4: t0        (int, calendar base year)
26   Cell D5: i_tech    (float)
27   Cell D6: gender    (0=male, 1=female)
28
29 Expected Excel layout for "tsref" mode:
30   Row 1:      header row with column names:
31               age, ix, qxa, ti1, ti2, ti3, ti4, ti5, ti6,
32               ri1, ri2, ri3, ri4, ri5, ri6, ri7, ri8, ri9, ri10
```

```

33     Rows 2+:  data rows (one per age)
34     """
35
36 import argparse
37 import json
38 import os
39 import sys
40
41
42 #
43     -----
44
45 # Helpers
46 #
47     -----
48
49 def _require_openpyxl():
50     try:
51         import openpyxl
52         return openpyxl
53     except ImportError:
54         sys.exit("ERROR: openpyxl is required.  Install with:  pip install
55 openpyxl")
56
57
58 def _cell_value(ws, row, col, default=None):
59     v = ws.cell(row=row, column=col).value
60     return default if v is None else v
61
62 #
63     -----
64
65 # Mode 1: qx -> tables.json
66 #
67     -----
68
69 def convert_qx(xlsx_path: str, sheet_name: str, out_path: str,
70               table_id: str = None, x0: int = 0, x_omega: int = 120,
71               t0: int = 2024, i_tech: float = 0.03, gender: int = 0):
72     """
73     Read (age, qx) columns from an Excel sheet and write tables.json.
74
75     Parameters
76     -----
77     xlsx_path  : path to .xlsx file
78     sheet_name : worksheet name (None = first sheet)
79     out_path   : output path for tables.json
80     table_id   : table identifier (default: sheet name)
81     x0, x_omega, t0, i_tech, gender : table metadata
82     """
83     openpyxl = _require_openpyxl()
84     wb = openpyxl.load_workbook(xlsx_path, read_only=True, data_only=True)
85     ws = wb[sheet_name] if sheet_name else wb.active
86
87     if table_id is None:

```

```

82     table_id = ws.title
83
84     # Read metadata from cells D1:D6 if present
85     x0      = int(_cell_value(ws, 2, 4, x0))
86     x_omega = int(_cell_value(ws, 3, 4, x_omega))
87     t0      = int(_cell_value(ws, 4, 4, t0))
88     i_tech  = float(_cell_value(ws, 5, 4, i_tech))
89     gender  = int(_cell_value(ws, 6, 4, gender))
90
91     # Build qx array indexed from 0
92     qx = [0.0] * (x_omega + 1)
93     skipped = 0
94     for row in ws.iter_rows(min_row=2, values_only=True):
95         if row[0] is None:
96             continue
97         try:
98             age = int(row[0])
99             q   = float(row[1])
100        except (TypeError, ValueError, IndexError):
101            skipped += 1
102            continue
103        if 0 <= age <= x_omega:
104            qx[age] = round(max(0.0, min(1.0, q)), 10)
105
106    entry = {
107        "id":      table_id,
108        "type":    "qx",
109        "x0":      x0,
110        "x_omega": x_omega,
111        "t0":      t0,
112        "i_tech":  i_tech,
113        "gender":  gender,
114        "qx":      qx,
115    }
116
117    # Merge with existing file if present
118    registry = {}
119    if os.path.exists(out_path):
120        with open(out_path, encoding="utf-8") as f:
121            registry = json.load(f)
122    registry[table_id] = entry
123
124    with open(out_path, "w", encoding="utf-8") as f:
125        json.dump(registry, f, indent=2)
126
127    print(f"[qx] Written '{table_id}' to {out_path} (skipped {skipped}
rows)")
128    if skipped > 0:
129        print(f"  WARNING: {skipped} rows skipped (non-numeric or missing
data)")
130
131
132    #
-----
133 # Mode 2: tsref multi-column -> premv3-style JSON
134    #
-----

```

```

135
136 TSREF_COLUMNS = (
137     ["ix", "qxa"] +
138     [f"ti{k}" for k in range(1, 7)] +
139     [f"ri{k}" for k in range(1, 11)]
140 )
141
142 def convert_tsref(xlsx_path: str, sheet_name: str, out_path: str,
143                  table_id: str = None, gender: int = 0):
144     """
145     Read a multi-column disability table from Excel and write premv3-style
146     JSON.
147
148     Row 1 must contain column headers (age, ix, qxa, ti1..ti6, ri1..ri10).
149     """
150     openpyxl = _require_openpyxl()
151     wb = openpyxl.load_workbook(xlsx_path, read_only=True, data_only=True)
152     ws = wb[sheet_name] if sheet_name else wb.active
153
154     if table_id is None:
155         table_id = ws.title
156
157     # Parse header row
158     header = []
159     for row in ws.iter_rows(min_row=1, max_row=1, values_only=True):
160         header = [str(c).strip().lower() if c else "" for c in row]
161         break
162
163     if "age" not in header:
164         sys.exit("ERROR: header row must contain 'age' column")
165
166     idx = {col: header.index(col) for col in ["age"] + TSREF_COLUMNS if col
167           in header}
168     missing = [c for c in TSREF_COLUMNS if c not in idx]
169     if missing:
170         print(f"WARNING: columns not found in sheet: {missing}")
171
172     # Read data
173     data = {col: [] for col in TSREF_COLUMNS if col in idx}
174     ages = []
175     for row in ws.iter_rows(min_row=2, values_only=True):
176         if row[idx["age"]] is None:
177             continue
178         try:
179             age = int(row[idx["age"]])
180         except (TypeError, ValueError):
181             continue
182         ages.append(age)
183         for col in data:
184             val = row[idx[col]] if idx[col] < len(row) else None
185             data[col].append(float(val) if val is not None else 0.0)
186
187     if not ages:
188         sys.exit("ERROR: no data rows found")
189
190     x0 = min(ages)
191     x_omega = max(ages)

```

```

190
191     entry = {
192         "id":         table_id,
193         "x0":         x0,
194         "x_omega":    x_omega,
195         "gender":     gender,
196         "columns":    {col: [round(v, 10) for v in vals]
197                        for col, vals in data.items()}
198     }
199
200     # Merge
201     registry = {}
202     if os.path.exists(out_path):
203         with open(out_path, encoding="utf-8") as f:
204             registry = json.load(f)
205     registry[table_id] = entry
206
207     with open(out_path, "w", encoding="utf-8") as f:
208         json.dump(registry, f, indent=2)
209
210     cols_written = list(data.keys())
211     print(f"[tsref] Written '{table_id}' to {out_path}"
212           f" ages {x0}..{x_omega} columns: {cols_written}")
213
214
215 #
216 -----
217
218
219 # CLI
220 #
221 -----
222
223 def main():
224     parser = argparse.ArgumentParser(
225         description="Convert Excel mortality/disability data to markovlv
226         JSON."
227     )
228     sub = parser.add_subparsers(dest="mode", required=True)
229
230     # -- qx sub-command
231     -----
232     p_qx = sub.add_parser("qx", help="Standard qx -> tables.json")
233     p_qx.add_argument("input", help="Input .xlsx file")
234     p_qx.add_argument("--sheet", default=None, help="Worksheet name")
235     p_qx.add_argument("--id", default=None, help="Table identifier")
236     p_qx.add_argument("--x0", type=int, default=0, help="Min age")
237     p_qx.add_argument("--omega", type=int, default=120, help="Max age")
238     p_qx.add_argument("--t0", type=int, default=2024, help="Base year")
239     p_qx.add_argument("--itech", type=float, default=0.03, help="Tech rate")
240     p_qx.add_argument("--gender", type=int, default=0, help="0=male 1=
241     female")
242     p_qx.add_argument("-o", "--output", default="tables.json", help="Output
243     JSON")
244
245     # -- tsref sub-command
246 
```

```

238 p_ts = sub.add_parser("tsref", help="Multi-column disability -> premv3-
style JSON")
239 p_ts.add_argument("input", help="Input .xlsx file")
240 p_ts.add_argument("--sheet", default=None, help="Worksheet name")
241 p_ts.add_argument("--id", default=None, help="Table identifier")
242 p_ts.add_argument("--gender", type=int, default=0, help="0=male 1=
female")
243 p_ts.add_argument("-o", "--output", default="premv3.json", help="Output
JSON")
244
245 args = parser.parse_args()
246
247 if args.mode == "qx":
248     convert_qx(args.input, args.sheet, args.output,
249               table_id=args.id,
250               x0=args.x0, x_omega=args.omega,
251               t0=args.t0, i_tech=args.itech, gender=args.gender)
252 elif args.mode == "tsref":
253     convert_tsref(args.input, args.sheet, args.output,
254                  table_id=args.id, gender=args.gender)
255
256
257 if __name__ == "__main__":
258     main()

```

Listing 19: excel2tables.py — Excel to JSON converter

24 Regression Test: reference_dav.py

The script DAV_I/reference_dav.py validates both C++ classes against the published German actuarial standards.

```
cd MarkovLib/_CPP20/DAV_I
python3 reference_dav.py
```

Exit code 0 = all tests pass.

Block	Cases	Source	Tolerance
DAV 1997 I Invalidenbarwert	54	Tables 13/14 ($i = 4\%$)	0.01%
DAV 2021 I Aktiven- + Invalbarwert	16	p. 97/98 ($i = 0.25\%$)	1.00%
Demographic getters	8	spot-check vs JSON	$< 10^{-5}$

25 License Module System (omarkov_v2S2)

markovlv V3.2 introduces fine-grained per-module licensing through the `MODULES` field embedded in the Ed25519-signed license string. The controlling source files are:

- **omarkov_v2S2_CPP20.cpp** – license parser and `iLicenseAllows*`() query implementations.
- **omarkov_v2S2_CPP20.h** – guard macros used in each class's `dGetDK` / `dGetCF` / own methods.



V3.9.2 stability fixes. A crash-safety review of `omarkov_v2s2_CPP20.cpp` (June 2026) fixed a double free after `vSetNrStates/vSetDefaultNrMoments`, out-of-bounds writes for moment counts $\geq \text{MAXMOMENTE}$, an out-of-bounds read for large `lMoment` arguments to `dGetDK`, and a fast-path/full-path interaction in `dGetDK` that could dereference a null moment matrix or double-count higher moments after a `vSetStartTime/vSetStopTime` invalidation. All fixes are numerically neutral; full details and regression evidence (CPP20 test suite, licensed ANNUITYLV valuation, Rust cross-validation) are in `Omarkov_v3s2/change_report.pdf`.

Token syntax

The `MODULES` field is a **semicolon-separated** list of case-sensitive tokens, e.g. `ANNMOD; CLIENT3`. Token parsing happens inside `iSetLicense()` after the Ed25519 signature is verified.

Token	Meaning
ALL	Unlock every module and feature
DK	Unlock <code>dGetDK</code> for base MARKOVLV family
CF	Unlock <code>dGetCF</code> for base MARKOVLV family
Widow2S	Unlock WIDDOW2SPEC <code>dGetDK</code> / <code>dGetCF</code>
REF	Unlock REF_DAV97I / REF_DAV22I
ANNMOD	Unlock ANNMOD own methods; implies DK, CF, REF
GLMOD	Unlock GLMOD own methods; implies DK, CF, REF
CLIENT1 ... CLIENT10	Unlock client slot <i>n</i> ; implies DK, CF, REF

Implication table

Higher-level module tokens automatically grant access to the base features they depend on internally. **ANNMOD** and **GLMOD** call `MARKOVLV::dGetDK` and may use **REF_DAV97I/22I** internally, so they imply DK, CF, and REF. **WIDDOW2SPEC** is *not* implied by any other token – it requires `Widow2S` or `ALL` explicitly.

Token	DK	CF	Widow2S	REF	ANNMOD	GLMOD	CLIENTn
ALL	Y	Y	Y	Y	Y	Y	all
DK	Y						
CF		Y					
Widow2S			Y				
REF				Y			
ANNMOD	Y	Y		Y	Y		
GLMOD	Y	Y		Y		Y	
CLIENTn	Y	Y		Y			<i>n</i>

Query functions

After a successful `iSetLicense()` call the following functions report per-module access:

Function	Returns 1 when
<code>iLicenseAllowsDK()</code>	DK, ANNMOD, GLMOD, any CLIENT _n , or ALL.
<code>iLicenseAllowsCF()</code>	CF, ANNMOD, GLMOD, any CLIENT _n , or ALL.
<code>iLicenseAllowsWidow2S()</code>	Widow2S or ALL only .
<code>iLicenseAllowsREF()</code>	REF, ANNMOD, GLMOD, any CLIENT _n , or ALL.
<code>iLicenseAllowsANNMOD()</code>	ANNMOD or ALL.
<code>iLicenseAllowsGLMOD()</code>	GLMOD or ALL.
<code>iLicenseAllowsClient(n)</code>	CLIENT _n or ALL ($n = 1 \dots 10$).

Guard macros

Each class inserts one guard macro at the top of its result methods. If the check fails, a one-time `stderr` message is printed and the function returns the supplied default value.

Macro	Insert in
<code>LIC_GUARD_DK(rv)</code>	MARKOVLV::dGetDK
<code>LIC_GUARD_CF(rv)</code>	MARKOVLV::dGetCF
<code>LIC_GUARD_WIDOW2S(rv)</code>	WIDDOW2SPEC::dGetDK, dGetCF
<code>LIC_GUARD_REF(rv)</code>	REF_DAV97I::dGetDK/CF, REF_DAV22I::dGetDK/CF
<code>LIC_GUARD_ANNMOD(rv)</code>	ANNMOD own methods
<code>LIC_GUARD_GLMOD(rv)</code>	GLMOD own methods
<code>LIC_GUARD_CLIENT(n, rv)</code>	Client- <i>n</i> -specific methods

When `WITH_LICENSE_CHECK` is not defined (development builds) all macros expand to nothing.

Issuing a license

Use `genlicense.py` with the private key to generate a signed license string. Set the `MODULES` field to the desired semicolon-separated token list:

```
python genlicense.py sign \
  --privkey markovlv_private.key \
  --customer "Muster AG" \
  --expiry 2027-12-31 \
  --modules "ANNMOD;CLIENT3"
```

The resulting base64 string is handed to `iSetLicense()` at runtime. A license with `MODULES=ANNMOD;CLIENT3` grants **ANNMOD** full access, client-slot-3 functionality, and the implicitly required DK, CF, and REF features. It does *not* grant **WIDDOW2SPEC** (Widow2S) or **GLMOD**.

Example: checking module access from Python

```
import markovlv as ml
ml.iSetLicense(open("my.lic").read().strip())

print(ml.iLicenseAllowsDK())           # 1 if DK/ANNMOD/GLMOD/CLIENT* licensed
print(ml.iLicenseAllowsANNMOD())       # 1 only if ANNMOD or ALL
print(ml.iLicenseAllowsClient(3))     # 1 only if CLIENT3 or ALL
```

26 Build and Installation

The Python extension is compiled with SWIG and a standard `setup.py`:

```
cd /path/to/MarkovLib/CPP20
python setup.py build_ext --inplace
```

This produces `_markovlv.cpython-3xx-<platform>.so` and `markovlv.py` in the `CPP20` directory.

Requirements

- C++20-capable compiler (GCC ≥ 10 , Clang ≥ 12 , MSVC 2019+)
- SWIG ≥ 4.0
- Python ≥ 3.9
- (optional) `openpyxl` for the Excel converter

Post-installation integrity check

After installing `markovlv` it is strongly recommended to run the bundled integrity check to verify that the engine produces correct results on your platform. The check is self-contained: all reference values were computed on the reference build and are shipped as JSON files in `IntegrityCheck/RefJson/`.

What is checked. For each of the 139 test cases (13 module types $\times$ 10 parameter sets, plus 9 random Markov stress cases) the script performs three independent verifications:

1. **Rebuild** — re-creates the model with the installed `markovlv` and exports a fresh JSON snapshot to `LocalResults/`.
2. **Compare** — loads both the freshly generated JSON and the reference JSON from `RefJson/` and compares every DK and CF value field by field (absolute tolerance 10^{-10}).
3. **Round-trip** — loads the raw `pij/disc/pre/post` tables from the reference JSON into a plain `MARKOVLV` object (bypassing all higher-level wrapper classes), recomputes DK and CF from scratch via Thiele backward induction, and compares against the stored values. This step verifies the `omarkov` core engine independently of every module wrapper (`ANNUNITYLV`, `DISABILITYLV`, `REF_DAV97I`, ...).

Modules covered: [ANNUITYLV](#), [ANNUITYLV2](#), [CAPITALLV](#), [CAPITAL2LV](#), [CAPITALNLV](#), [ANNUITYNLV](#), [WIDDOWLV](#), [WIDDOW2SPEC](#), [DISABILITYLV](#), [MULTDECRLV](#), [MARKOVLV](#), [REF_DAV97I](#), [REF_DAV22I](#).

Running the check.

```
cd Manual/IntegrityCheck
python3 checkIntegrity.py
```

The script can also be invoked from any other working directory:

```
python3 /path/to/IntegrityCheck/checkIntegrity.py
```

LocalResults/ and the LaTeX report integrity_report_<date>.tex are written next to the script. Expected output on a correct installation:

```
checkIntegrity.py -- tolerance 1e-10
  ANNUITYLV          ..... 10/10 PASS
  ANNUITYLV2         ..... 10/10 PASS
  ...
  OMARKOV_STRESS     ..... 9/9 PASS
=====
Cases       : 139          Passed : 139          Failed : 0
Overall    : PASS
=====
```

A failed case is reported with the module name, label, and the step that failed (BUILD, DIFF, RT, or RT-ERR), together with the number of mismatching entries and the maximum absolute error found.

File layout.

File	Purpose
checkIntegrity.py	Main script — the only file you run
_cases.py	Test-case parameter definitions
importjson.py	JSON→MARKOVLV loader (standard-library only)
RefJson/	139 reference JSON snapshots

Version history

Version	Date	Changes
3.9.2	Jun 2026	omarkov_v2S2_CPP20.cpp stability fixes (review 2026-06-11): vSetNrState
3.8.4	May 2026	ANNUITYNLV / CAPITALNLV: N-life generalisations of ANNUITYLV2 / CAPITAL2LV;
3.8.3	May 2026	CAPITAL2LV: new two-life capital insurance class (capital2lv_CPP20.h/cpp);
3.8.2	May 2026	WIDDOW2SPEC TZ: dGetRP/dGetSP/dGetRegP; dGetRegP added to all classe
3.6.4	May 2026	Per-module MODULES: ANNMOD, GLMOD, CLIENT1–10 tokens
3.6.3	May 2026	MSVC build fixes; const char*;static constexpr
3.6.2	May 2026	DISABILITYLV V0.62: sub-12-month Wartefrist
3.6.1	Apr 2026	DISABILITYLV V0.61: inception-year WP timing fix
3.5.x	2025	REF_DAV22I, REF_DAV97I added

27 Other Language Implementations

In addition to the reference C++ library (exposed to Python via the SWIG extension `_markovlv`) and the pure-Python mirror `markovlib.py`, the core Markov engine and all actuarial classes have been translated into two further languages:

- **Rust** — single-file library `markovlib.rs` in `MarkovLib/OtherLanguages/`.
- **Java** — Maven project `markovlib_java/` in `MarkovLib/OtherLanguages/markovlib_java` package `markovlib`.

Each translation implements the same mathematical algorithms (Thiele backward recursion, cash-flow projection, risk/saving premium decomposition) as the C++ reference. The tables below compare every public interface method of the reference C++ API to its equivalent in each target language.

Class availability across languages

Class	C++/Python (SWIG)	Python (<code>markovlib.py</code>)	Rust	Java
MARKOVLV	✓	✓	<code>MarkovLv</code>	<code>MarkovLv</code>
ANNUITYLV	✓	✓	<code>AnnuityLv</code>	<code>AnnuityLv</code>
ANNUITYLV2	✓	✓	<code>AnnuityLv2</code>	<code>AnnuityLv2</code>
CAPITALLV	✓	✓	<code>CapitalLv</code>	<code>CapitalLv</code>
WIDDOWLV	✓	✓	<code>WiddowLv</code>	<code>WiddowLv</code>
GLMOD	✓	✓	<code>GlMod</code>	<code>GlMod</code>
ANNMOD	✓	✓	<code>AnnMod</code>	<code>AnnMod</code>
DISABILITYLV	✓	✓	—	—
MULTDECRLV	✓	✓	—	—
CAPITAL2LV	✓	✓	—	—
ANNUITYNLV	✓	✓	—	—

Type aliases in Rust (`pub use MarkovLv as MARKOVLV; etc.`) allow the upper-case names to be used interchangeably.

Python (`markovlib.py`)

The pure-Python translation `markovlib.py` exposes an API that is **identical** to the C++/SWIG extension: class names, method names, argument order, and return types all match the reference exactly. The following points summarise the minor behavioural differences.

- **Constructor keyword arguments.** **MARKOVLV** accepts the parameters as keyword arguments (`max_times`, `max_states`, `nr_moments`) in addition to positional form.
- **No license gate.** `dGetDK` and `dGetCF` are never blocked; the license functions `iSetLicense`, `iLicenseValid`, `pcLicenseInfo` exist but are no-ops.
- **Table loading.** Tables are loaded from `tables.json` via `iAddTables(path)` exactly as in C++.
- **No simulation.** `vGenerateTrajectory` / `vGetState` / `dGetRandCF` are not implemented in the pure-Python version.

- **JSON export.** Right after `vPrintTeXFileName`, `markovlib.py` adds the free function `vExportJson` along with `lGetDKCalculated` and `lGetCFCalculated`. The output matches the C++ exporter (Section 5) field for field — same block layout, same nil filtering, same “export only what is already cached” rule — so files produced by either engine are interchangeable. Internally the base-class methods are called explicitly, e.g. `MARKOVLV.dGetDK(m, ...)` rather than `m.dGetDK(...)`, so that **CAPITALLV** and other subclasses export correctly.

All other methods listed in Sections 27–26 apply without change.

Rust (`markovlib.rs`)

In Rust all method names follow `snake_case` convention. The mapping rule is mechanical: prefix letters (`d`, `v`, `i`, `l`) are retained, camelCase humps are replaced by underscores, and the `mut` receiver is required on all setters.

Constructors

C++ / Python (SWIG)	Rust	Notes
<code>MARKOVLV(t, s, m)</code>	<code>MarkovLv::new(t, s, m)</code>	
<code>ANNUITYLV()</code>	<code>AnnuityLv::new(mt, pre)</code>	default: (250, 1.0)
<code>ANNUITYLV2()</code>	<code>AnnuityLv2::new(mt, pre)</code>	default: (250, 1.0)
<code>CAPITALLV()</code>	<code>CapitalLv::new(mt)</code>	default: 250
<code>WIDDOWLV()</code>	<code>WiddowLv::new(mt)</code>	default: 250
<code>GlMod()</code>	<code>GlMod::new()</code>	
<code>AnnMod()</code>	<code>AnnMod::new()</code>	

MARKOVLV / MarkovLv

C++ / Python	Rust	Notes
<code>vSetNrStates(n)</code>	<code>v_set_nr_states(n)</code>	
<code>vSetStartTime(t)</code>	<code>v_set_start_time(t)</code>	
<code>vSetStopTime(t)</code>	<code>v_set_stop_time(t)</code>	
<code>vReset()</code>	<code>v_reset()</code>	
<code>lSetFolgezustand(f, t)</code>	<code>l_set_folgezustand(f, t)</code>	
<code>dSetPij(t, i, j, p)</code>	<code>d_set_pij(t, i, j, p)</code>	
<code>dSetPre(t, i, j, a)</code>	<code>d_set_pre(t, i, j, a)</code>	
<code>dSetPost(t, i, j, a)</code>	<code>d_set_post(t, i, j, a)</code>	
<code>dSetDisc(t, i, j, v)</code>	<code>d_set_disc(t, i, j, v)</code>	
<code>dGetDK(t, s, m)</code>	<code>d_get_dk(t, s, m)</code>	
<code>dGetCF(t, init, s)</code>	<code>d_get_cf(t, init, s)</code>	
<code>dGetRP(t, s)</code>	<code>d_get_rp(t, s)</code>	
<code>dGetSP(t, s)</code>	<code>d_get_sp(t, s)</code>	
<code>dGetRegP(t, s)</code>	<code>d_get_reg_p(t, s)</code>	&self only
<code>vSetInitState(s)</code>	<code>v_set_init_state(s)</code>	simulation
<code>vNewSeed(seed)</code>	<code>v_new_seed(seed)</code>	simulation

C++ / Python	Rust	Notes
vGenerateTrajectory()	v_generate_trajectory()	simulation
vGetState(t)	v_get_state(t)	simulation
dGetRandCF(t)	d_get_rand_cf(t)	simulation
<i>Distribution (CDF) — §17</i>		
distrBounds()	distr_bounds()	returns tuple
vSetDistrGrid(lo,hi,n)	v_set_distr_grid(lo,hi,n)	
vAutoDistrGrid(n)	v_auto_distr_grid(n)	
vSetDistrBoundary(s,thr)	v_set_distr_boundary(s,thr)	
dGetDKDistr(t,s,x)	d_get_dk_distr(t,s,x)	
dGetDKQuantile(t,s,p)	d_get_dk_quantile(t,s,p)	
dGetDKDistrBucket(t,s,b)	d_get_dk_distr_bucket(t,s,b)	
lBucketNr(x)	l_bucket_nr(x)	
dValueOfBucket(b)	d_value_of_bucket(b)	
dGetDistrLow()	d_get_distr_low()	
dGetDistrHigh()	d_get_distr_high()	
lGetNrBuckets()	l_get_nr_buckets()	
dGetBucketWidth()	d_get_bucket_width()	
lGetDKDistrCalculated()	l_get_dk_distr_calculated()	

ANNUITYLV / AnnuityLv

C++ / Python	Rust	Notes
iSetTable(id)	i_set_table(id)	
vSetStartTime(t)	v_set_start_time(t)	
vSetStopTime(t)	v_set_stop_time(t)	
vSetSAge(t)	v_set_s_age(t)	
vSetG(t)	v_set_g(t)	
vSetMaxProj(n)	v_set_max_proj(n)	
dSetQx(t,q)	d_set_qx(t,q)	
dSetFx(t,f)	d_set_fx(t,f)	
dSetSx(t,s)	d_set_sx(t,s)	
dSetDisc(t,v)	d_set_disc(t,v)	
dSetBaseYear(y)	d_set_base_year(y)	
dSetActualYear(y)	d_set_actual_year(y)	
dSetPre(p)	d_set_pre_frac(p)	name differs
dSetRelativeQxForTime(t,r)	d_set_relative_qx_for_time(t,r)	
vLeistReset()	v_leist_reset()	
vSetLeistLinear(t0,t1,s,i)	v_set_leist_linear(t0,t1,s,i)	
vSetLeistExp(t0,t1,s,f)	v_set_leist_exp(t0,t1,s,f)	
dGetDK(t)	d_get_dk(t)	
dGetCF(t)	d_get_cf(t)	
dGetQx(t,year)	d_get_qx(t,year)	
dGetTpx(k)	d_get_tpx(k)	

C++ / Python	Rust	Notes
dGetTqx(k)	d_get_tqx(k)	

ANNUITYLV2 / AnnuityLv2

C++ / Python	Rust	Notes
vSetStartTime(t)	v_set_start_time(t)	
vSetStopTime(t)	v_set_stop_time(t)	
vSetSAge1(t)	v_set_s_age1(t)	
vSetSAge2(t)	v_set_s_age2(t)	
dSetQx1(t,q)	d_set_qx1(t,q)	
dSetFx1(t,f)	d_set_fx1(t,f)	
dSetQx2(t,q)	d_set_qx2(t,q)	
dSetFx2(t,f)	d_set_fx2(t,f)	
dSetDisc(t,v)	d_set_disc(t,v)	
dSetBaseYear(y)	d_set_base_year(y)	
dSetActualYear(y)	d_set_actual_year(y)	
dSetY_Minus_X(y,x)	d_set_y_minus_x(y,x)	
dSetBenefit(s,v)	d_set_benefit(s,v)	
dSetPre(p)	d_set_pre_frac(p)	name differs
vLeistReset()	v_leist_reset()	
vSetLeistLinear(t0,t1,s,i)	v_set_leist_linear(t0,t1,s,i)	
dGetDK(t,state)	d_get_dk(t,state)	
dGetCF(t)	d_get_cf(t)	
dGetQx1(t,year)	d_get_qx1(t,year)	
dGetQx2(t,year)	d_get_qx2(t,year)	

CAPITALLV / CapitalLv

C++ / Python	Rust	Notes
iSetTable(id)	i_set_table(id)	
vSetStartTime(t)	v_set_start_time(t)	
vSetStopTime(t)	v_set_stop_time(t)	
vSetSurvival(t,v)	v_set_survival(t,v)	
vSetSurvivalGen(t,v)	v_set_survival_gen(t,v)	
vSetDeath(v)	v_set_death(v)	
vSetDeathGen(t,v)	v_set_death_gen(t,v)	
vSetPremium(v)	v_set_premium(v)	
vSetPremiumGen(t,v)	v_set_premium_gen(t,v)	
dSetQx(t,q)	d_set_qx(t,q)	
dSetFx(t,f)	d_set_fx(t,f)	
dSetDisc(t,v)	d_set_disc(t,v)	
dSetBaseYear(y)	d_set_base_year(y)	
dSetActualYear(y)	d_set_actual_year(y)	

C++ / Python	Rust	Notes
dSetSolaCapitalInMR(t, v)	d_set_sola_capital(t, v)	EV; name shortened
dGetDK(t)	d_get_dk(t)	simplified EV
dGetCF(t)	d_get_cf(t)	
dGetEV(t)	d_get_ev(t)	
dGetQx(t, year)	d_get_qx(t, year)	

Note: The full set of embedded-value setters (dSetQx2Level, dSetRDR, dSetSurrenderPenaltyInMR, dSetSHMarginInMR, dSetInvReturn, dSetSx2) available in C++ are not implemented in Rust; dGetEV in Rust returns $V_t - \text{SolaCapital}(t)$ as a simplified approximation.

WIDDOWLv / WiddowLv

C++ / Python	Rust	Notes
vSetStartTime(t)	v_set_start_time(t)	insured survivor
vSetStopTime(t)	v_set_stop_time(t)	
dSetQx(t, q)	d_set_qx(t, q)	
dSetQy(t, q)	d_set_qy(t, q)	
dSetFx(t, f)	d_set_fx(t, f)	widow probability
dSetFy(t, f)	d_set_fy(t, f)	
dSetHx(t, h)	d_set_hx(t, h)	
dSetYx(t, a)	d_set_yx(t, a)	
dSetDisc(t, v)	d_set_disc(t, v)	widow's age per time-step
—	d_set_disc_all(v)	Rust extra: set all slots
dSetBaseYear(y)	d_set_base_year(y)	direct field access
dSetActualYear(y)	d_set_actual_year(y)	
dSetPre(p)	d_pre (public field)	
vLeistReset()	(benefit defaults to 1.0)	no explicit method
vSetLeistLinear(t_0, t_1, s, i)	v_set_leist_linear(t_0, t_1, s, i)	
dGetDK(t)	d_get_dk(t)	
dGetCF(t)	d_get_cf(t)	
dGetQx(t, year)	d_get_qx(t, year)	

Portfolio modules: GLMOD / ANNMOD

Both modules are implemented in Rust as GlMod / AnnMod with the same snake_case convention. The portfolio-management methods are:

C++ / Python	Rust	Notes
GLMOD / GIMod		
<code>vAddAnnuity(...)</code>	<code>v_add_annuity(...)</code>	
<code>vAddEndowment(...)</code>	<code>v_add_endowment(...)</code>	
<code>vAddWidow(...)</code>	<code>v_add_widow(...)</code>	
<code>dGetStatDK()</code>	<code>d_get_stat_dk()</code>	
<code>dGetFVDK()</code>	<code>d_get_fv_dk()</code>	
<code>vStress(type, amount)</code>	<code>v_stress(type, amount)</code>	
ANNMOD / AnnMod		
<code>vAddAnnuity1(...)</code>	<code>v_add_annuity1(...)</code>	
<code>vAddAnnuity0(...)</code>	<code>v_add_annuity0(...)</code>	
<code>vAddAnnuity2XY(...)</code>	<code>v_add_annuity2_xy(...)</code>	
<code>vAddAnnuity2XYBar(...)</code>	<code>v_add_annuity2_xy_bar(...)</code>	
<code>vAddAnnuity2XToY(...)</code>	<code>v_add_annuity2_x_to_y(...)</code>	
<code>vAddAnnuity2YToX(...)</code>	<code>v_add_annuity2_y_to_x(...)</code>	
<code>dGetDK(t)</code>	<code>d_get_dk(t)</code>	
<code>dGetStatDK()</code>	<code>d_get_stat_dk()</code>	
<code>dGetFVDK()</code>	<code>d_get_fv_dk()</code>	

Remark 27.1 (JSON export in the `markovlib_rs` crate). The actively-maintained Rust crate `OtherLanguages/markovlib_rs` — distinct from the single-file reference translation `markovlib.rs` tabulated above — extends `MarkovLvCore` with an export method, `v_export_json`, and a matching pair of cache-size getters, `l_get_dk_calculated` and `l_get_cf_calculated`. Together these produce JSON that is byte-for-byte identical in schema and semantics to the C++ and Python exporters (compare Sections 5 and 27). The exporter itself is a plain `String` builder — no `serde_json` dependency — using `{:.17e}` formatting so that IEEE-754 doubles round-trip exactly. The file `src/python.rs` re-exposes all three methods, under their camelCase C++ names, on `PyMarkovLv` and `PyCapitalLv` via `PyO3`, so the Python call sites read exactly like the `markovlib.py` ones above. See the dedicated *JSON Export & Import* manual (`json_export.pdf`, section “JSON Export in the Other-Language Implementations”) for the full description, the `maturin` build instructions, and the cross-validation results.

Java (`markovlib_java`)

The Java translation uses the same Hungarian-notation method names as C++ with two systematic exceptions:

- `dSetPre(double)` on **ANNUITYLV** / **ANNUITYLV2** (which sets the pre-numerando *fraction*) is renamed `dSetPreFrac(double)` to distinguish it from the four-argument `dSetPre(t, i, j, value)` inherited from **MARKOVLV**.
- `dSetY_Minus_X(yAge, xAge)` becomes `dSetYMinusX(yAge, xAge)` (the underscore in the original C++ name is dropped).

All other method names are **identical** to the C++ reference. The tables below list only the methods that differ, followed by a complete per-class summary for convenience.

Methods that differ from C++ reference



C++ / Python (SWIG)	Java	Affected class
dSetPre(double p)	dSetPreFrac(double p)	AnnuityLv, AnnuityLv2
dSetY_Minus_X(y, x)	dSetYMinusX(y, x)	AnnuityLv2

MARKOVLV / MarkovLv

All methods identical to C++. Java types: int for long params, double preserved.

C++ / Python	Java	Notes
vSetNrStates(n)	vSetNrStates(n)	
vSetStartTime(t)	vSetStartTime(t)	
vSetStopTime(t)	vSetStopTime(t)	
vReset()	vReset()	
lSetFolgezustand(f, t)	lSetFolgezustand(f, t)	
dSetPij(t, i, j, p)	dSetPij(t, i, j, p)	
dSetPre(t, i, j, a)	dSetPre(t, i, j, a)	4-arg form unchanged
dSetPost(t, i, j, a)	dSetPost(t, i, j, a)	
dSetDisc(t, i, j, v)	dSetDisc(t, i, j, v)	
dGetDK(t, s, m)	dGetDK(t, s, m)	
dGetCF(t, init, s)	dGetCF(t, init, s)	
dGetRP(t, s)	dGetRP(t, s)	
dGetSP(t, s)	dGetSP(t, s)	
dGetRegP(t, s)	dGetRegP(t, s)	
vSetInitState(s)	vSetInitState(s)	simulation
vNewSeed(seed)	vNewSeed(seed)	simulation
vGenerateTrajectory()	vGenerateTrajectory()	simulation
vGetState(t)	vGetState(t)	simulation
dGetRandCF(t)	dGetRandCF(t)	simulation
dGetMeanCF(t, s, n)	dGetMeanCF(t, s, n)	Java extra
dGetMeanDK(t, s, n)	dGetMeanDK(t, s, n)	Java extra

ANNUITYLV / AnnuityLv

C++ / Python	Java	Notes
iSetTable(id)	iSetTable(id)	
vSetStartTime(t)	vSetStartTime(t)	
vSetStopTime(t)	vSetStopTime(t)	
vSetSAge(t)	vSetSAge(t)	
vSetG(t)	vSetG(t)	
vSetMaxProj(n)	vSetMaxProj(n)	
dSetQx(t,q)	dSetQx(t,q)	
dSetFx(t,f)	dSetFx(t,f)	
dSetSx(t,s)	dSetSx(t,s)	
dSetDisc(t,v)	dSetDisc(t,v)	
dSetBaseYear(y)	dSetBaseYear(y)	
dSetActualYear(y)	dSetActualYear(y)	
dSetPre(p)	dSetPreFrac(p)	name differs
dSetRelativeQxForTime(t,r)	dSetRelativeQxForTime(t,r)	
vLeistReset()	vLeistReset()	
vSetLeistLinear(t0,t1,s,i)	vSetLeistLinear(t0,t1,s,i)	
vSetLeistExp(t0,t1,s,f)	vSetLeistExp(t0,t1,s,f)	
dGetDK(t)	dGetDK(t)	
dGetCF(t)	dGetCF(t)	
dGetQx(t,year)	dGetQx(t,year)	
dGetTpx(k)	dGetTpx(k)	
dGetTqx(k)	dGetTqx(k)	

ANNUITYLV2 / AnnuityLv2

C++ / Python	Java	Notes
iSetTable1(id)	iSetTable1(id)	
iSetTable2(id)	iSetTable2(id)	
vSetStartTime(t)	vSetStartTime(t)	
vSetStopTime(t)	vSetStopTime(t)	
vSetSAge1(t)	vSetSAge1(t)	
vSetSAge2(t)	vSetSAge2(t)	
dSetQx1(t, q)	dSetQx1(t, q)	
dSetFx1(t, f)	dSetFx1(t, f)	
dSetQx2(t, q)	dSetQx2(t, q)	
dSetFx2(t, f)	dSetFx2(t, f)	
dSetDisc(t, v)	dSetDisc(t, v)	
dSetBaseYear(y)	dSetBaseYear(y)	
dSetActualYear(y)	dSetActualYear(y)	
dSetY_Minus_X(y, x)	dSetYMinusX(y, x)	name differs
dSetBenefit(s, v)	dSetBenefit(s, v)	
dSetPre(p)	dSetPreFrac(p)	name differs
vLeistReset()	vLeistReset()	
vSetLeistLinear(t0, t1, s, i)	vSetLeistLinear(t0, t1, s, i)	
dGetDK(t, state)	dGetDK(t, state)	
dGetCF(t)	dGetCF(t)	
dGetQx1(t, year)	dGetQx1(t, year)	
dGetQx2(t, year)	dGetQx2(t, year)	

CAPITALIV / CapitalLv

C++ / Python	Java	Notes
iSetTable(id)	iSetTable(id)	
vSetStartTime(t)	vSetStartTime(t)	
vSetStopTime(t)	vSetStopTime(t)	
vSetSurvival(t, v)	vSetSurvival(t, v)	
vSetSurvivalGen(t, v)	vSetSurvivalGen(t, v)	
vSetDeath(v)	vSetDeath(v)	
vSetDeathGen(t, v)	vSetDeathGen(t, v)	
vSetPremium(v)	vSetPremium(v)	
vSetPremiumGen(t, v)	vSetPremiumGen(t, v)	
dSetQx(t, q)	dSetQx(t, q)	
dSetFx(t, f)	dSetFx(t, f)	
dSetDisc(t, v)	dSetDisc(t, v)	
dSetBaseYear(y)	dSetBaseYear(y)	
dSetActualYear(y)	dSetActualYear(y)	
dSetSolaCapitalInMR(t, v)	dSetSolaCapital(t, v)	name shortened
dGetDK(t)	dGetDK(t)	
dGetCF(t)	dGetCF(t)	
dGetEV(t)	dGetEV(t)	simplified EV
dGetQx(t, year)	dGetQx(t, year)	

Note: As in Rust, the full EV setters (dSetQx2Level, dSetRDR, dSetSurrenderPenaltyInMR,

`dSetSHMarginInMR`, `dSetInvReturn`, `dSetSx2`) are not implemented in Java; `dGetEV` returns $V_t - \text{SolaCapital}(t)$.

WIDDOWLv / WiddowLv

C++ / Python	Java	Notes
<code>vSetStartTime(t)</code>	<code>vSetStartTime(t)</code>	
<code>vSetStopTime(t)</code>	<code>vSetStopTime(t)</code>	
<code>dSetQx(t, q)</code>	<code>dSetQx(t, q)</code>	insured survivor
<code>dSetQy(t, q)</code>	<code>dSetQy(t, q)</code>	
<code>dSetFx(t, f)</code>	<code>dSetFx(t, f)</code>	
<code>dSetFy(t, f)</code>	<code>dSetFy(t, f)</code>	
<code>dSetHx(t, h)</code>	<code>dSetHx(t, h)</code>	
<code>dSetYx(t, a)</code>	<code>dSetYx(t, a)</code>	
<code>dSetDisc(t, v)</code>	<code>dSetDisc(t, v)</code>	
<code>dSetDiscAll(v)</code>	<code>dSetDiscAll(v)</code>	Java/Rust extra
<code>dSetBaseYear(y)</code>	<code>dSetBaseYear(y)</code>	
<code>dSetActualYear(y)</code>	<code>dSetActualYear(y)</code>	
<code>vSetLeistLinear(t0, t1, s, i)</code>	<code>vSetLeistLinear(t0, t1, s, i)</code>	
<code>dGetDK(t)</code>	<code>dGetDK(t)</code>	
<code>dGetCF(t)</code>	<code>dGetCF(t)</code>	
<code>dGetQx(t, year)</code>	<code>dGetQx(t, year)</code>	